

Дисциплина: «Введение в глубокое обучение»

Общая информация

Длительность	Три недели
Семестр	2-й семестр
Аттестация	Экзамен

Авторы и преподаватели

Дмитрий Коробченко

- NVIDIA, Deep Learning R&D-инженер.
- Исследователь и разработчик в областях машинного обучения, нейронных сетей, компьютерного зрения и обработки сигналов.
- Занимался задачами, связанными с медицинскими изображениями, распознаванием, обработкой изображений, 3D-анимацией.
- Время от времени читает вводные научно-популярные лекции по Deep Learning.
- Преподаватель Skillfactory по нейронным сетям, AI-разработке, ML и DL.

Константин Некрасов

- АО «Газпромбанк», директор по разработке ML-моделей.
- Ведёт проекты в области машинного обучения для решения прикладных задач финтеха.
- Эксперт в глубоком обучении, в частности в направлениях обработки естественного языка (NLP).
- Разрабатывает и внедряет ML-модели в продуктовую среду.
- Преподаёт дисциплины по Data Science, Data Engineering, Machine Learning, DevOps и Web Development в ведущих вузах страны: МИФИ, МФТИ, ВШЭ.

О дисциплине

Дисциплина посвящена практическому освоению методов глубокого обучения. Вы реализуете полный цикл разработки модели машинного обучения — от загрузки и анализа данных до построения, обучения и применения модели на пользовательских данных. Особое внимание уделяется сравнению подходов и инструментов, используемых в современных фреймворках глубокого обучения, таких как PyTorch и TensorFlow.

К 2026 году глубокое обучение стало ключевой технологией, лежащей в основе большинства интеллектуальных систем — от генеративных моделей и цифровых ассистентов до систем компьютерного зрения и анализа данных в промышленности. Компании ожидают от специалистов не поверхностного знакомства с инструментами, а глубокого понимания принципов работы моделей, умения анализировать их поведение и адаптировать под реальные задачи.

Вы научитесь проводить исследовательский анализ данных, разрабатывать и обучать нейронные сети, оценивать качество моделей и анализировать ошибки. Важная часть дисциплины — интерпретация результатов, выявление ограничений моделей и применение методов улучшения качества, включая регуляризацию. Также рассматриваются вопросы воспроизводимости экспериментов и корректного оформления результатов работы.

Цель дисциплины

Сформировать практические навыки разработки и анализа моделей глубокого обучения, а также умение реализовывать полный цикл решения задачи классификации — от подготовки данных до инференса и интерпретации результатов.

В рамках дисциплины вы:

- научитесь загружать и подготавливать данные для задач компьютерного зрения;
- освоите построение и обучение нейронных сетей в PyTorch и TensorFlow;
- научитесь проводить исследовательский анализ данных (EDA) и визуализацию;
- освоите методы оценки качества моделей и анализа ошибок (включая confusion matrix);
- изучите подходы к сравнению моделей и фреймворков;
- получите опыт улучшения моделей и борьбы с переобучением;
- реализуете инференс модели на пользовательских данных.

Пререквизиты и постреквизиты

Что нужно знать для прохождения дисциплины:

- Основы программирования на Python.
- Базовые знания линейной алгебры и основ машинного обучения.

Что можно будет изучать после прохождения дисциплины:

- Продвинутые методы глубокого обучения.
- Компьютерное зрение.
- Обработка естественного языка.
- Исследовательские направления в машинном обучении.

Содержание дисциплины

№	Раздел	Образовательные результаты модуля
1	Модуль 1. Введение в нейронные сети	► Машинное обучение и типы данных. ► Нейронные сети. ► Как обучить нейронную сеть. ► Задача компьютерного зрения. ► Где взять ядро свёртки. ► Что такое Deep Learning. Типы слоёв. ► Обработка последовательностей. ► От распознавания к синтезу. ► Составлятельные сети
2	Модуль 2. Фреймворки для глубокого обучения	► Граф вычислений. ► MLP. ► TensorFlow. ► Пример обучения. ► Детали градиентного спуска: цепное правило. ► Граф вычисления производных. ► Пример одного нейрона. ► Цепное правило и граф производных
3	Модуль 3. Основы работы с PyTorch	► Введение в PyTorch и тензоры. ► Работа с устройствами (CPU/GPU). ► Autograd: автоматическое дифференцирование. ► Построение моделей: torch.nn.Module. ► Функции активации, функции потерь и регуляризация. ► Загрузка данных: Dataset и DataLoader. ► Оптимизаторы и сохранение моделей. ► Валидация и режимы обучения

Практика

Практическая подготовка по дисциплине происходит в формате синхронных онлайн-занятий и асинхронной самостоятельной работы на образовательной платформе.

Синхронные занятия проводятся в формате практико-ориентированных онлайн-семинаров с разбором прикладных кейсов и архитектурных решений.

Асинхронная практика включает интерактивные задания: тесты (выбор одного или нескольких ответов), задания с вводом ответа и написание кода с автоматической проверкой. Упражнения направлены на освоение инструментов и методов глубокого обучения.

- По завершении дисциплины выполняется семестровое задание — **итоговое тестирование**. Срок выполнения — до трёх недель после окончания курса. Во время сессии задание сдать уже нельзя.
- Итогом дисциплины является **проект (сессионное задание)** — кейс «Классификация рукописных символов с использованием глубокого обучения». Проект сдаётся в течение двух недель после завершения дисциплины либо в период сессии.

Оценивание

Оценка за дисциплину суммируется из оценок за выполнение итогового тестирования (максимальное количество баллов — 50) и сессионного задания, за которое вы также сможете получить максимально 50 баллов.

Шкала перевода баллов в оценку

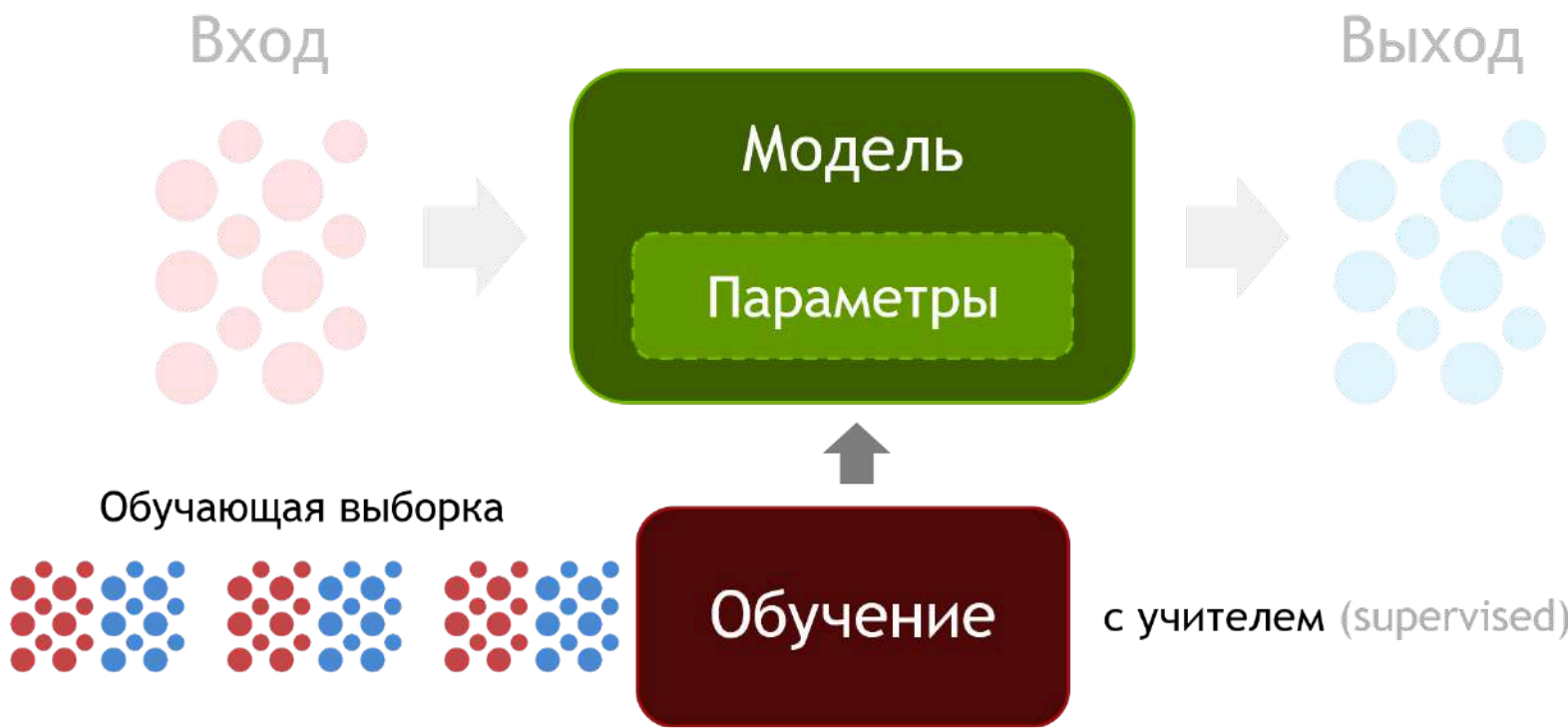
Оценка за дисциплину выставляется по шкале МИФИ

Баллы	Буквенная оценка	Оценка
90–100	A	Отлично
85–89	B	Хорошо
75–84	C	
70–74	D	
65–69	E	Удовлетворительно
60–64		
59 и ниже	F	Неудовлетворительно



Прежде чем переходить к Deep Learning (глубокому обучению), актуализируем информацию по машинному обучению.

Машинное обучение с высоты птичьего полёта можно рассмотреть как некоторое отображение входных данных в выходные данные. Как правило, этим занимается некоторая модель, которая зависит от заданных параметров. Эти параметры характеризуют то, как модель себя ведёт, как она предсказывает выходные значения по входным значениям. Подстройка этих параметров и есть машинное обучение.



Классическое обучение с учителем

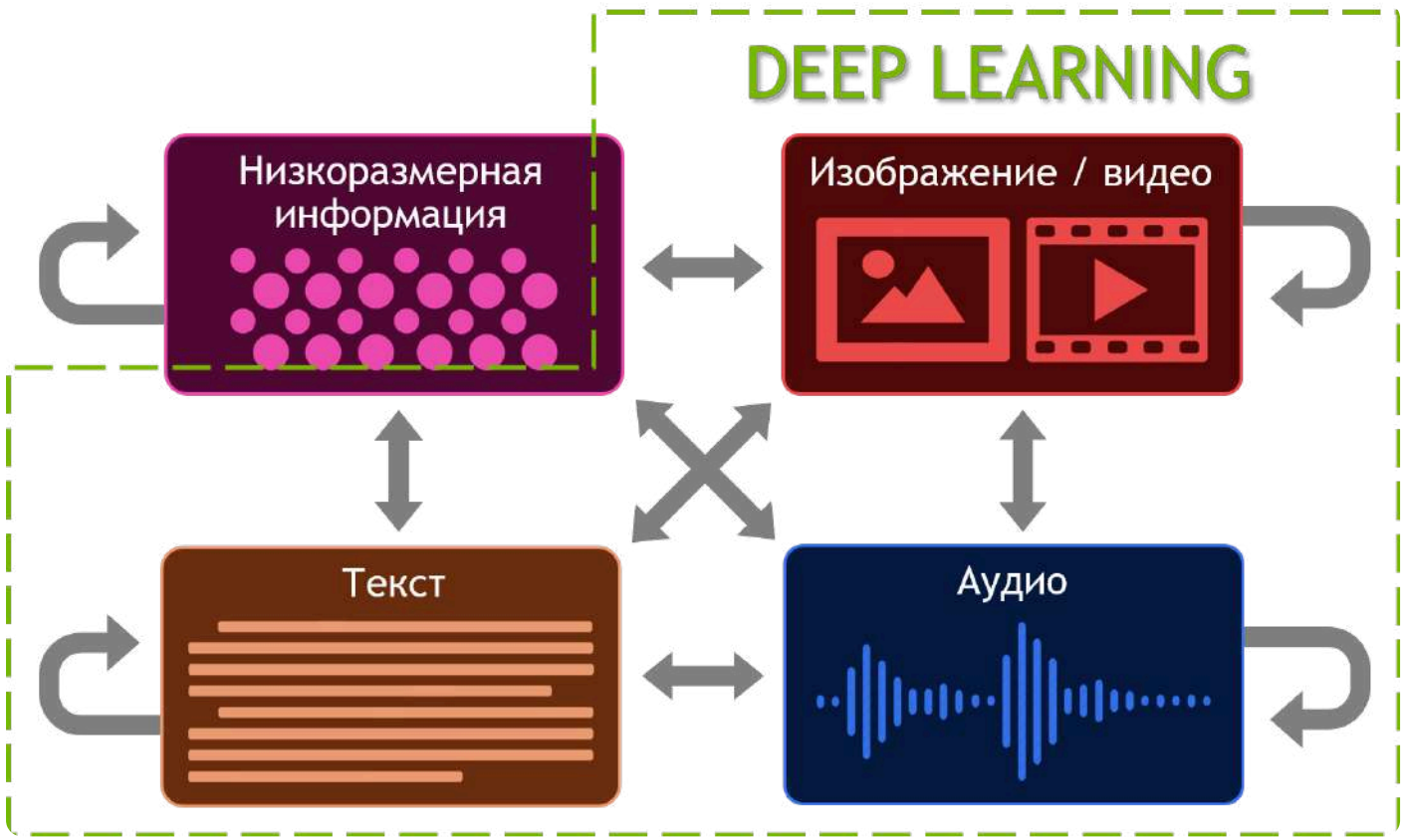
Как правило, в машинном обучении используется обучение с учителем (supervised learning), в котором у нас есть база данных, состоящая из пар объект – ответ. Мы показываем нашей модели эти пары одну за одной и корректируем параметры таким образом, чтобы на новых примерах наши модели предсказали хорошие правильные ответы.

Какие типы данных могут использоваться в таких задачах?

1. Низкоразмерная информация.
Например, это могут быть векторы, отвечающие за характеристики пользователей (рост, вес, возраст и т.д.), и мы хотим сделать их классификацию: то есть отображать низкоразмерный вектор в другой низкоразмерный вектор или скаляр.
2. Изображение или видео.
Это более сложный тип данных. В качестве примера здесь можно рассмотреть задачу компьютерного зрения: отображение визуальной информации в низкоразмерную высокоуровневую информацию.
3. Текст.
Пример: обработка текста и его отображение в качестве низкоразмерной высокоуровневой информации.
4. Аудио и звуковые сигналы.
Всем знакомый пример: распознавание речи и перевод аудио в текст.

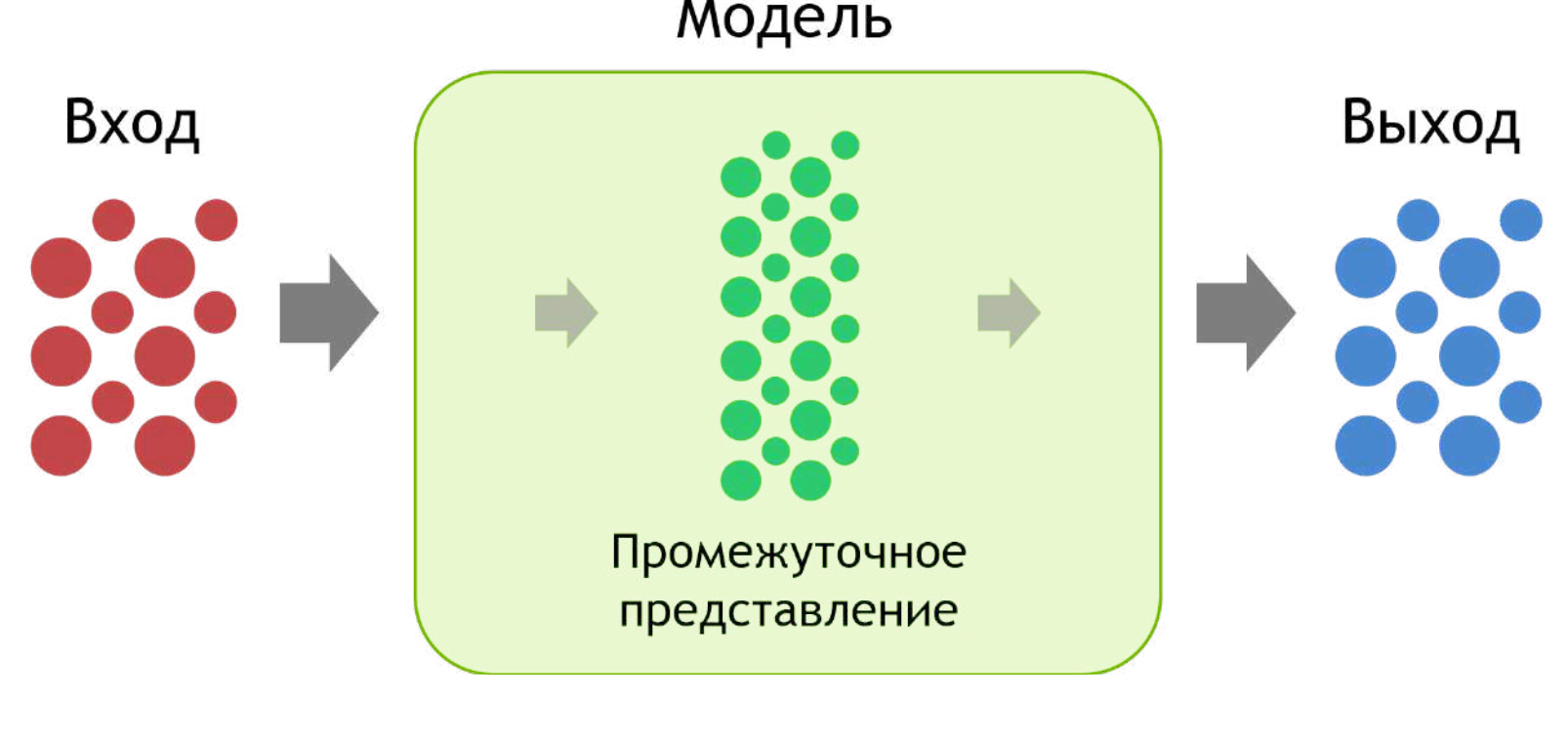
Мы можем делать практически любое отображение из любого типа данных в любой тип данных. При этом на каждое такое отображение есть соответствующая задача и методы её решения. Большинство этих задач наилучшим образом решаются с помощью нейронных сетей или технологии Deep Learning.

Важно понимать, что нейронные сети не всегда являются лучшим решением. Например, отображение низкоуровневой информации в саму себя лучше реализовать с помощью других алгоритмов (случайный лес, градиентный бустинг и др.). Но в случае с высокоразмерными сложными данными (изображения, звуки, тексты) очень хорошо работают именно нейронные сети.





Основная идея нейронных сетей — это обучение представления. Получение выхода из заданного входа начинается с извлечения признаков, когда из входных данных извлекается некоторое промежуточное представление.

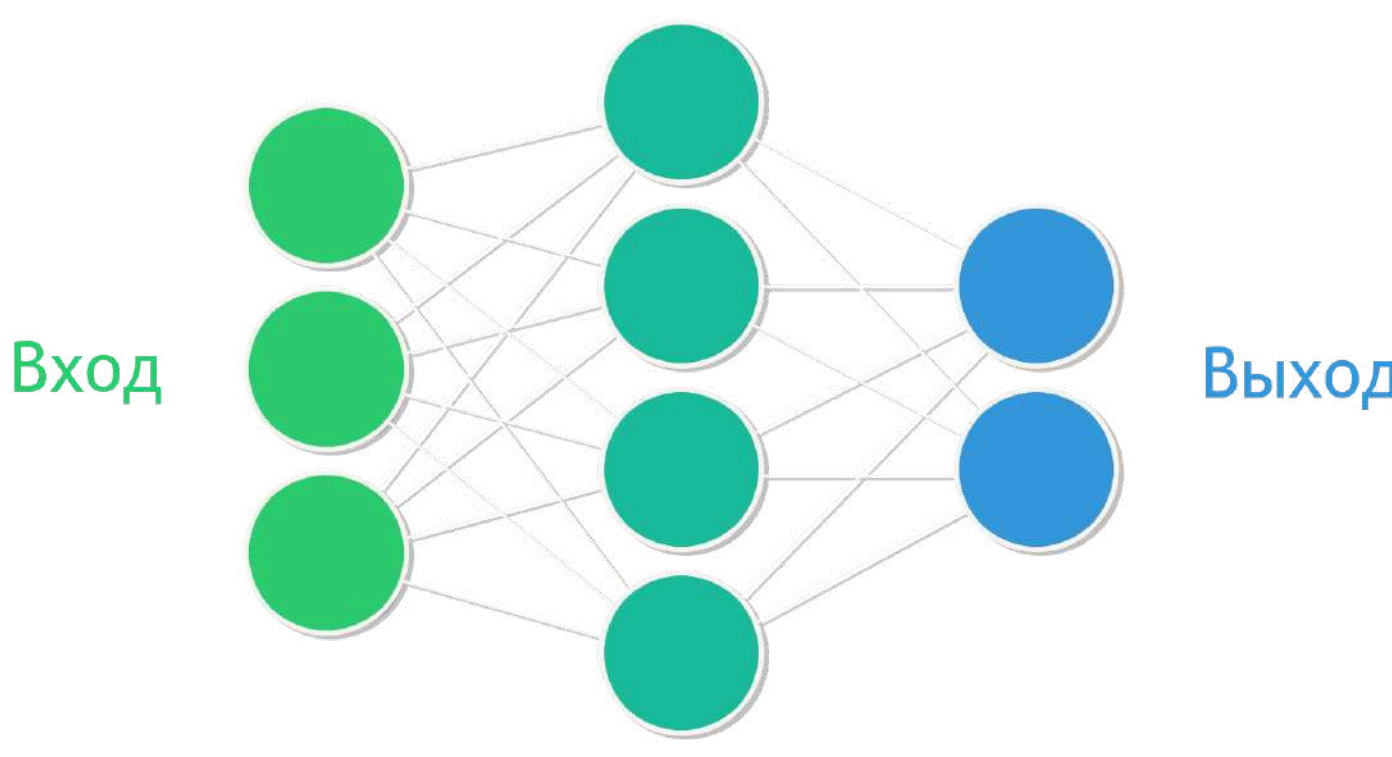


Многослойный перцептрон

Многослойный перцептрон — это простая нейронная сеть, которая состоит из:

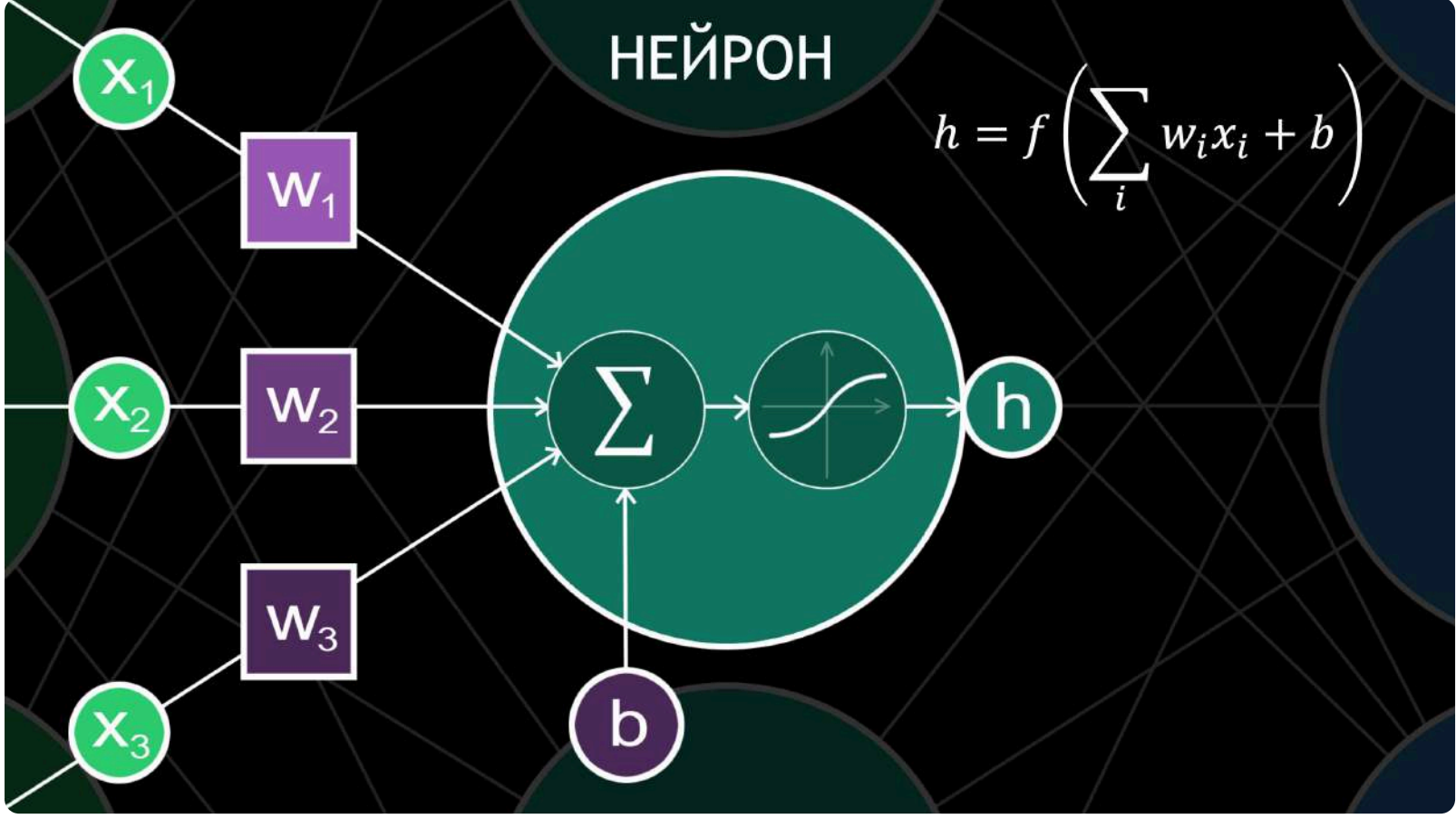
- ▶ входного вектора;
- ▶ выходного вектора;
- ▶ вектора промежуточного представления (скрытый слой).

Вычисление распространяется от входа к выходу, связям между нейронами соответствуют некоторые веса. Поэтому такая сеть является **полносвязной**.



Что в нейроне?

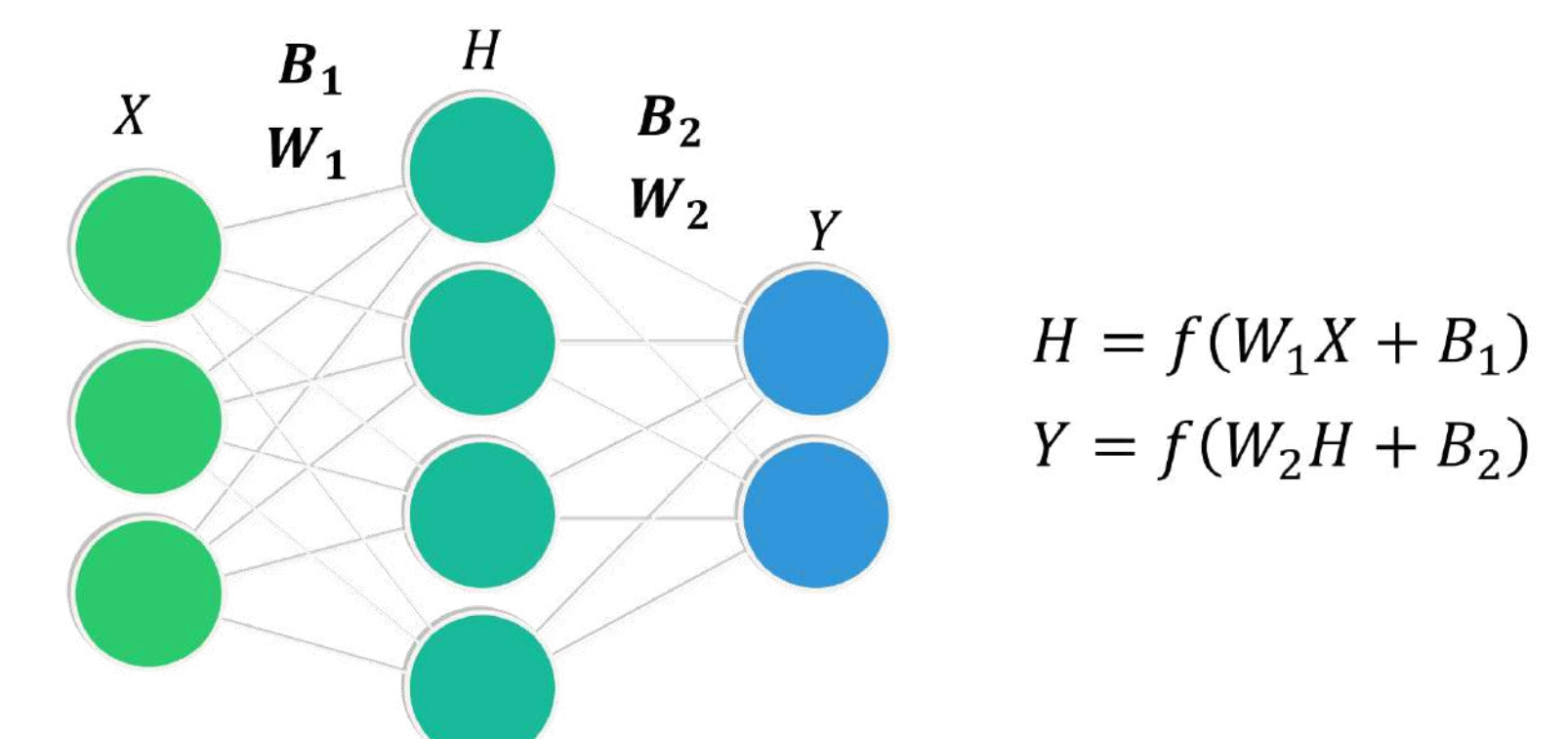
В нейрон входит несколько значений x_1, x_2, x_3 с несколькими связями. Связям соответствуют некоторые коэффициенты w_1, w_2, w_3 (если у нас три входа в нейрон).



Дальше внутри нейрона происходит вычисление двух операций, а точнее композиция линейной и нелинейной операции:

- ▶ В линейной операции мы делаем взвешивание суммы всех входных значений (x_1 умножаем на w_1 , x_2 на w_2 и так далее), всё это суммируем и прибавляем некоторое значение смещения b .
- ▶ В нелинейной операции от полученных на предыдущем шаге значений мы берём нелинейную функцию.

Так вычисляется выходное значение в одном нейроне. Дальше эти нейроны можно уже агрегировать в большую нейронную сеть.



Одному слою такой нейронной сети уже будет соответствовать некоторая **матрица параметров** и некоторый **вектор смещения B**.

Таким образом проходят вычисления в одном слое полносвязной нейронной сети и вычисляется матричное умножение и прибавление вектора и потом взятие поэлементно нелинейности.

Как работают нейроны?

В одном нейроне нейронной сети у нас происходит локальное принятие решения: мы взвешиваем и суммируем входные данные и, основываясь на полученных результатах, локально что-то предсказываем, принимая маленькое решение на каждом шаге. Все полученные решения агрегируются и подаются на следующий слой, и уже новые значения используются для более сложного высокоуровневого принятия решений.

В математическом смысле нейронная сеть — это **универсальный аппроксиматор*** (может аппроксимировать любую функцию при определенных условиях).

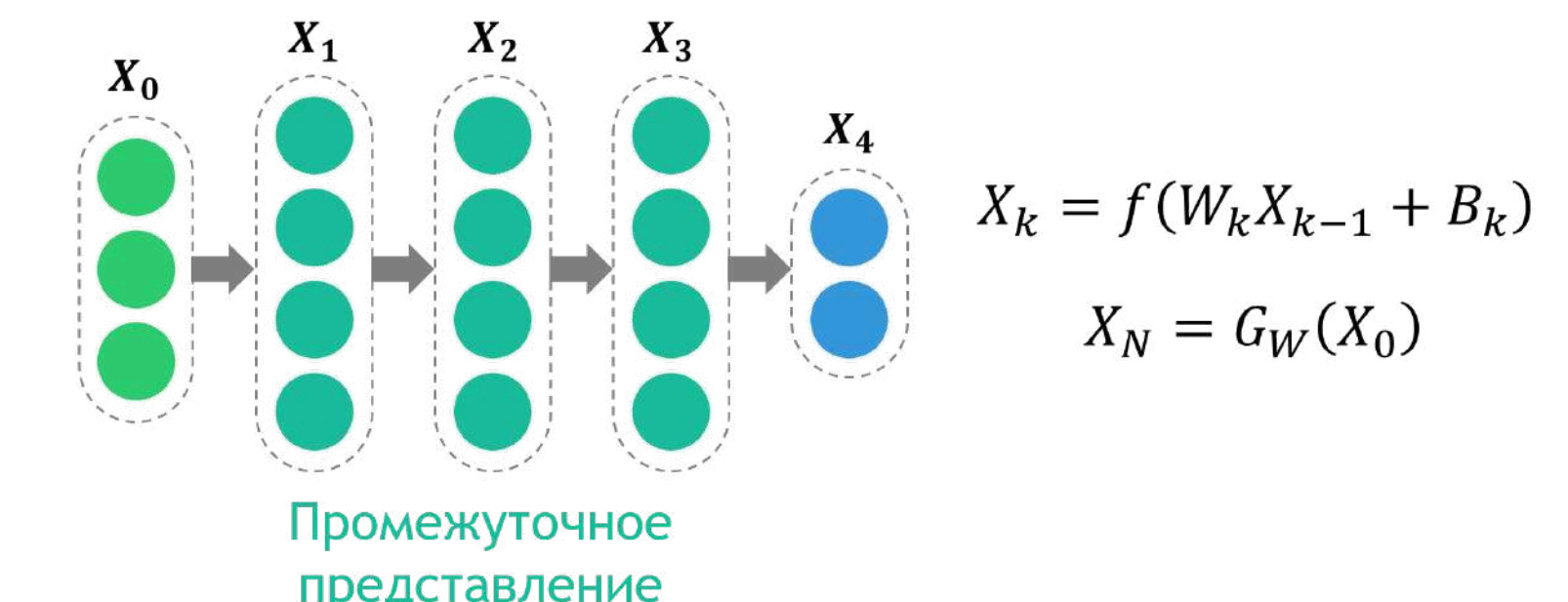
Если есть какая-то сложная зависимость между входом и выходом, то можно с помощью нейронной сети описать эту зависимость.

Если брать больше слоёв, то это уже будет многослойная нейронная сеть, и у неё уже будет более сложное промежуточное представление.

*Аппроксимация (от лат. grōxīma — ближайшая), или приближение — научный метод, состоящий в замене одних объектов другими, в каком-то смысле близкими к исходным, но более простыми.

Вычисления в многослойной нейронной сети

Вычисления в такой сети можно записать с помощью следующего рекуррентного соотношения в многослойной нейронной сети:



Выход каждого слоя — это есть вход, умноженный на какую-то матрицу, плюс вектор смещения, и от всего этого берётся нелинейность. Все веса, которые соответствуют смещениям bias, мы назовём параметрами нашей модели.

Типы параметров нейронных сетей

Обычно у нейронных сетей выделяют два типа параметров: просто параметры (W) и гиперпараметры.

Гиперпараметры — параметры нашей системы, которые мы не обучаем, а «создаём руками», как конструкторы проектируют нейронную сеть. Мы определяем количество нейронов в слое, количество слоёв, какую функцию активации мы используем и так далее. Это мы задаём вручную, а вот параметры связи между нейронами и bias — это вещи, которые получаются автоматически в процессе обучения.

Какую функцию активации взять?

Долгое время использовались **сигмоидальные функции**, или sigmoid. В этой функции мы помогаем выходу из нейрона принять какое-то бинарное решение. То есть отображаем все его значения во что-то больше или меньше нуля. Главное — что это нелинейная функция.

Если бы мы использовали просто обычную линейную функцию (или вообще не использовали бы никакую функцию), то композиция слоёв без нелинейности давала бы нам одну большую линейную операцию. И поэтому не имело бы никакого смысла настраивать много слоёв в нейронной сети: они все были бы эквиваленты какому-то одному слою.

А так, если мы ставим нелинейность между слоями, наш аппроксиматор становится более сложным, он уже может аппроксимировать достаточно сложные функции. Это функция sigmoid. Она использовалась раньше, но у неё есть некоторые проблемы, и поэтому сейчас, как правило, используют **функцию ReLU** или её модификации.

В этой функции всё, что меньше 0, мы зануляем, а всё, что больше 0, оставляем как есть. У этой функции очень простая производная, а именно производная этой функции будет участвовать в процессе обучения и в алгоритме обратного распространения ошибки.

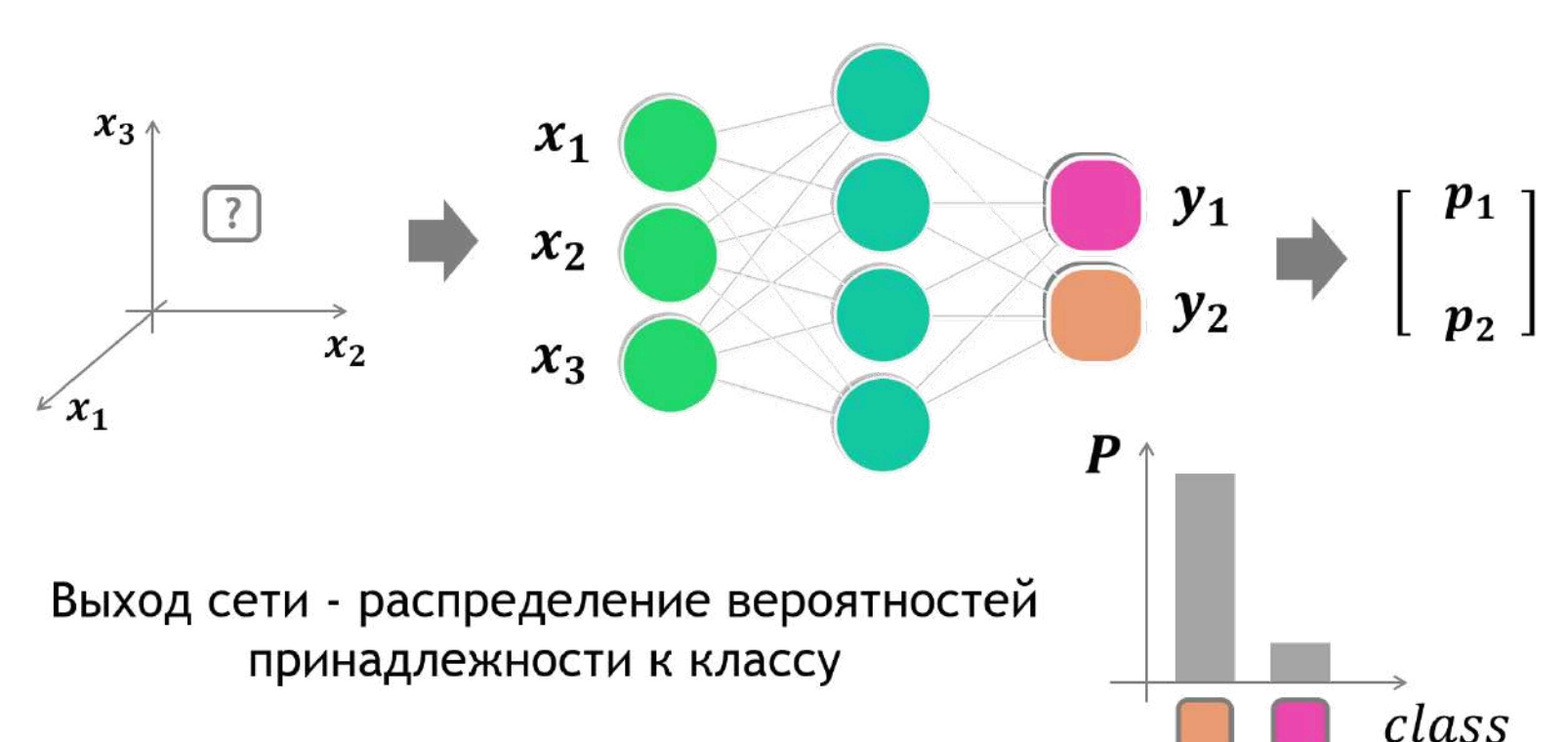
Как применить нейронную сеть для задачи классификации?

Представим, что у нас есть объекты в признаковом пространстве.

Объекты задаются тремя числами, и у нас есть три компонента этого вектора.

Мы так построили нашу нейронную сеть, что у неё есть три входных нейрона, как раз соразмерно нашему входному вектору. И, например, мы хотим сделать бинарную классификацию на 2 класса: фиолетовый и оранжевый. Поэтому мы сделали два выходных нейрона в нашей сети. Допустим, мы её уже как-то обучили. Так как её теперь использовать?

Ставим на вход сети наш вектор из трёх компонентов, делаем прямое распространение по тем формулам, которые описаны выше и получаем два значения на выходе. Они уже отвечают на вопрос, к какому классу принадлежит наш объект, но ещё после некоторого специального нормирующего преобразования мы получаем другие два числа: P_1, P_2 . Именно они уже явно характеризуют вероятность принадлежности к одному или второму классу. На выходе нейронной сети в случае классификации — **распределение вероятностей принадлежности к тому или иному классу**.



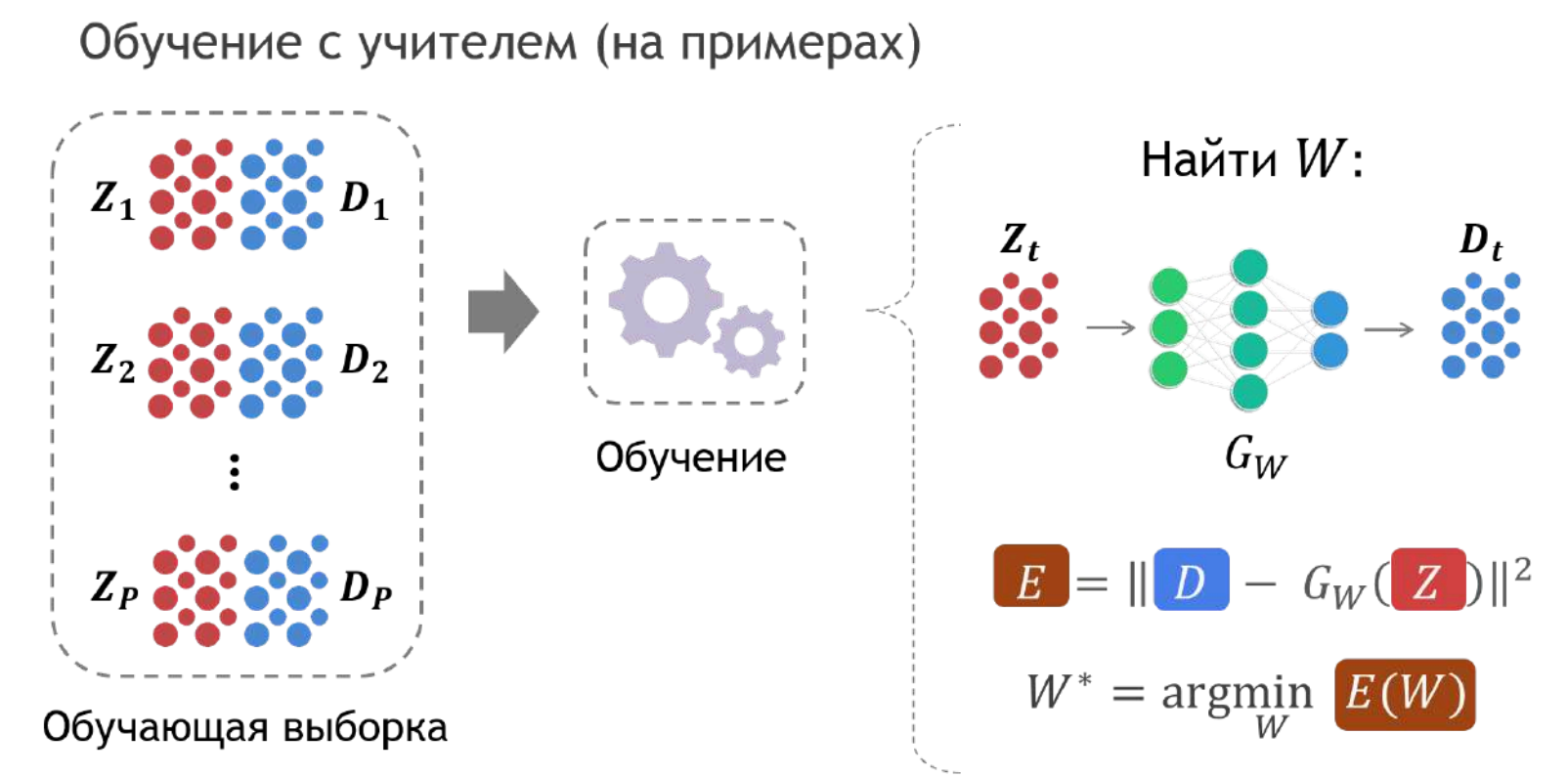


SKILLFACTORY

Чаще всего нейронные сети обучаются с учителем на размеченных данных. Для этого нам нужна обучающая выборка, которая состоит из пар входной объект — выходной объект. Мы подаём эту обучающую выборку в процесс обучения, который состоит в том, чтобы найти такие параметры модели W , чтобы наша нейронная сеть предсказывала правильно те самые ответы, которые мы уже знаем.

Таким образом мы решаем задачу минимизации или задачу оптимизации: мы минимизируем ошибку на тех примерах, которые есть в нашей обучающей выборке.

Ошибку можно записать по-разному:



Как решать задачу оптимизации?

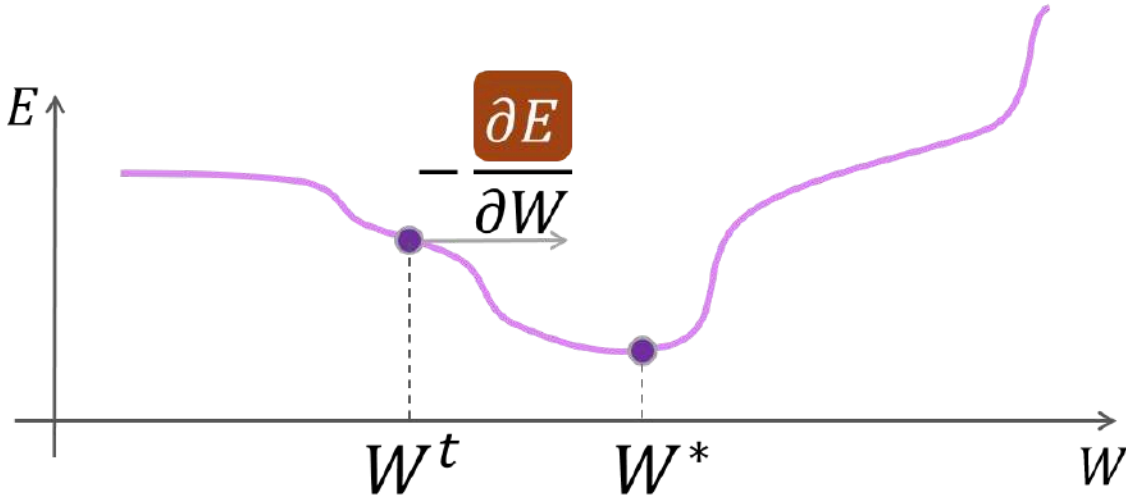
Есть различные способы. В теории оптимизации есть такой известный алгоритм как **градиентный спуск**. Смысл объясним на примере.

Допустим, у вас есть какая-то функция (здесь представлена одномерная функция, но в общем случае она может быть и многомерной), и вы хотите найти её минимум. Вы стоите в некоторой точке, и вам нужно понять, в какую сторону вам с этой точки нужно сдвинуться, чтобы приблизиться к минимуму. Есть вектор, который называется **градиент**. И этот вектор смотрит в сторону возрастания функции. Поэтому градиент со знаком минус смотрит в сторону убывания функции.

Подобный алгоритм предлагает двигаться в сторону антиградиента и таким образом приближаться к локальному минимуму.

Итак, мы вычислили градиент, и с некоторым **параметром α** , который ещё называют **Learning rate** (скорость обучения), мы этот антиградиент прибавляем к нашим текущим весам, и так получается итерационное движение к минимуму.

Градиентный спуск



Итерационный процесс

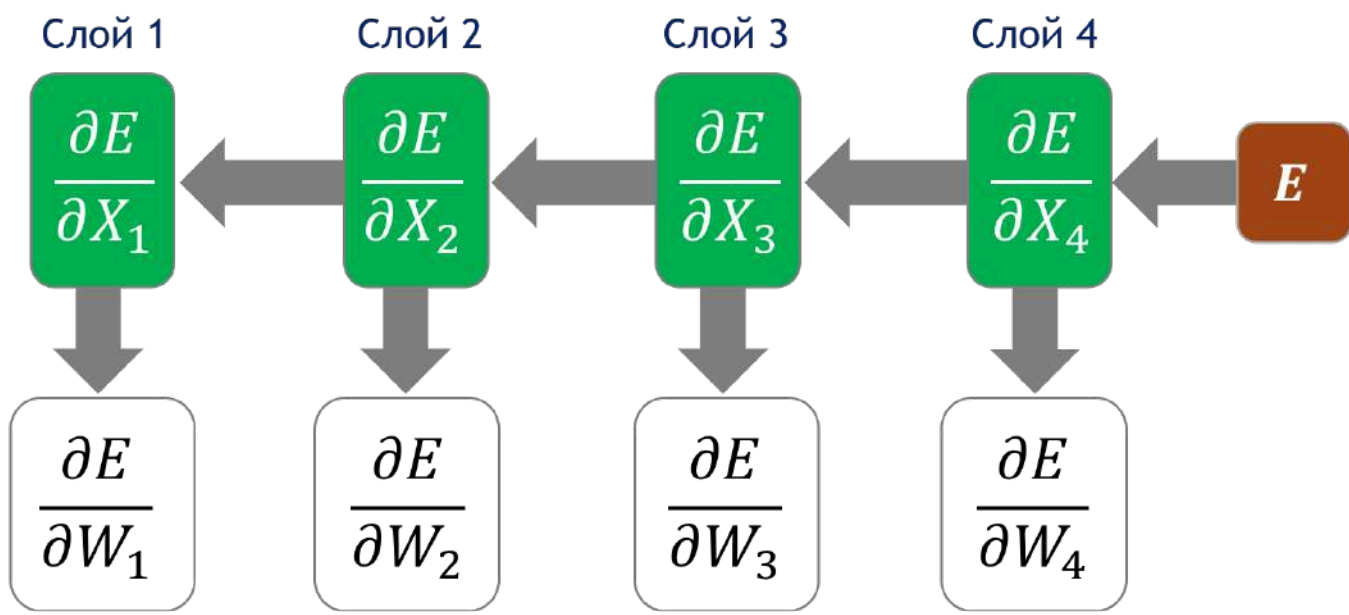
$$W^{t+1} = W^t - \alpha \frac{\partial E}{\partial W}(W^t)$$

В случае нейронных сетей используется небольшая модификация — **стохастический градиентный спуск**. Отличие от предыдущего градиентного спуска в том, что мы вычисляем градиент не сразу на всех образцах нашей выборки, а только на одном образце за одну итерацию или на группе образцов.

Как вычислить градиент?

Это вектор или даже некоторый тензор, размерность которого совпадает с размерностью всех наших параметров обучаемых.

Обратное распространение ошибки



Градиент может особенно зависеть от слоев, которые далеки от той самой ошибки, от которой мы считаем градиент. Например, у нас ошибка зависит от параметров первого слоя очень сложным образом, и, тем не менее, мы всё равно можем вычислить градиент с помощью **алгоритма обратного распространения ошибки**, который заключается в том, что мы используем правила дифференцирования сложной функции.

Нейронная сеть, даже если она представляет собой сложную функцию — на самом деле просто композиция каких-то маленьких простых вещей. Например, умножили на матрицу, прибавили вектор, взяли поэлементную матрицу и так далее. Используя такое дифференцирование сложной функции, можно узнать, как наша ошибка зависела от параметров второго слоя через это цепное правило. Именно таким образом можно вычислить градиент по любому весу и по всем весам. Это обратное распространение ошибки, или ещё его называют **backpropagation**.

Нейронные сети могут применяться для решения различных типов задач:

- классификация;
- регрессия;
- задачи генерации данных.

Они также широко используются в различных областях:

- компьютерное зрение;
- обработка текста;
- обработка речи.

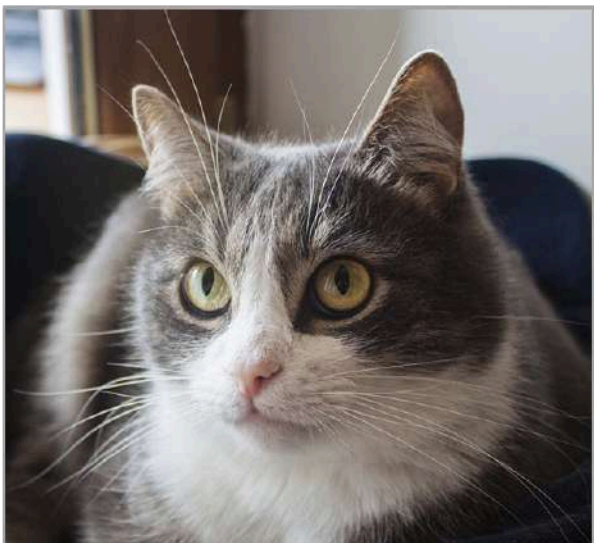
SKILLFACTORY

Задача компьютерного зрения состоит в отображении визуальной информации (изображения или видео) в высокоуровневую, семантическую информацию.

В классификации изображений на входе находится картинка, на выходе — метка класса.

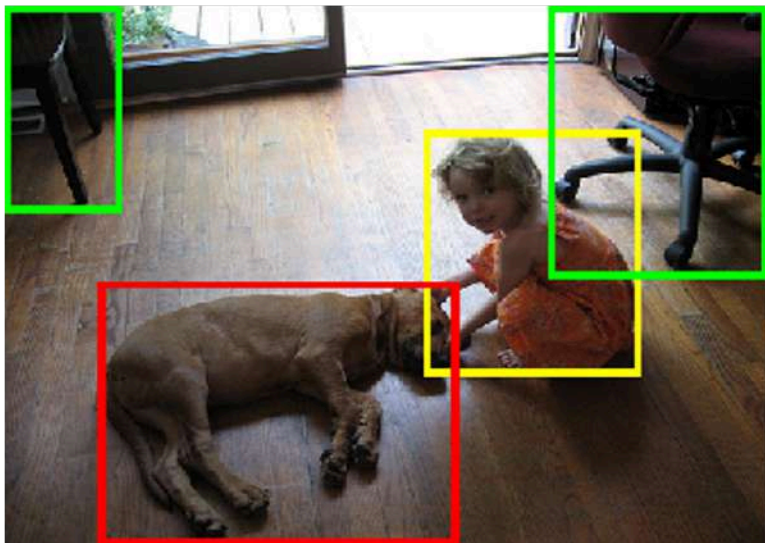
При детектировании и локализации объектов, помимо классификации, даётся ограничивающий прямоугольник, или локализация, где тот или иной объект присутствует на изображении.

Классификация



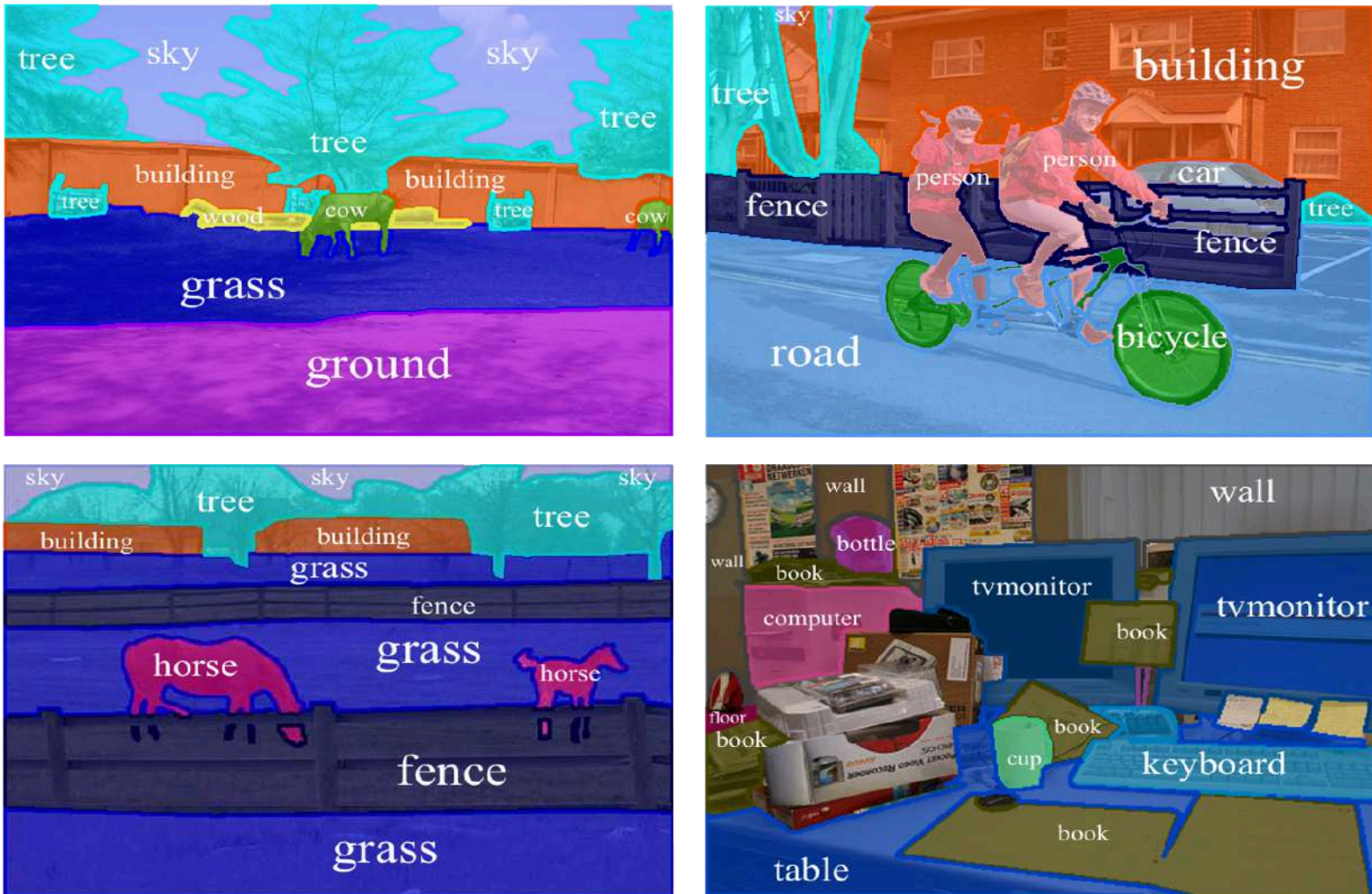
“Котик”

Детектирование объектов



- Человек
- Собака
- Стул

Семантическая сегментация — более сложная задача, в ней классифицируется каждый пиксель изображения, или выделяются некоторые сегменты на изображении, и каждому сегменту присваивается какая-то метрика.

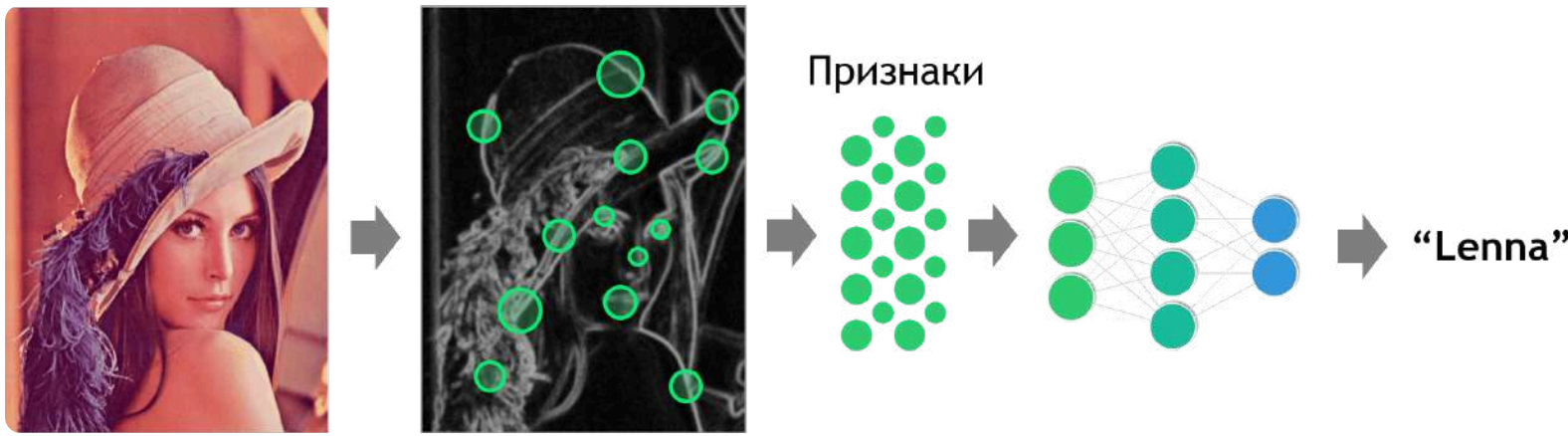


Семантическая сегментация

Основная трудность этих задач в том, что картинка или изображение представляет собой огромное количество неструктурированной информации, к которой непонятно, как подойти: это просто какие-то числа RGB. В машинном зрении ещё до прихода нейронных сетей использовались признаки. Признаки — это то, что описывает изображения. На картинке находятся какие-то особенности, они записываются в виде уже осмысленного высокоуровневого вектора и дальше к этому вектору применяется алгоритм классификации и получается ответ.

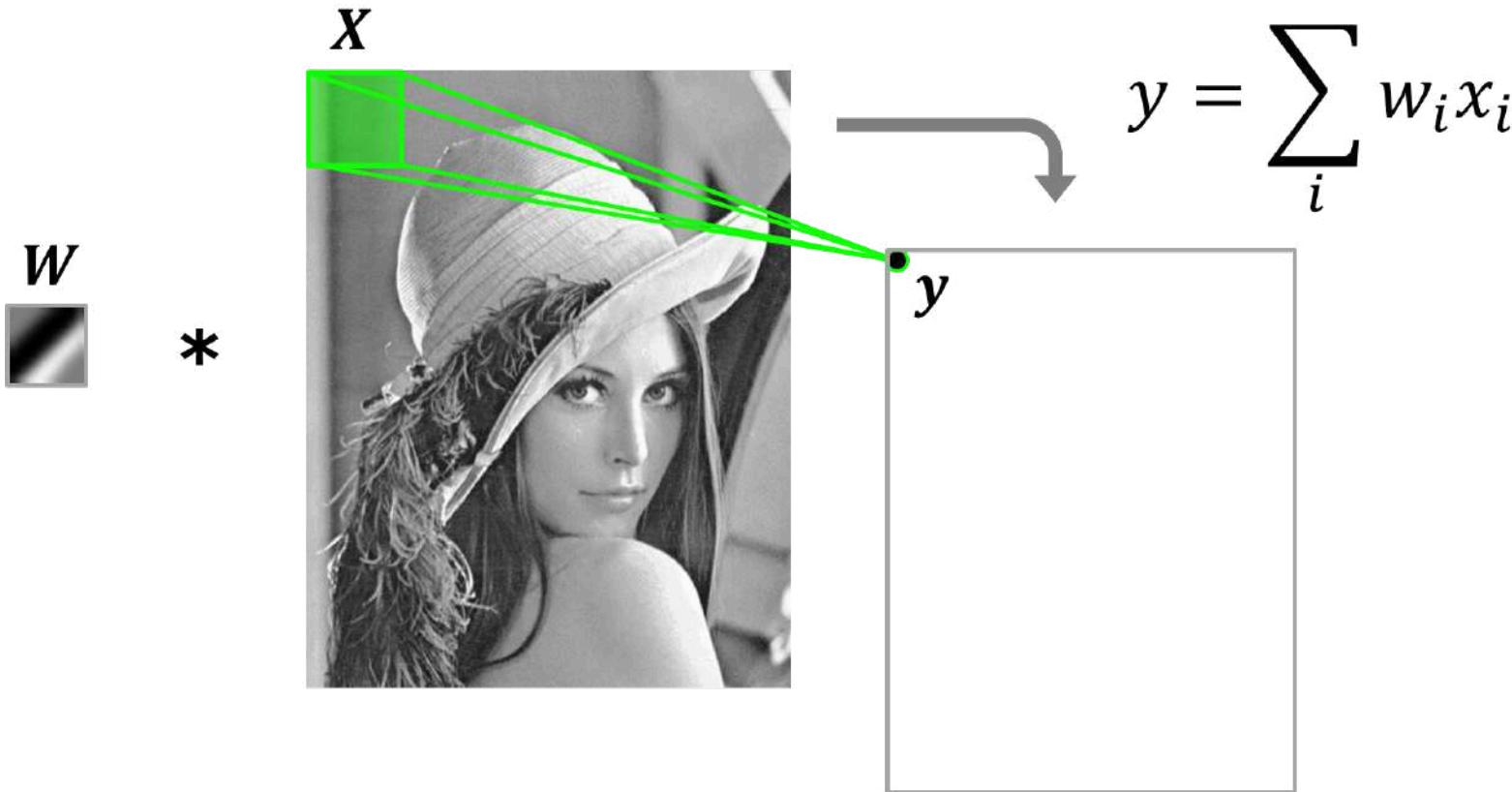
В итоге решение задачи сводится к двум частям:

- извлечение признаков;
- подача признаков в алгоритм машинного обучения.



До появления машинного зрения все признаки создавались вручную специальными инженерами, которые анализировали изображение и вручную кодировали то, как признаки должны выглядеть.

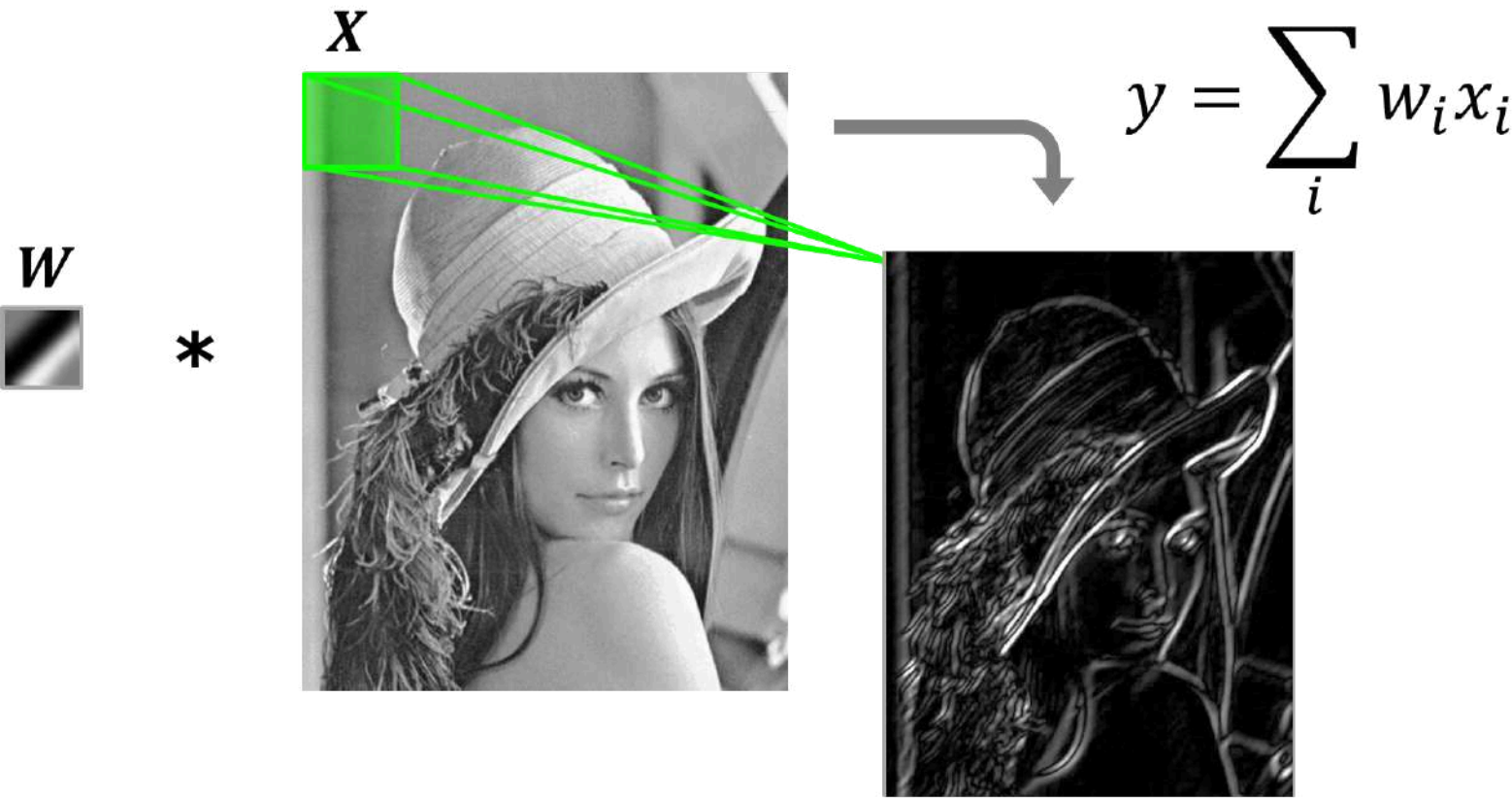
Свёртка — это способ извлечения признаков.





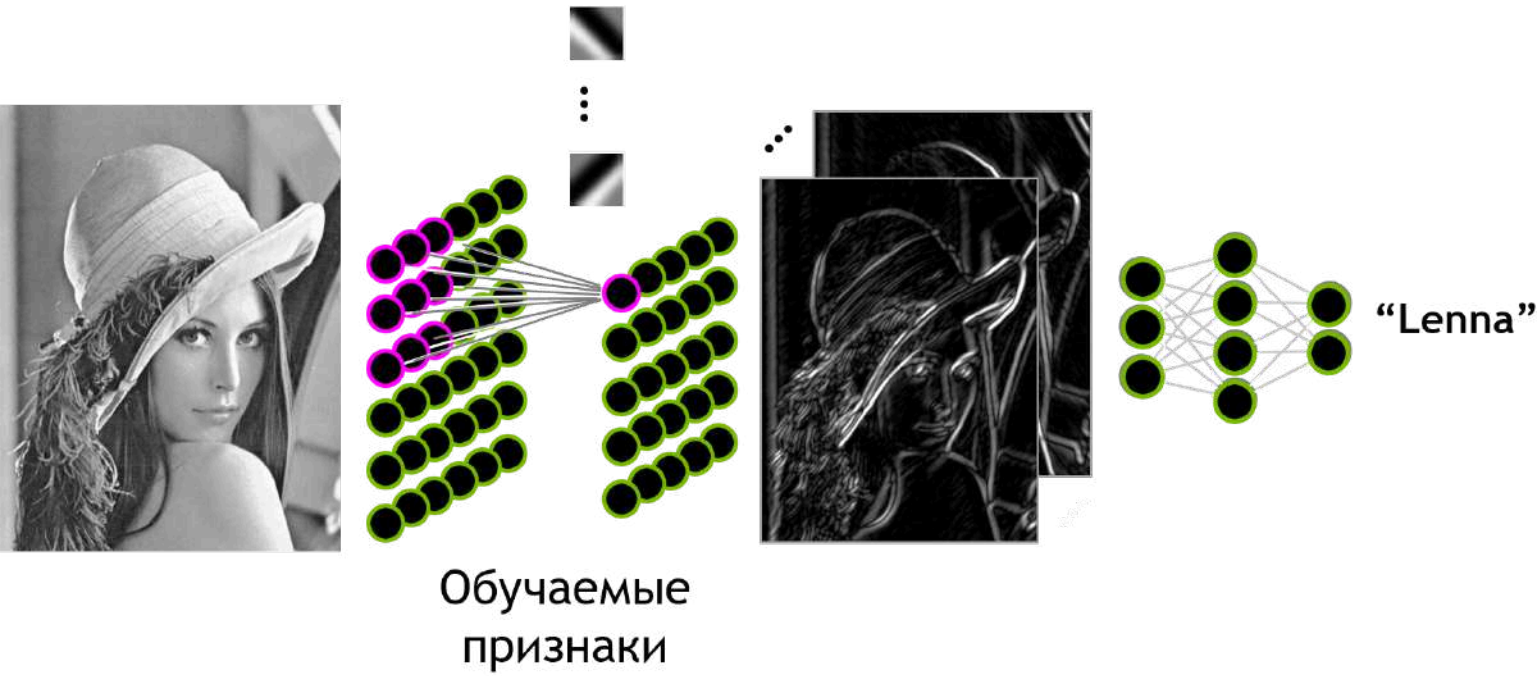
Идея свёрточных нейронных сетей состоит в том, чтобы использовать свёртку как специальный тип слоя в нейронной сети.

Обратимся к предыдущему примеру:



Посмотрим на свёртку: здесь видно, что у нас используется некоторая линейная операция для вычисления значения в выходной матрице. Такая же линейная операция использовалась нами в нейронных сетях.

Вернёмся:



Разница в том, что нейроны уже связаны в соседних тензорах. Не каждый с каждым, а только какая-то группа нейронов во входном слое связана с одним нейроном в выходном слое. Именно так и делается в свёрточных нейронных сетях.

Веса в ядре свёртки получаются с помощью оптимизации. Теперь они соответствуют связям между нейронами. Таким образом, мы получили автоматически обучаемые ядра свёрток или, другими словами, **обучаемые признаки** (в отличие от тех признаков в компьютерном зрении, которые использовались ранее, когда люди задавали их вручную).

Основная идея глубокого обучения — не просто изучение признаков, а извлечение их иерархии: извлечение признаков вышестоящего уровня в пространстве нижестоящего.

7/17

Что такое Deep Learning? Типы слоёв

текущий урок

SKILLFACTORY

Deep Learning — это обучение иерархии признаковых представлений.



Deep Learning (глубокое обучение) — это подход в машинном обучении, основанный на использовании многослойных параметризованных моделей (обычно нейронных сетей), которые автоматически обучают иерархические представления данных путём оптимизации функции потерь с использованием градиентных методов.

Что здесь важно:

- Тут есть признаки: сначала из входных данных извлекаются признаки, и дальше работа идёт уже с ними .
- Тут есть обучаемые признаки: мы делаем так, чтобы нейронная сеть их учила автоматически.
- Тут не просто один слой признаков — здесь иерархия признаков.

Какие типы слоёв бывают в таких сетях?

- свёрточный слой;
- понижение размерности (Pooling):
 - локальное усреднение;
 - локальный максимум;
- полносвязный слой (Fully-connected).

← Предыдущий урок

Следующий урок →

8/17

Обработка последовательностей

текущий урок



Обработка последовательностей в нейронных сетях заключается в том, что модель учитывает порядок элементов и использует информацию о предыдущих шагах при обработке текущего элемента.

В классическом подходе для этого применялись **рекуррентные нейронные сети (RNN)**, в которых используется скрытое состояние (hidden state), позволяющее сохранять информацию о предыдущих элементах последовательности.

Более продвинутые варианты рекуррентных сетей:

- **LSTM (Long Short-Term Memory)**
- **GRU (Gated Recurrent Unit)**

Эти архитектуры были разработаны для того, чтобы лучше сохранять долгосрочные зависимости в данных и частично решать проблему затухания градиента.

Однако в современных задачах обработки последовательностей чаще используют **архитектуры на основе трансформеров (Transformers)**, которые позволяют эффективно учитывать зависимости между элементами последовательности без рекуррентных вычислений.

Тем не менее рекуррентные сети остаются важным базовым подходом и используются в ряде задач и образовательных целей.

Где применяются модели для обработки последовательностей:

- Предиктивный ввод текста (умные клавиатуры).
- Анализ текста и комментариев.
- Машинный перевод.
- Чат-боты и диалоговые системы.
- Распознавание речи.
- Генерация описаний изображений и мультимодальные задачи.

<

Предыдущий урок

Следующий урок

>

SKILLFACTORY

Синтез — это обратная задача, то есть перевод информации высокого уровня в информацию низкого уровня.

На примере человека — восприятие и творчество:



На примере работы техники— распознавание и синтез:





Состязательная сеть (англ. Generative Adversarial Networks, GAN).

Идея состоит в том, чтобы построить вторую сеть, которая называется **дискриминатор**. Это просто бинарный классификатор: на входе — картинка, а на выходе — ответ с предположением, реальная картинка или синтезированная.



Важный момент: генератор должен учиться обманывать дискриминатор. Они учатся параллельно, играя в антагонистическую игру, и в итоге мы получаем идеальный генератор и идеальный дискриминатор.

Примеры применения:

- ▶ синтез изображений;
- ▶ преобразование текста в изображение;
- ▶ отображение изображения в изображение;
- ▶ отображение видео в видео;
- ▶ синтез речи (Wavenet);
- ▶ аудио в видео.

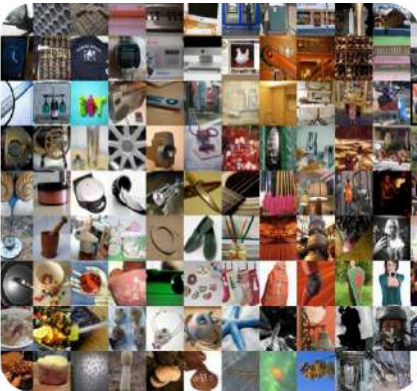
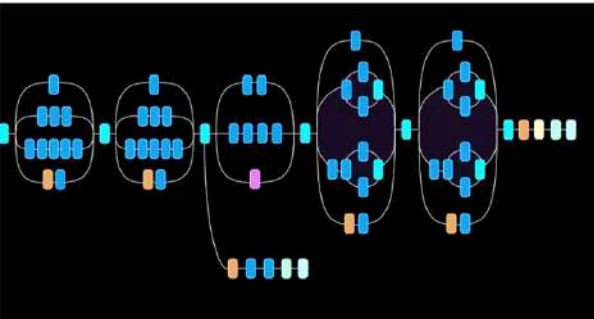
← Предыдущий урок

Следующий урок →



Причины успеха глубокого обучения:

- ▶ совершенствующиеся алгоритмы и архитектуры;
- ▶ доступные объёмы данных;
- ▶ ускорение обучения и вывода с помощью GPU.



< Предыдущий урок

Следующий урок >

12/17

Теоретическое резюме



Решаемые задачи: отображение X в Y:

- Текст, изображения, видео, звук, низкоразмерные данные.

Типы архитектур:

- Для распознавания и синтеза визуальных данных → **свёрточные** сети (CNN).
- Для распознавания и синтеза последовательностей → **рекуррентные** сети (RNN).
- Для улучшения качества синтеза → **состязательные** сети (GAN).

Факторы успеха: топология, данные, железо (GPU).

◀

Предыдущий урок

Следующий урок

▶

текущий урок

В основе современных фреймворков глубокого обучения лежит концепция **вычислительного графа** — удобного способа представления сложной функции в виде последовательности простых операций. Такое представление позволяет эффективно применять цепное правило и автоматически вычислять градиенты, что является ключевым механизмом обучения нейронных сетей.

Особое внимание уделим:

- ▶ принципам работы градиентного спуска,
- ▶ применению цепного правила в вычислительных графах,
- ▶ построению графа вычисления производных,
- ▶ связи этих концепций с реальными алгоритмами обучения.

В модуле вас ждёт не только теория, но и практика:

- ▶ разбор конкретных примеров (включая модель одного нейрона),
- ▶ работа в Google Colab,
- ▶ реализация простого вычислительного графа с использованием TensorFlow.

Завершает модуль практическое задание с самопроверкой, направленное на закрепление ключевых идей.

К концу модуля вы будете понимать, как фреймворки автоматически вычисляют производные и обучают модели, а также сможете самостоятельно реализовать базовые элементы этого процесса.

[← Предыдущий урок](#)

Следующий урок >

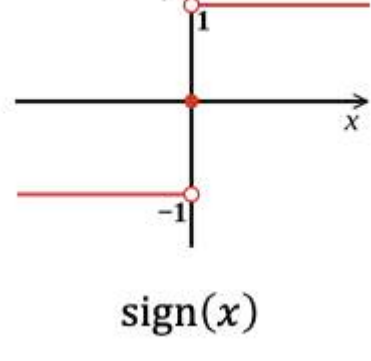


Прежде чем переходить к нейронным сетям, важно понять, из каких идей они вырастают. В этом уроке мы разберём базовые модели классификации — линейную классификацию и логистическую регрессию — и увидим, какие у них есть ограничения.

Это позволит понять, почему для решения более сложных задач требуются более мощные модели и как из простых линейных преобразований постепенно появляются нейронные сети.

Линейная классификация

Представьте, что у нас есть признаки $x = (x_1, x_2)$ и есть выборка положительных и отрицательных точек $y \in \{+1, -1\}$

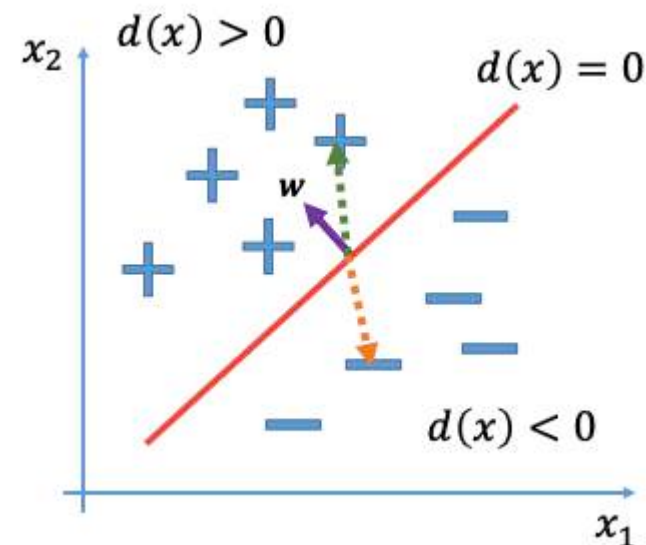


Нам нужно найти разделяющую гиперплоскость между ними. В данном случае это просто линия. Линия задается тремя коэффициентами:

$$d(x) = w_0 + w_1x_1 + w_2x_2$$

Нам нужно найти три коэффициента w , которые зададут линию. Далее мы можем взять точку x и понять, где она находится относительно линии: выше или ниже. Для этого нам нужно узнать знак линейной комбинации. Вектор весов w задаёт нормаль к нашей линии, то есть он перпендикулярен ей (фиолетовый вектор на графике ниже).

Линейная комбинация — это скалярное произведение и длина проекции какого-нибудь другого вектора на наш вектор w .



Поэтому проекция становится разных знаков. Из этих соображений мы делаем линейный классификатор. Наш алгоритм:

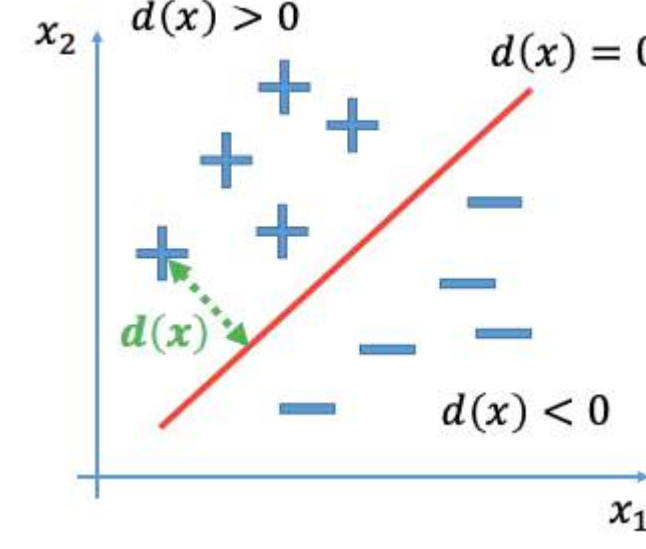
$$a(x) = \text{sign}(d(x))$$

Здесь появляется знак нашей линейной комбинации. Настроить линейный классификатор — значит, найти эти коэффициенты.

Логистическая регрессия

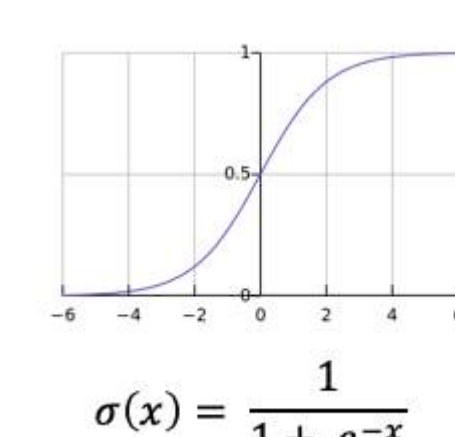
Она тоже решает задачу классификации, но в конце применяется не функция знака, а сигмоидная функция. Она превращает длину проекции в уверенность.

Уверенность и неуверенность появляется из-за краевых эффектов: на границе классов может быть какой-то шум, и в классификации точек, которые находятся рядом с красной разделяющей линией, мы не очень уверены.



А если мы уходим далеко от линии вглубь классов, то предполагается, что мы более уверены в этом предсказании. Сигмоида делает именно это — превращает длину проекции линейной комбинации в уверенность.

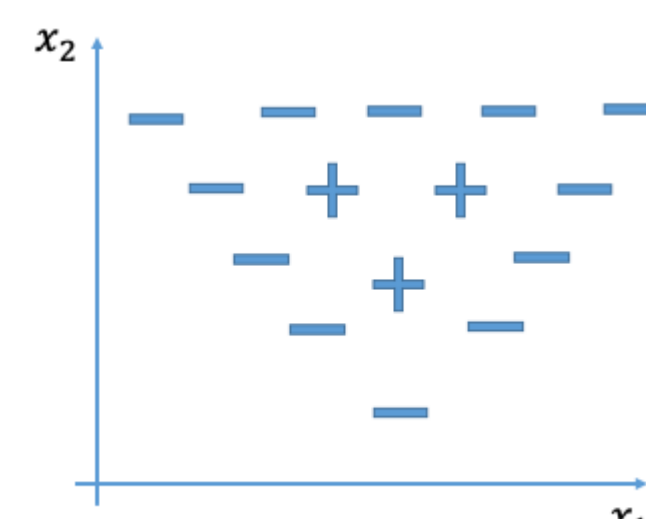
Сигмоида устроена не случайным образом:



Если длина проекции 0 (точка ровно на красной линии), то сигмоида даёт 0.5. Логистическая регрессия предсказывает вероятность положительного класса. Вероятность отрицательного будет единица минус предсказанная вероятность положительного класса.

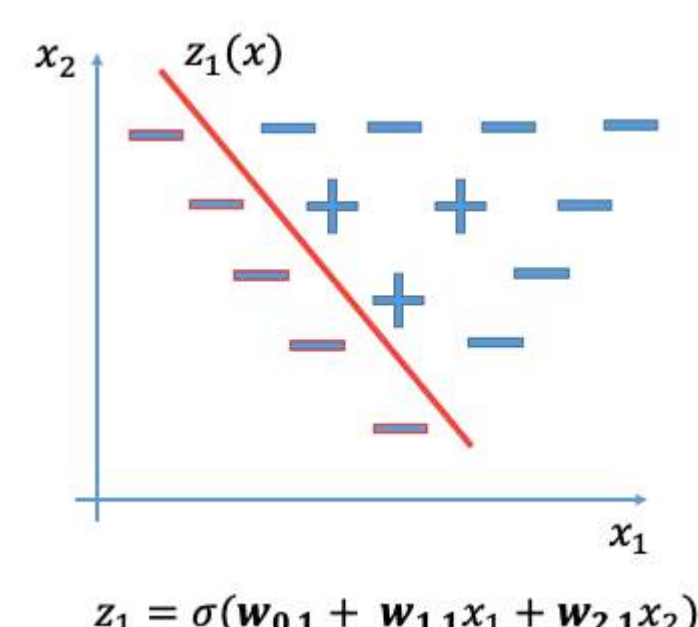
Другой пример

Представим, что у нас есть следующая задача:



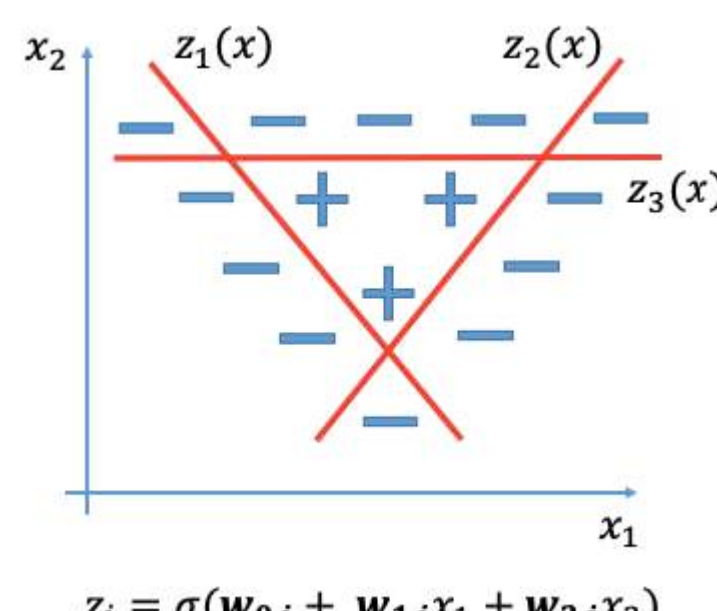
Чтобы решить подобную задачу, мы можем «подпереть» треугольник тремя линиями и сделать алгоритм, используя только логистическую регрессию.

Для начала мы отделим минусы слева и построим логистическую регрессию:



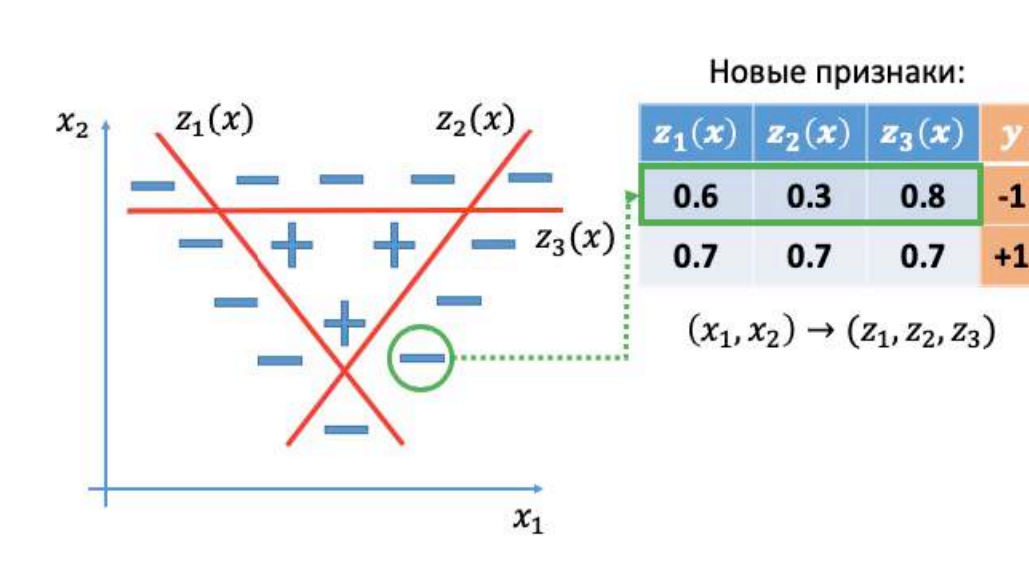
Эти данные подадим для обучения логистической регрессии и получим коэффициенты красной линии.

Отделим остальные минусы:



Все коэффициенты мы получили практически вручную. Сейчас у нас есть коэффициенты трёх логистических регрессий, каждая из которых решает свою маленькую задачу.

Теперь возьмём какую-нибудь точку и посмотрим, какие три предсказания дают эти линии в точке:



Новые признаки:			
$z_1(x)$	$z_2(x)$	$z_3(x)$	y
0.6	0.3	0.8	-1
0.7	0.7	0.7	+1

$(x_1, x_2) \rightarrow (z_1, z_2, z_3)$

Что делать с этими тремя значениями?

В координатах x_1 и x_2 эта задача не решается. Поэтому полученные коэффициенты мы можем рассматривать как новые координаты.

Получим три признака, каждый из которых говорит, где мы находимся относительно каждой стороны треугольника.

Давайте возьмём наш целевой признак y , добавим его в нашу новую таблицу, где наши новые признаки с предсказаниями, и попробуем решить её с новыми признаками с помощью линейной логистической регрессии. На новой выборке получим логистическую регрессию:

$$a(x) = \sigma(w_0 + w_1z_1(x) + w_2z_2(x) + w_3z_3(x))$$

Теперь она даёт нам некоторые коэффициенты и взвешивает уже не признаки, а предсказания. То, что мы получили — простейшая нейросеть.

Задание 2.1

0 points possible (ungraded)

Что определяет линейный классификатор?

- ☐ Расстояние до ближайшей точки обучающей выборки
- ☐ По какую сторону от разделяющей линии находится точка
- ☐ Вероятность принадлежности к классу
- ☐ Среднее значение признаков

Отправить

Задание 2.2

0 points possible (ungraded)

Что меняется при переходе от линейной классификации к логистической регрессии?

- ☐ Модель становится нелинейной
- ☐ Вместо знака используется сигмоида, и появляется вероятность
- ☐ Добавляется скрытый слой
- ☐ Используется другой набор признаков

Отправить



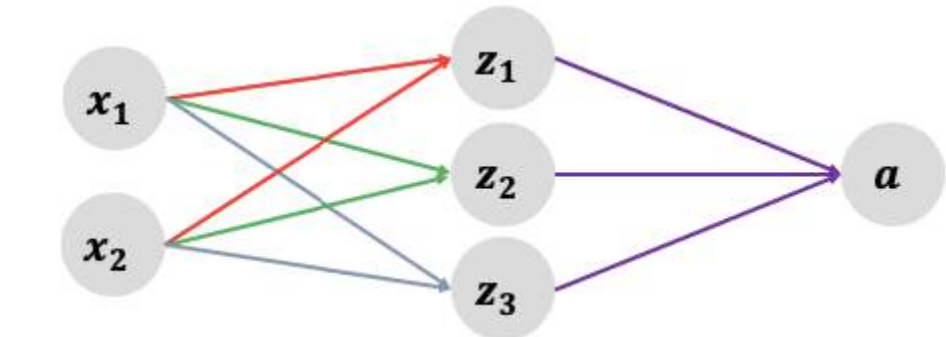
Граф вычислений

На данный момент мы «руками» нашли параметры всех линий:

$$z_l = \sigma(w_{0,l} + w_{1,l}x_1 + w_{2,l}x_2)$$
$$a(x) = \sigma(w_0 + w_1z_1(x) + w_2z_2(x) + w_3z_3(x))$$

При этом на нашу сложную комбинацию функций можно посмотреть как на граф вычислений. У графа есть вершины и каждая вершина — вычисляемое значение.

Также есть ребра — зависимости, которые имеют направления (на картинке ниже это стрелочки). Ребро идет от x_1 к z_1 в случае, если нам необходим x_1 , чтобы вычислить значение z_1 . Это граф зависимости между вычисляемыми значениями:

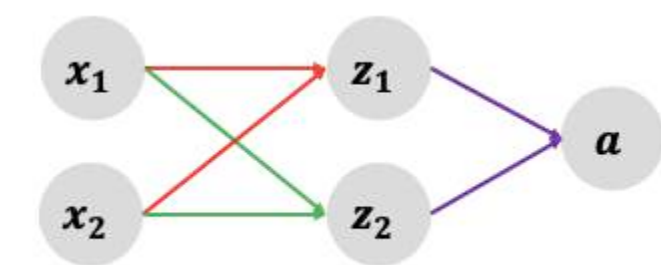


Граф соответствует комбинации наших функций. Такой граф называют многослойным персептроном, и здесь уже можно видеть некоторые слои:

- ▶ входной слой (признаки);
- ▶ скрытый слой (нейроны);
- ▶ выходной слой (предсказания).

Нелинейности в нейронах

Возьмём в качестве примера следующий граф:



Если нелинейности убрать, то на этом примере видно, что наша модель станет очень простой: мы можем подставить выражения для z_1 и z_2 в нашу модель a

$$z_1 = w_{1,1}x_1 + w_{2,1}x_2$$
$$z_2 = w_{1,2}x_1 + w_{2,2}x_2$$
$$a = w_1z_1 + w_2z_2$$

Мы можем раскрыть скобки, привести подобные слагаемые и всё, что мы получим — линейную комбинацию x_1 и x_2 . При этом модель сложнее не становится.

$$a = (w_1w_{1,1} + w_2w_{1,2})x_1 + (w_1w_{2,1} + w_2w_{2,2})x_2$$

Задание 3.1

0 points possible (ungraded)

Какие утверждения следуют из рассмотрения нейронной сети без нелинейностей? Выберите два правильных ответа.

- ☒ Даже при наличии нескольких слоёв модель остаётся линейной
- ☐ Увеличение числа слоёв делает модель сложнее и позволяет решать более сложные задачи
- ☒ Все преобразования можно свести к одной линейной комбинации входных признаков
- ☐ Такая модель не может обучаться с помощью градиентного спуска
- ☐ Для работы такой модели обязательно нужны функции активации

✔

Ответ
Верно:
Верно. В тексте прямо сказано, что после упрощения всех преобразований получается линейная комбинация входных признаков.
Верно. Это напрямую следует из рассуждения: подставляя выражения и упрощая, мы приходим к одной линейной функции.

Show answer

Отправить

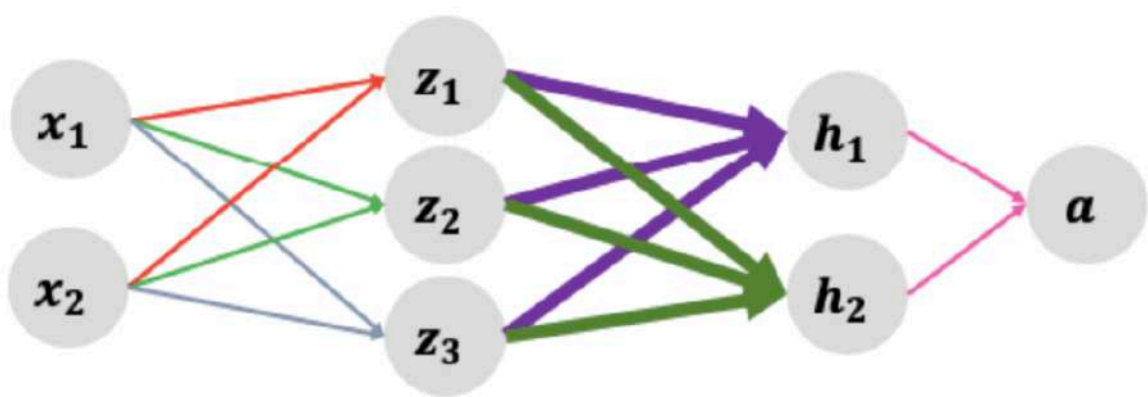


SKILLFACTORY

В предыдущих уроках вы увидели, что линейные модели ограничены в своих возможностях, а также разобрали, как с помощью последовательных преобразований и нелинейностей можно строить более сложные зависимости. Теперь рассмотрим одну из самых базовых архитектур, которая реализует эти идеи на практике, — многослойный персептрон.

Что такое MLP

MLP (Multi-Layer Perceptron, многослойный персептрон) — один из простейших и наиболее фундаментальных типов нейронных сетей.



Wikipedia / wikipedia.org

Его можно рассматривать как граф вычислений, в котором входные данные последовательно проходят через несколько слоёв преобразований, формируя всё более сложные представления.

Каждый слой состоит из нейронов. Нейрон выполняет два шага:

- 1. Линейное преобразование входных данных (взвешенная сумма).
- 2. Применение функции активации (нелинейности).

Именно наличие нелинейностей позволяет сети моделировать сложные зависимости и строить нелинейные границы между классами.

Слои в MLP

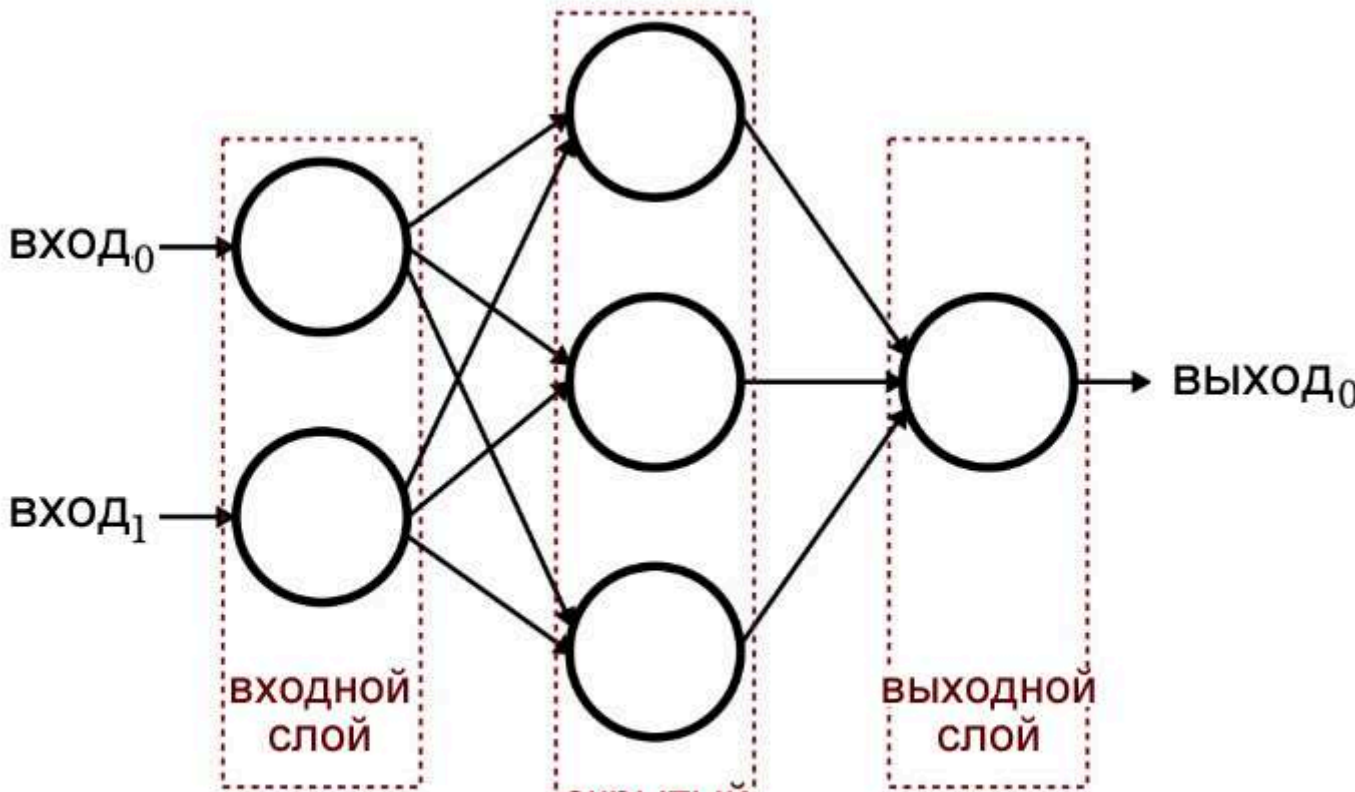
Слои в MLP называются:

- Dense layer (плотный слой).
- Fully-connected layer (полносвязный слой).

Это означает, что каждый нейрон текущего слоя связан со всеми нейронами предыдущего слоя.

В MLP выделяют три типа слоёв:

- Входной слой — принимает исходные признаки.
- Скрытые слои — выполняют преобразования данных.
- Выходной слой — формирует предсказание.



Radio Prog / radioprogram.ru

Архитектура MLP

Архитектура MLP определяется несколькими ключевыми параметрами:

- Количество слоёв.
- Количество нейронов в каждом слое.
- Функциями активации.

Изменяя эти параметры, можно управлять сложностью модели и её способностью описывать данные.

MLP — это естественное продолжение идей, которые мы уже рассматривали:

- Линейная модель → один слой без нелинейности.
- Добавление нелинейности → возможность моделировать сложные зависимости.
- Несколько слоёв → последовательное преобразование признаков.

Таким образом, MLP можно рассматривать как формализованную и обобщённую версию тех конструкций, которые мы строили вручную ранее.

Где применяется MLP

Несмотря на простоту, MLP используется во многих задачах:

- Классификация (например, определение класса объекта по признакам).
- Регрессия (предсказание числовых значений).
- Обработка табличных данных.
- Базовые задачи компьютерного зрения и обработки текста (в составе более сложных моделей).

Задание 4.1

0 points possible (ungraded)

Зачем в многослойном персептроне (MLP) используются функции активации?

☐ Чтобы ускорить обучение модели

☒ Чтобы сделать модель способной описывать сложные (нелинейные) зависимости

☐ Чтобы уменьшить количество параметров в модели

☐ Чтобы преобразовать входные данные в числовой формат

✔
Ответ
Верно. Без функций активации сеть сводится к линейной модели, а с ними может строить сложные зависимости и нелинейные границы.

Show answer

Отправить

✔ Верных

← Предыдущий урок

Следующий урок →

TensorFlow — фреймворк для глубокого обучения

TensorFlow — один из самых популярных фреймворков для разработки и обучения нейронных сетей. Он используется как в исследованиях, так и в индустрии, позволяя работать с моделями разной сложности — от простых многослойных персептронов до современных архитектур глубокого обучения.



TensorFlow / tensorflow.org

Основой всех вычислений в TensorFlow являются **тензоры** — многомерные массивы данных. С ними выполняются все операции внутри модели — от простых арифметических преобразований до сложных вычислений, связанных с обучением нейронных сетей. По сути, любая модель в TensorFlow — это последовательность операций над тензорами.

TensorFlow — это система, в которой тензоры (данные) проходят через цепочку вычислений (flow), образуя модель.

В современных версиях TensorFlow (2.x) используется режим **eager execution**, при котором операции выполняются сразу, как обычный Python-код. Это делает разработку более удобной: вы можете запускать код построчно, проверять значения переменных и легче отлаживать модель. Такой подход значительно снижает порог входа по сравнению с более ранними версиями фреймворка.

Однако важно понимать, что «под капотом» TensorFlow по-прежнему опирается на представление вычислений в виде **графа операций**. В этом графе каждая операция принимает на вход тензоры и возвращает новые тензоры, а связи между операциями задают порядок вычислений. Такое представление позволяет оптимизировать выполнение кода, эффективно использовать ресурсы (CPU, GPU, TPU) и ускорять обучение моделей.

На практике это означает, что, хотя вы пишете код в привычном стиле, TensorFlow может автоматически преобразовать его в более эффективную форму выполнения. Это особенно важно при работе с большими моделями и объёмами данных, где производительность играет ключевую роль.

Кроме того, TensorFlow предоставляет высокоуровневый интерфейс Keras, который позволяет описывать нейронные сети декларативно — через последовательность слоёв. Это избавляет от необходимости вручную управлять вычислениями и позволяет сосредоточиться на архитектуре модели: количестве слоёв, числе нейронов, функциях активации и выборе функции потерь.

Таким образом, TensorFlow сочетает в себе два важных свойства:

- ▶ С одной стороны — простоту использования и понятный Python-интерфейс.
- ▶ С другой — высокую производительность и оптимизацию вычислений за счёт внутреннего графового представления.

В дальнейшем мы будем использовать TensorFlow именно в таком формате — как удобный инструмент для реализации моделей, опираясь на уже изученные теоретические принципы.



Дополнительные материалы

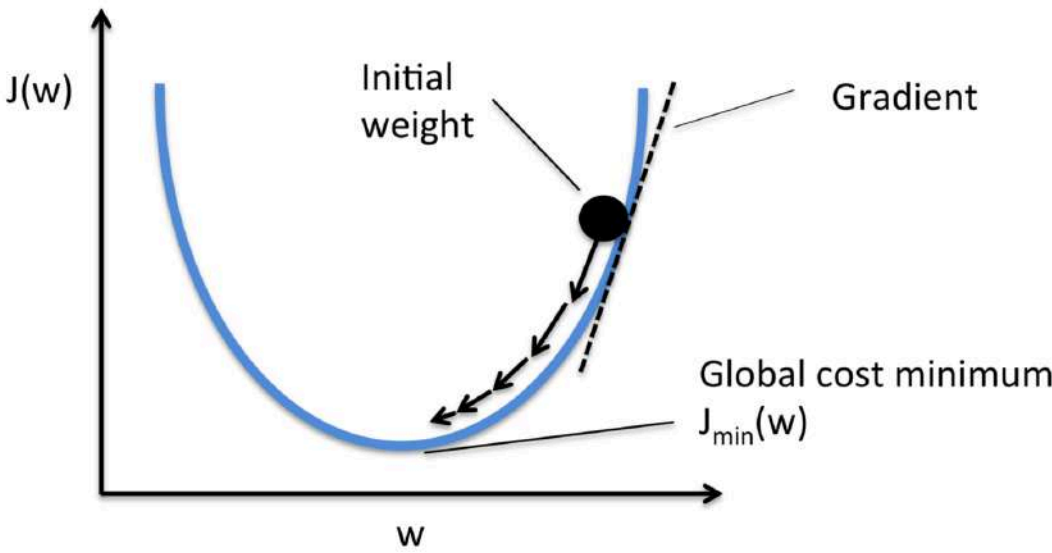
- ▶ [Официальный сайт TensorFlow \(документация и инструкции\)](#)
- ▶ [GitHub \(исходный код\)](#)
- ▶ [Google Colab \(без установки\)](#)
- ▶ [Учебники по TensorFlow](#)
- ▶ [Keras Documentation](#)

< Предыдущий урок

Следующий урок >

6/13

Пример обучения



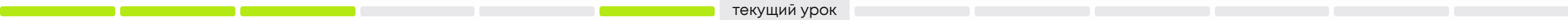
Функция потерь обычно дифференцируемая. Если представить, что у нас есть параметр w , который нужно оптимизировать, то мы можем начать с любой точки (initial weight) и посчитать в ней производную. В данном случае это tg наклона касательной.

Дополнительные материалы

Визуализация обучения нейронной сети с помощью [TensorFlow Playground](#)



7/13 Детали градиентного спуска: цепное правило



Умеем дифференцировать:

$$\frac{dx^2}{dx} = 2x \quad \frac{de^x}{dx} = e^x \quad \frac{d\ln(x)}{dx} = \frac{1}{x}$$

Возьмем сложную функцию:

$$\begin{aligned} z_1 &= z_1(x_1, x_2) \\ z_2 &= z_2(x_1, x_2) \\ p &= p(z_1, z_2) \end{aligned}$$

Цепное правило:

$$\frac{\partial p}{\partial x_1} = \frac{\partial p}{\partial z_1} \frac{\partial z_1}{\partial x_1} + \frac{\partial p}{\partial z_2} \frac{\partial z_2}{\partial x_1}$$

Пример для h(x)=f(x)g(x):

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial f} \frac{\partial f}{\partial x} + \frac{\partial h}{\partial g} \frac{\partial g}{\partial x} = g \frac{\partial f}{\partial x} + f \frac{\partial g}{\partial x}$$

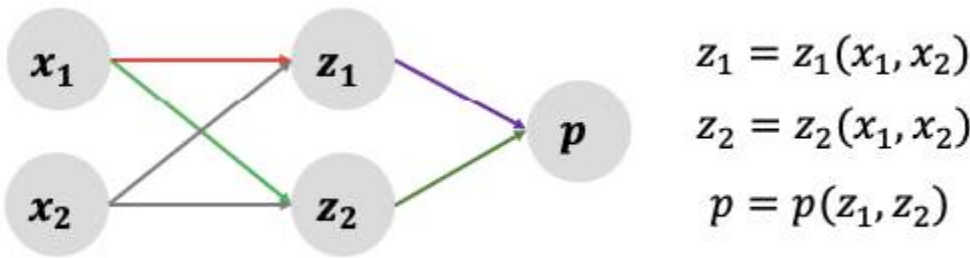
← Предыдущий урок

Следующий урок →

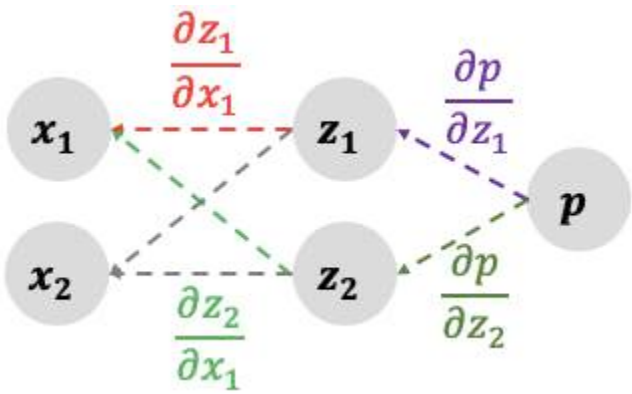


Граф для вычисления производных

Граф для вычисления нашей композиции:



Построим из него граф из производных:



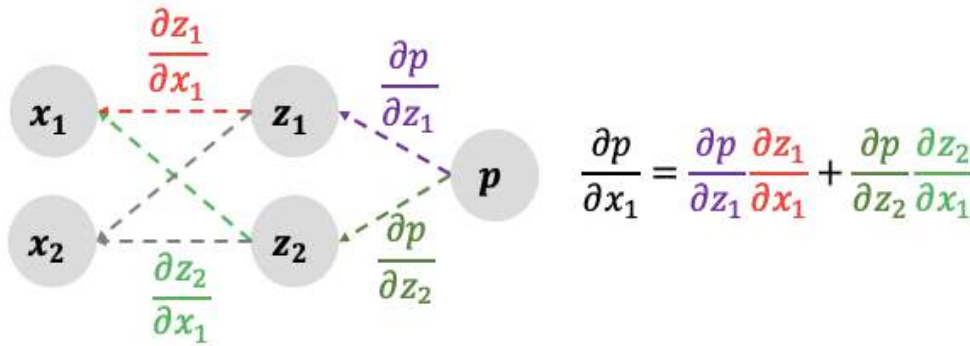
Каждому ребру приписана производная начала по концу. Можно догадаться, как работает цепное правило:

$$\frac{\partial p}{\partial x_1} = \frac{\partial p}{\partial z_1} \frac{\partial z_1}{\partial x_1} + \frac{\partial p}{\partial z_2} \frac{\partial z_2}{\partial x_1}$$

Алгоритм вычисления производной в графе

Как посчитать производную *a* по *b*:

- Находим непосещённый путь из *a* в *b*;
- Перемножаем значения на ребрах в пути;
- Добавляем в сумму;
- Возвращаемся к п.1.



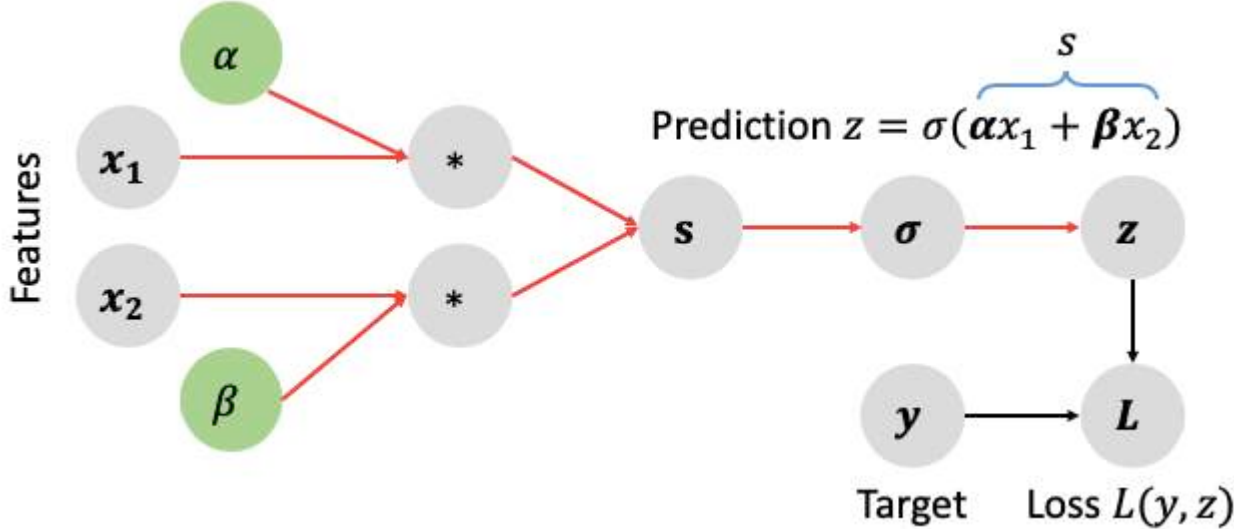
9/13

Пример одного нейрона

текущий урок



Посмотрим на прямой и обратный граф на примере одного нейрона.



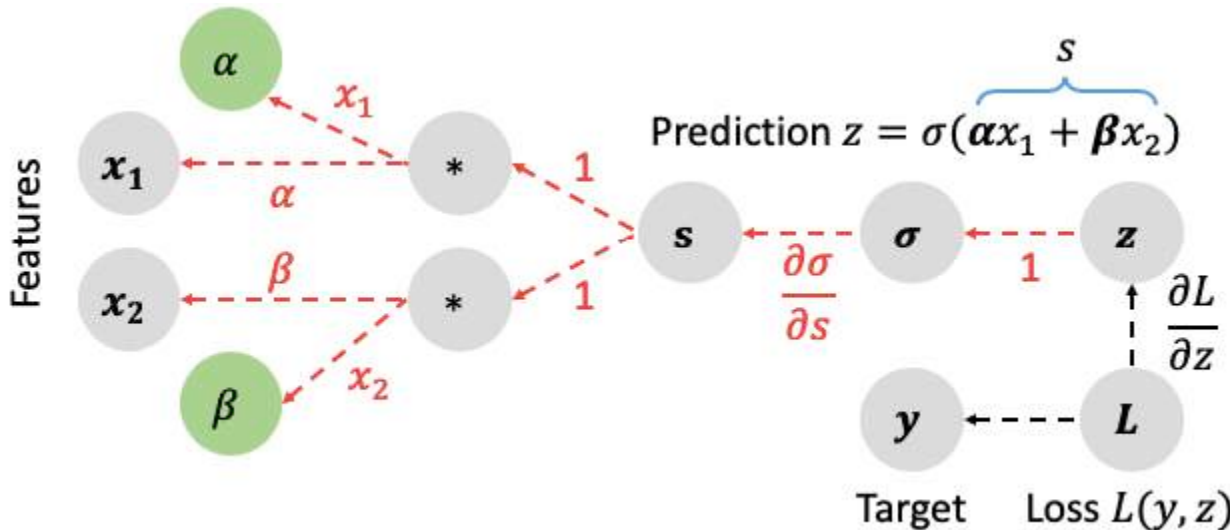
Рассмотрим, из чего он состоит по порядку:

- ▶ x_1, x_2 — признаки. Это то, что подаётся на вход (плейсхолдеры);
- ▶ a, b — веса модели (переменные);
- ▶ $*$ — операции произведения. После идёт подсчёт суммы;
- ▶ s — применяем сигмоиду;
- ▶ z — записываем в новую переменную;
- ▶ L — функция потерь, которая берёт на вход правильные ответы y , предсказанные ответы z и говорит, на сколько они близки друг к другу;
- ▶ y — плейсхолдер, правильный ответ.

Производные функции нам нужны, чтобы узнать как нужно изменить a и b , чтобы минимизировать наши потери:

$$\frac{\partial L}{\partial \alpha}, \frac{\partial L}{\partial \beta}$$

Для того, чтобы взять производную, мы переворачиваем наш граф:



Для каждой стрелочки считаем производную. По сути нам нужно найти обратный путь от L до a :

$$\frac{\partial L}{\partial \alpha} = \frac{\partial L}{\partial z} \frac{\partial \sigma}{\partial s} x_1$$

Так же считается и по b

$$\frac{\partial L}{\partial \beta} = \frac{\partial L}{\partial z} \frac{\partial \sigma}{\partial s} x_2$$

В *TF* все производные строятся автоматически и достаточно быстро.

<

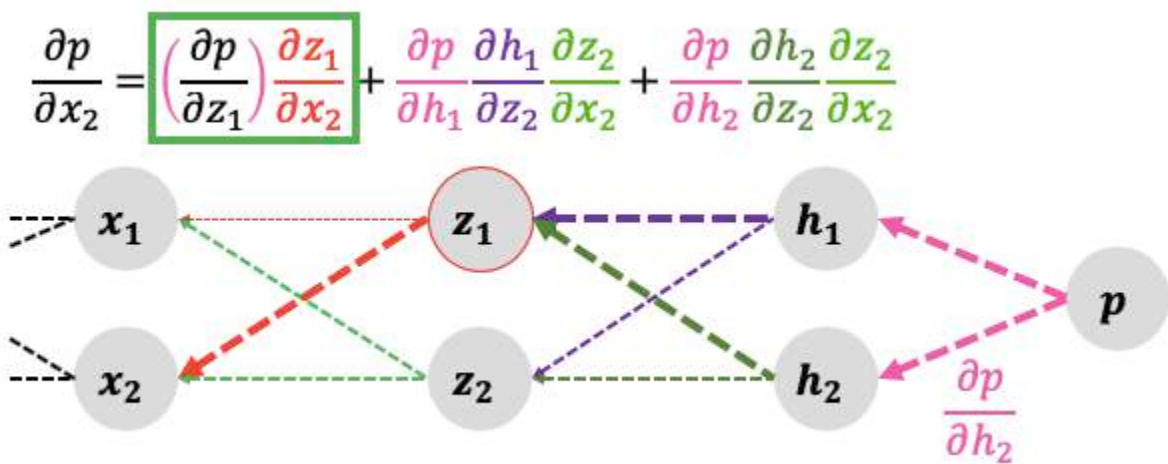
Предыдущий урок

Следующий урок

>

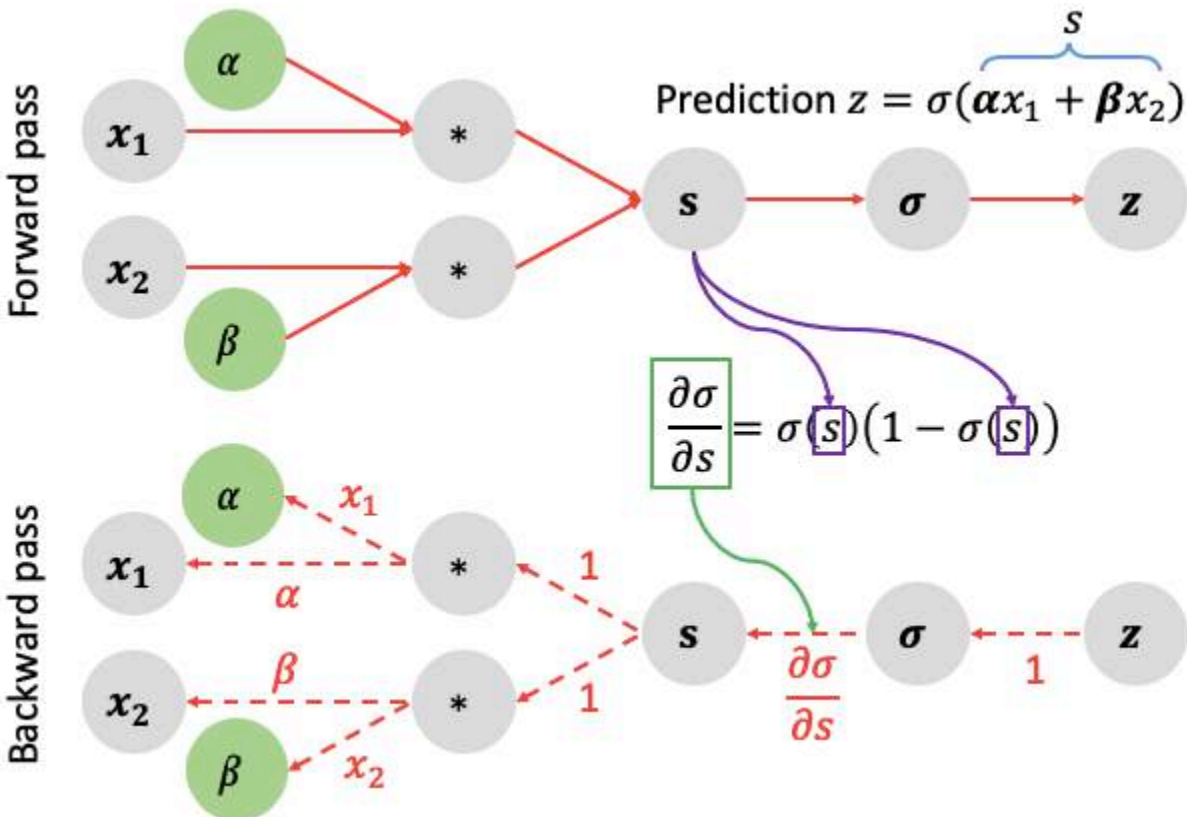


Теперь у нас есть алгоритм для подсчета производных для любых дифференцируемых графов вычислений. Эффективный способ вычисления всех производных называется **back-propagation** (обратное распространение ошибки):



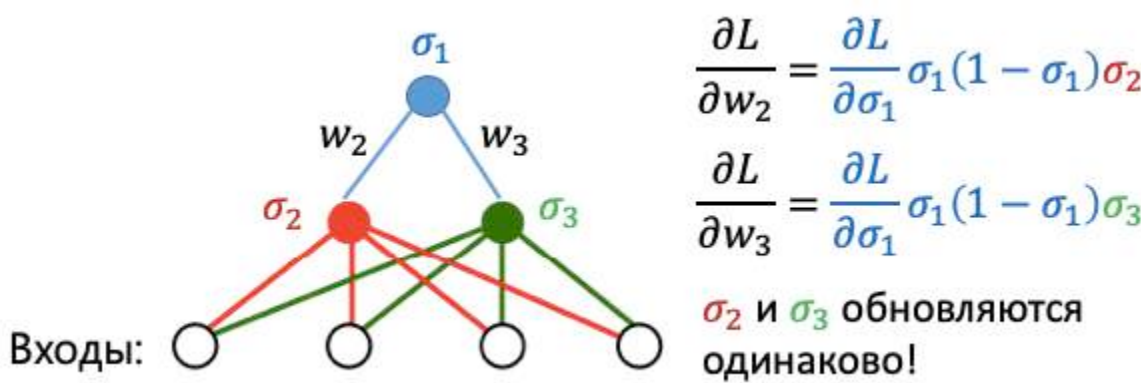
Back-propagation (Back-prop)

В *Back-propagation* есть два прохода: прямой и обратный. Те производные, которые считает обратный граф — нужно считать в определенной точке. Именно для этого нужен прямой проход — он рассчитает аргументы всех этих производных:



Инициализация весов

Мы не можем инициализировать нулями.



Если на данном примере мы стартуем на нулевых весах, то w_2 и w_3 будут обновляться симметрично, а, значит, мы не научим никакую сложную комбинацию за счёт простоты сети. Это называют симметрией.

Чтобы сломать симметрию, мы можем использовать случайный шум.

Дополнительные материалы

В качестве дополнительной литературы рекомендуем вам прочесть статью [«Step-by-step backpropagation example»](#)



Практика. Google Colab

Этот урок вводный в нашу практическую часть. Мы вспомним или познакомимся с основами работы с Google Colab – удобной облачной среды для работы с языком программирования Python прямо в вашем браузере. Этот инструмент позволит нам создавать и исполнять код онлайн, открывая широкие возможности для разработки и анализа данных.

Если вы уже знакомы с Google Colab, смело переходите на следующий урок.



Ключевые тезисы:

- ▶ В видео рассмотрим среду **Google Colab**, облачную среду для работы с Python в браузере.
- ▶ Перед началом работы нужно войти в свой Google Account и **загрузить ноутбук с платформы SkillFactory**.
- ▶ В Google Colab работаем с ноутбуком, разбитым на ячейки с текстом и кодом.
- ▶ Выполняя ячейки, видим изменение статуса на коннектинг, поскольку код выполняется на виртуальной машине в облаке Google.
- ▶ Имеется возможность настройки ноутбука с помощью специальных магических команд.
- ▶ В Google Colab предустановлены многие пакеты, такие как TensorFlow.
- ▶ Можно управлять версиями пакетов и выбирать устройства для вычислений, такие как CPU, TPU или GPU.
- ▶ Для добавления новых ячеек используются графический интерфейс или шорткаты. (Вы можете сразу изучить все комбинации клавиш во вкладке Tools — Keyboard shortcuts, например: Ctrl + Enter для запуска ячейки.)
- ▶ Google Colab предоставляет интуитивно понятную среду для работы с ноутбуками.

< Предыдущий урок

Следующий урок >



Практика. Реализация простого графа вычислений с помощью TensorFlow

В этом уроке мы начнем наше знакомство с фреймворком TensorFlow с реализацией графа операций для простого вычисления. В последней части нашей практики мы познакомимся с интерфейсом framework Keras.

Обратите внимание, мы используем **ту же тетрадь**, что и в прошлом уроке.



Ключевые тезисы:

- ▶ TensorFlow использует граф потоков данных для вычислений, состоящий из узлов операций и ребер потоков данных. В версии 1 API TensorFlow используется для работы с графом, в то время как в версии 2 работа с графом скрыта.
- ▶ Подключаемся к ресурсам, импортируем TensorFlow, сбрасываем граф вычислений, выключаем режим eager execution. Создаем тензорные плейсхолдеры, перемножаем тензоры, создаем граф вычислений, оцениваем сессию вычислений, инициализируем переменные, закрываем сессию.
- ▶ Решаем задачу оптимизации для функции $f(x)=x^2$, находим минимум в точке (0,0).
- ▶ Решаем задачу оптимизации функции x^2 с использованием градиентного спуска в TensorFlow. Определяем переменную x, создаем граф вычислений, задаем оптимизатор, запускаем сессию, обновляем переменную x, применяем tensorboard для логирования.
- ▶ Обучаем линейную модель с плейсхолдерами для фич, целевых значений, весов, используя градиентный спуск и функцию потерь. Далее передаем функцию потерь в метод minimize, обучаем модель на 300 шагах.
- ▶ Знакомимся с интерфейсом Keras, создаем и обучаем нейронную сеть с одним полносвязанным слоем, компилируем модель, используем stochastic gradient descent в качестве оптимизации, выбираем функцию потерь среднеквадратичной ошибки. Запускаем обучение, логируем метрики в tensorboard, анализируем весовые коэффициенты.

Введение

Вы только что закончили модуль по TensorFlow, научились строить модели на Keras, и вдруг мы предлагаем выучить ещё один фреймворк. Зачем? Неужели одного мало?

Честный ответ: можно прожить и без PyTorch. Многие компании отлично работают только на TensorFlow. Но без PyTorch значительно сложнее оставаться в современном контексте, если вы хотите заниматься исследованиями, читать свежие статьи и понимать, что происходит в мире глубокого обучения прямо сейчас.

PyTorch — фреймворк машинного обучения для языка Python с открытым исходным кодом, созданный на базе **Torch**. Используется для решения различных задач: компьютерное зрение, обработка естественного языка. Разрабатывается преимущественно группой искусственного интеллекта Meta AI*, которая используется для создания и обучения нейронных сетей.

* Meta Platforms Inc. признана экстремистской, её продукты запрещены на территории РФ.

Попробуйте зайти на сайты главных конференций по машинному обучению — **NeurIPS**, **ICML**, **ICLR**. Посмотрите, на чём написан код к статьям. В девяти случаях из десяти это будет PyTorch. Откройте любую модель от Hugging Face — **BERT**, **GPT**, **T5**. Базовая реализация — это PyTorch. Спросите исследователей из OpenAI, Tesla или DeepMind, на чём они пишут прототипы. Ответ вас не удивит. Если посмотреть на российские компании, картина будет похожей. Яндекс активно использует PyTorch в своих исследованиях и разработке. Команда Yandex Research публикует статьи с кодом на PyTorch. В «Сбере», особенно в лаборатории AI Labs, тоже работают на PyTorch. В стартапах и небольших командах выбор почти всегда падает на PyTorch, потому что он гибче и быстрее для экспериментов.

Оба фреймворка, TensorFlow и Pytorch, активно используются в продакшене, выбор зависит от инфраструктуры и задач. Хотите развернуть модель на миллионах android-устройств? Берите TensorFlow с его TFLite. Нужен высоконагруженный сервис с предсказаниями в реальном времени? TF Serving справится отлично. Но если ваша задача — экспериментировать, пробовать новые идеи, быстро прототипировать, индустрия давно сделала выбор.

Чем они отличаются на практике?

В TensorFlow 2.x вы чаще всего работаете через Keras. Пять строк — и модель готова:

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy')
model.fit(x_train, y_train, epochs=5)
```

В PyTorch тот же результат потребует больше строк. Зато каждая из них будет прозрачна:

```
# создаём экземпляр нашей нейронной сети
model = MyNeuralNetwork()
# инициализируем оптимизатор Adam, передаём ему все обучаемые параметры модели
optimizer = torch.optim.Adam(model.parameters())
# запускаем цикл обучения на 5 эпох (полных проходов по датасету)
for epoch in range(5):
    # перебираем данные мини-батчами: x_batch — входы, y_batch — правильные ответы
    for x_batch, y_batch in dataloader:
        # обнуляем градиенты, чтобы они не накапливались с предыдущего шага
        optimizer.zero_grad()
        # прогоняем батч через модель (прямой проход) и получаем предсказания
        outputs = model(x_batch)
        # вычисляем функцию потерь — насколько предсказания далеки от правильных ответов
        loss = loss_fn(outputs, y_batch)
        # обратное распространение ошибки: вычисляем градиенты для всех параметров
        loss.backward()
        # обновляем веса модели на основе вычисленных градиентов
        optimizer.step()
```

Батч — это группа объектов, которые модель обрабатывает одновременно.

Здесь нет скрытой логики. Вы сами решаете, когда обнулять градиенты, когда считать потери, когда делать шаг оптимизатора.

Если вы знаете NumPy, то уже знаете большую часть PyTorch. Если вы знаете NumPy, вам будет гораздо проще освоить базовый синтаксис PyTorch. По сути, это NumPy с двумя дополнительными способностями: умение считать на видеокарте и автоматически вычислять градиенты.

Сравнение базовых операций

Операция	NumPy	TensorFlow 2.x	PyTorch
Создать массив	<code>np.array([1, 2])</code>	<code>tf.constant([1, 2])</code>	<code>torch.tensor([1, 2])</code>
Узнать размер	<code>arr.shape</code>	<code>tensor.shape</code>	<code>tensor.shape</code>
Тип данных	<code>arr.dtype</code>	<code>tensor.dtype</code>	<code>tensor.dtype</code>
Сложить два массива	<code>a + b</code>	<code>a + b</code>	<code>a + b</code>
Матричное умножение	<code>a @ b</code>	<code>a @ b</code>	<code>a @ b</code>
Превратить обратно в NumPy	—	<code>tensor.numpy()</code>	<code>tensor.detach().cpu().numpy()</code>

Главное различие в работе с GPU. TensorFlow чаще автоматически управляет размещением вычислений, хотя разработчик также может явно задавать устройства. В PyTorch вы явно переносите тензоры на видеокарту методом `.to('cuda')`. Чуть больше ручной работы, зато вы всегда точно знаете, где находятся ваши данные.

Что нас ждёт в этом уроке?

Мы начнём с фундамента — с тензоров. Научимся их создавать: из списков, из NumPy-массивов, с помощью специальных функций вроде `torch.zeros()` и `torch.ones()`. Разберёмся с их главными свойствами: формой, типом данных и устройством. И конечно, попробуем базовые операции: индексирование, изменение формы, сложение и умножение.

К концу урока вы напишете свой первый код на PyTorch. И, подозреваю, удивитесь тому, как это просто. А через пару недель, когда начнёте читать код исследовательских команд Яндекса или «Сбера», сможете узнавать знакомые конструкции.

Тензоры: знакомые массивы с новыми возможностями

Тензор, по сути, является многомерным массивом. Всё, к чему вы привыкли в NumPy, здесь тоже применимо:

```
import torch
import numpy as np

t = torch.tensor([1, 2, 3])           # создание из списка
t = torch.tensor([[1, 2], [3, 4]])    # создание матрицы
t = torch.from_numpy(np.array([5, 6, 7])) # конвертация из NumPy
```

Менять форму, суммировать, перемножать можно привычным способом. Также доступны удобные функции для быстрого создания: `→ rch.zeros()`, `→ rch.o ≠ s()`, `→ rch.r and()`.

Три главных атрибута

- **.shape** — размерность. Например, **(3, 4)** — 3 строки, 4 столбца.
- **.dtype** — тип данных. Для нейросетей чаще всего используют **float32**. Целочисленные тензоры по умолчанию **int64**.
- **.device** — где лежит тензор: на процессоре (**cpu**) или видеокарте (**cuda : 0**). Это одно из ключевых отличий от NumPy.

Перенос на видеокарту делается методом `.to('cuda')`. На практике это обычно выглядит так:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
t = torch.tensor([1, 2, 3]).to(device)
```

После этого тензор находится на GPU, и все вычисления с ним будут происходить именно там.

Важно: различные типы данных потребляют разное количество памяти. **float32** занимает 4 байта на элемент, **float64** уже 8 байт. При работе с крупными тензорами разница может достигать гигабайтов. Видеопамять всегда ограничена, поэтому контролировать типы данных и расположение тензоров нужно на протяжении всей работы.

Операции с тензорами

Индексирование. Доступ к элементам осуществляется через квадратные скобки:

```
t = torch.tensor([[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]])
row0 = t[0]           # первая строка
col1 = t[:, 1]         # второй столбец
sub = t[1:, 2:]        # подматрица
```

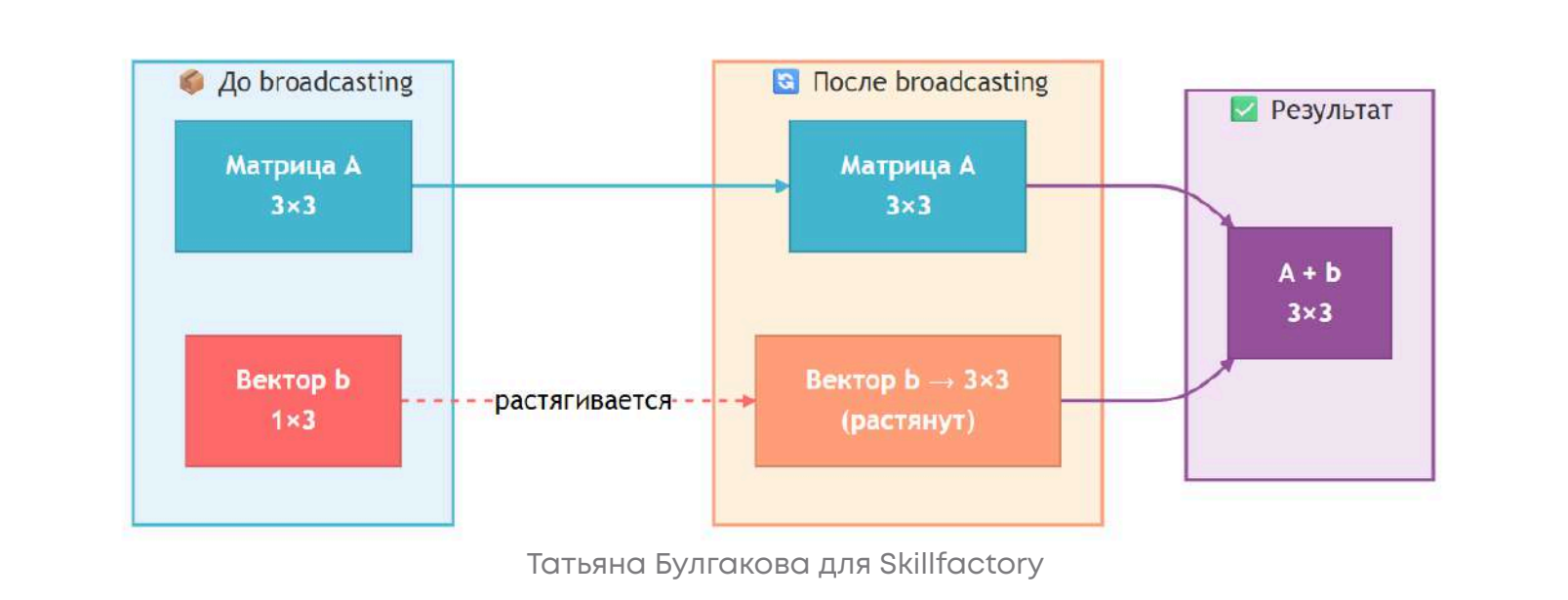
Важно: срезы возвращают не отдельную копию, а ссылку на те же данные. Модификация среза затронет и исходный тензор. Чтобы получить полностью независимый объект, вызывайте `.clone()`.

Изменение формы. Выполняется через **.reshape()**:

```
flat = t.reshape(-1)      # вытянуть в вектор
back = flat.reshape(3, 4) # обратно в матрицу
```

Математические операции

```
a + b      # элементное сложение
a * b      # элементное умножение
a @ b      # матричное умножение
t.sum()    # сумма всех элементов
t.float().mean() # среднее (требуется float)
t.max()    # максимум
```



Broadcasting работает как в NumPy: если размерности не совпадают, меньший тензор автоматически расширяется. Например, к матрице можно прибавить вектор, и он прибавится к каждой строке.

Вопрос 1.1

0 points possible (ungraded)

Что является главным отличием PyTorch от NumPy?

- ☐ PyTorch поддерживает только работу с текстом
- ☒ PyTorch умеет автоматически вычислять градиенты и работать на GPU
- ☐ PyTorch быстрее выполняет любые операции без исключений
- ☐ В PyTorch нельзя изменять форму массивов

✓

Show answer

Отправить

✓ Верных

Итоги урока

Мы познакомились с главным строительным блоком PyTorch — тензорами. Теперь вы умеете их создавать, узнавать их форму и тип, переносить на видеокарту и делать базовые операции. Этого уже достаточно, чтобы двигаться дальше.

В следующем уроке мы научимся управлять устройством ещё увереннее и познакомимся с тем, как PyTorch работает с данными. А пока — небольшое задание, чтобы закрепить материал.

← Предыдущий урок

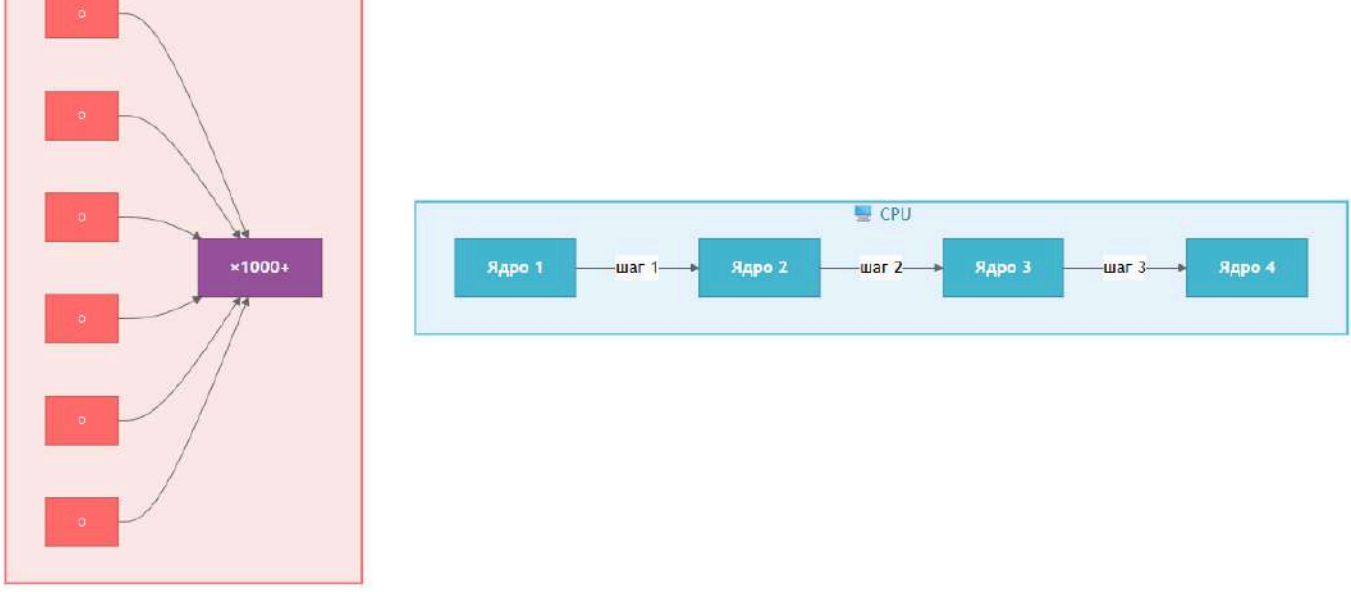
Следующий урок →



Введение

В предыдущем уроке мы разобрались, как создавать тензоры и выполнять базовые операции — сложение, умножение, изменение формы. Всё это происходило на процессоре и для учебных задач этого достаточно, но на практике всё устроено иначе.

Допустим, вы обучаете нейросеть на миллионе изображений. Каждое изображение проходит через сеть, вычисляется функция потерь, корректируются веса. На CPU одна эпоха может занять часы или даже сутки, а на GPU — считанные минуты. Мы видим ускорение в десятки раз.



Татьяна Булгакова для Skillfactory

Дело в архитектуре. CPU — это несколько мощных ядер, которые последовательно справляются со сложными разнородными задачами. GPU — это тысячи маленьких ядер, заточенных под массовые однотипные вычисления. Обучение нейросетей, собственно, и есть такая задача: миллионы операций сложения, умножения, дифференцирования. Именно то, для чего GPU и создан. Чтобы эффективно использовать GPU в PyTorch, достаточно освоить несколько базовых приёмов. В этом уроке мы разберём три ключевых момента.

1. Как проверить, доступна ли видеокарта в вашей системе. Без этого всё остальное бессмысленно.
2. Как размещать тензоры на GPU — создавать их там сразу или переносить с процессора. И как перемещать обратно на CPU.
3. Как избежать самой частой ошибки новичков в PyTorch — когда один тензор лежит на CPU, другой на GPU, и при попытке их сложить всё падает с непонятным сообщением. Научимся эту проблему распознавать и быстро чинить.

И да, всё это будет с примерами кода, которые можно сразу запустить.

Проверка доступности GPU

Прежде чем что-то переносить на видеокарту, хорошо бы убедиться, что она вообще есть. Звучит очевидно, но сколько копий сломано об эту простую проверку...

В PyTorch за работу с видеокартами NVIDIA отвечает модуль → **rch.cuda**. Название привязано к технологии CUDA от NVIDIA — когда PyTorch появился, это был фактический стандарт. Сейчас есть поддержка и других платформ: Apple Silicon через бэкэнд **mps**, AMD через ROCm (но это скорее как совместимости, чем единый модуль). Но в этом уроке мы сосредоточимся на → **rch.cuda** — это самый распространённый и зрелый вариант.

Самая первая и самая важная функция

```
import torch
print(torch.cuda.is_available())
```

Всё. Если вернулось True — у вас есть GPU и PyTorch его видит. Если False — видеокарты нет, или драйверы не установлены, или вы в среде без GPU (например, локально на ноутбуке без дискретной графики).

Ремарка

Возвращает True, только если:

1. PyTorch собран с поддержкой CUDA (не CPU-only версия),
2. в системе есть CUDA-совместимое устройство,
3. драйверы корректно установлены.

В Google Colab по умолчанию GPU отключён. Его нужно включить вручную: выбираем меню «Среда выполнения» → «Сменить среду выполнения» → «Аппаратный ускоритель» → «выбрать GPU». После этого перезапустите среду выполнения и проверьте снова.

А если видеокарта есть, но не одна?

Бывает и такое. В серверных конфигурациях могут стоять несколько GPU. PyTorch умеет с этим работать:

```
# Сколько видеокарт в системе?
print(torch.cuda.device_count())
```

Вернётся число — 1, 2, 4 или больше. Если видеокарты нет, вернётся 0.

Хочется узнать, что это за карта; любопытство — хорошее качество. Посмотреть название можно так:

```
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0)) # название первой карты
    if torch.cuda.device_count() > 1:
        print(torch.cuda.get_device_name(1)) # название второй
```

В Colab на бесплатном тарифе чаще всего дают NVIDIA T4. Не самая новая карта, но для учебных проектов хватает за глаза.

Создание тензора на устройстве

GPU у нас есть. Мы его проверили и убедились, что PyTorch его видит. Теперь нужно научиться им пользоваться.

В PyTorch тензор можно создать на видеокарте сразу, а можно сначала создать на CPU и потом перенести. Рассмотрим оба способа.

Способ 1. Создать тензор сразу на GPU

При создании тензора указываем параметр **device**:

```
import torch
t = torch.tensor([1, 2, 3], device='cuda')
print(t)
print(t.device) # cuda:0
```

Тензор появился на GPU, и все операции с ним будут выполняться там же.

Обратите внимание на **cuda : 0**. Цифра после двоеточия обозначает номер видеокарты. Если карта одна, она всегда будет под номером 0. Если карт несколько, можно написать **device = 'cuda : 1'**, и тензор окажется на второй.

Способ 2. Создать на CPU, потом перенести

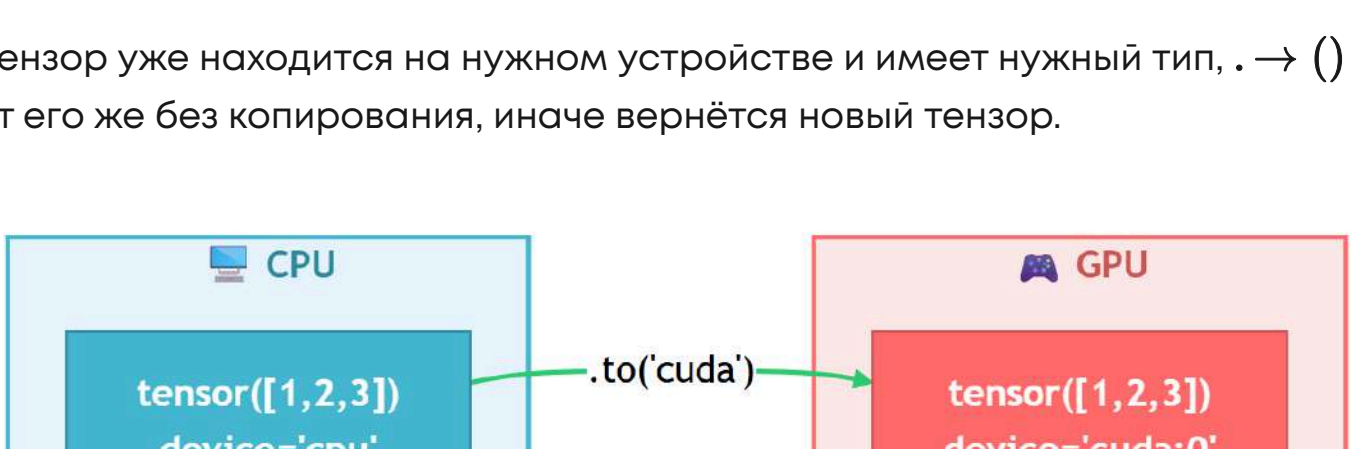
Часто данные изначально появляются на CPU: вы загрузили картинки с диска или сгенерировали массив в NumPy, тогда тензор создаётся обычным способом, а затем переносится на GPU методом .→ ():

```
t_cpu = torch.tensor([1, 2, 3])
print(t_cpu.device) # cpu
t_gpu = t_cpu.to('cuda')
print(t_gpu.device) # cuda:0
```

.→ () умеет не только перемещать тензоры между устройствами, но и менять тип данных:

```
t = torch.tensor([1, 2, 3])
t_float = t.to(torch.float32) # float, но всё ещё на CPU
t_float_gpu = t_float.to('cuda') # float и на GPU
```

Если тензор уже находится на нужном устройстве и имеет нужный тип, .→ () вернёт его же без копирования, иначе вернётся новый тензор.



Татьяна Булгакова для Skillfactory

Перенос обратно на CPU

Допустим, вы посчитали что-то на GPU, а теперь хотите сохранить результат на диск или нарисовать график. NumPy работает только с CPU, поэтому тензор нужно вернуть на процессор:

```
t_gpu = torch.tensor([1, 2, 3], device='cuda')
# Какие-то вычисления на GPU ...
t_cpu = t_gpu.to('cpu')
print(t_cpu.device) # cpu
```

Всё симметрично.

Проверка устройства

Убедиться, что тензор находится там, где вы ожидаете, можно через атрибут **.device**:

```
t = torch.tensor([1, 2, 3])
print(f'Тензор на {t.device}')
```

Для CPU увидите **cpu**, для GPU что-то вроде **cuda : 0**.

✖

Важно: все тензоры в операции должны находиться на одном устройстве. Если попытаться сложить тензор на CPU с тензором на GPU, PyTorch выдаст ошибку:

```
RuntimeError: Expected all tensors to be on the same device
```

Перед выполнением операции всегда проверяйте .device тензоров и при необходимости переносите их с помощью .to('cuda') или .to('cpu').

Вопрос 2.1

0 points possible (ungraded)

Какая функция в PyTorch позволяет проверить, доступен ли GPU?

☐ torch.gpu.is_available()

☒ torch.cuda.is_available()

☐ torch.device.is_available()

☐ torch.cuda.check()

✔

Show answer

Отправить

✔ Верных

Вопрос 2.2

0 points possible (ungraded)

У вас есть два тензора:

- a находится на CPU
- b находится на GPU

Что произойдёт при выполнении операции a + b?

☐ PyTorch автоматически перенесёт один из тензоров, и операция выполнится

☐ Операция выполнится на CPU

☒ Возникнет ошибка, так как тензоры находятся на разных устройствах

☐ Операция выполнится на GPU

✔

Show answer

Отправить

✔ Верных

3/12 Autograd: автоматическое дифференцирование

текущий урок

Введение

До сих пор мы учились создавать тензоры и управлять устройствами. Всё это важно, но одна из ключевых возможностей PyTorch — автоматическое вычисление производных.

Зачем это нужно? При обучении нейросети мы меняем веса в сторону уменьшения ошибки. Чтобы понять, в какую сторону менять каждый вес, нужно знать частные производные ошибки по этим весам. В нейросетях с миллионми параметров считать их вручную невозможно.

Autograd — подсистема PyTorch, которая автоматически строит вычислительный граф и собирает производные. Вам нужно только указать, какие тензоры вас интересуют, и вызвать `.backward()`.

В этом уроке разберёмся:

- как включать отслеживание градиентов через `requires_grad`,
- как устроен вычислительный граф,
- как работает `.backward()` и что хранится в `.∇`,
- почему градиенты накапливаются и когда их обнулять,
- какие ошибки встречаются чаще всего.

Тензоры с градиентами: requires_grad

По умолчанию тензоры создаются без возможности считать градиенты. Чтобы включить отслеживание градиентов, добавляем параметр `requires_grad=True`:

```
import torch

# Обычный тензор, градиенты не считаются
x1 = torch.tensor([1.0, 2.0, 3.0])
print(x1.requires_grad) # False

# Тензор, для которого будем считать градиенты
x2 = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
print(x2.requires_grad) # True
```

Когда у тензора включён `requires_grad`, PyTorch начинает запоминать все операции с ним и строит вычислительный граф, о котором мы говорили во введении.

Золотое правило: `requires_grad=True` нужно только для тех тензоров, которые мы будем обучать (обычно веса модели). Входные данные и константы обычно задаются с `requires_grad=False` — это экономит память.

```
# Входные данные — градиенты не нужны
inputs = torch.tensor([1.0, 2.0], [3.0, 4.0]), requires_grad=False)

# Веса модели — градиенты нужны
weights = torch.randn(2, 5, requires_grad=True)
```

Включить или выключить после создания можно методом `.requires_grad_()`:

```
x = torch.tensor([1.0, 2.0, 3.0])
x.requires_grad_() # включили
x.requires_grad_() # выключили
```

 **Важное предостережение**

Если тензор с `requires_grad=True` участвует в операции, результат тоже будет с `requires_grad=True`. Это свойство распространяется по графу вычислений.

```
a = torch.tensor([1.0], requires_grad=True)
b = torch.tensor([2.0], requires_grad=False) # градиенты не нужны
c = a * b # a вот тут интересно...
print(c.requires_grad) # True!
```

Почему? Потому что для вычисления градиента по `a` нам нужно знать, как устроена операция сложения. И результат этой операции (`c`) хранит эту информацию через `grad_fn`. Так что если среди операндов есть хотя бы один с `requires_grad=True`, то результат операции тоже получит `requires_grad=True`, даже если все остальные операнды имеют `requires_grad=False`.

Вычислительный граф

Когда мы выполняем операции с тензорами, у которых `requires_grad=True`, PyTorch строит вычислительный граф. Каждый узел в этом графе — операция или исходное значение.

Что такое вычислительный граф

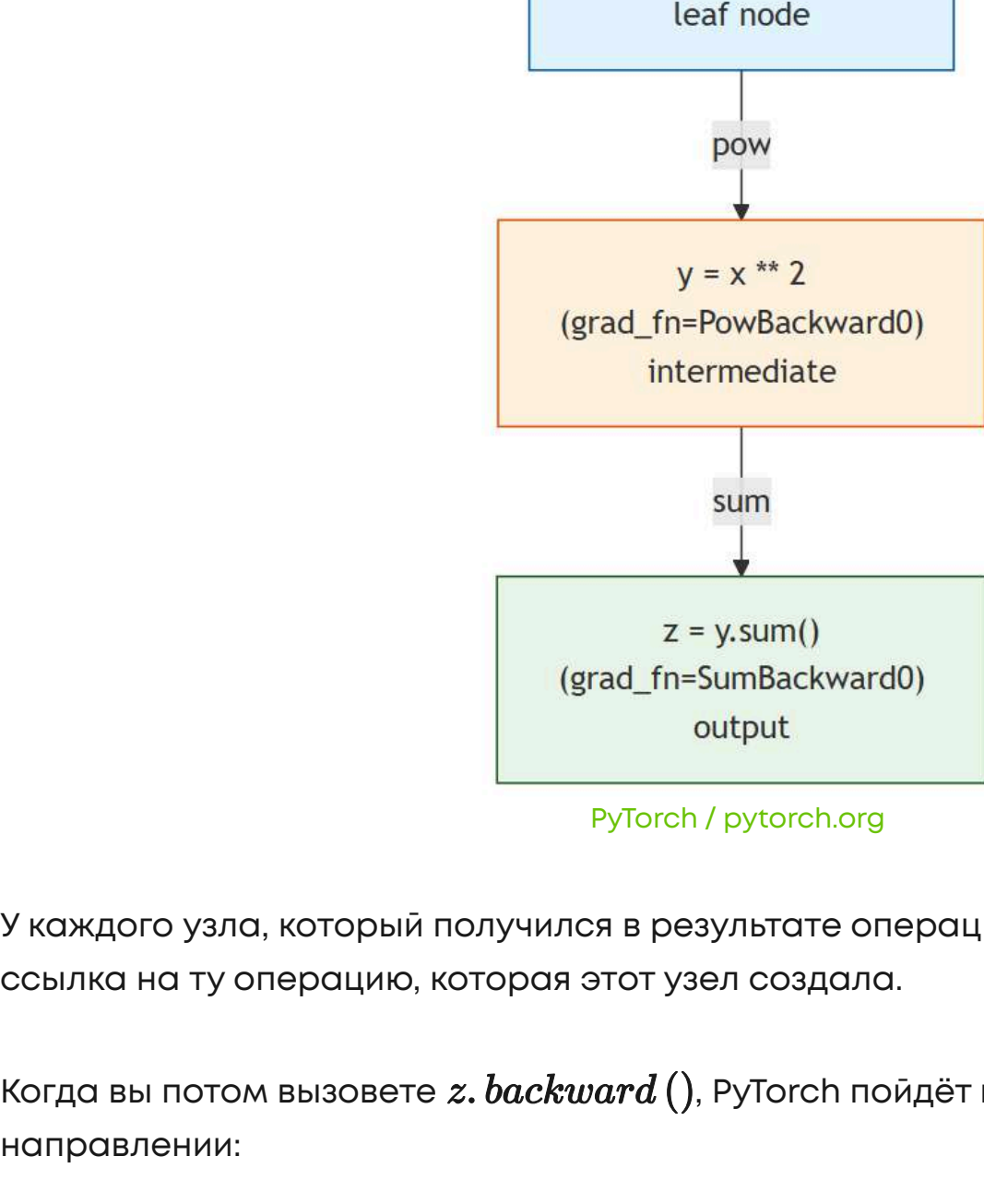
Представьте, что вы решаете пример по математике и записываете каждый шаг на бумаге. В конце у вас получается цепочка вычислений, где видно, что из чего следует.

Вычислительный граф устроен похожим образом, только в виде схемы. Каждый узел в этой схеме — это либо исходный тензор, либо операция. Стрелки показывают, как данные перетекают от одного шага к следующему.

Давайте посмотрим на простом примере:

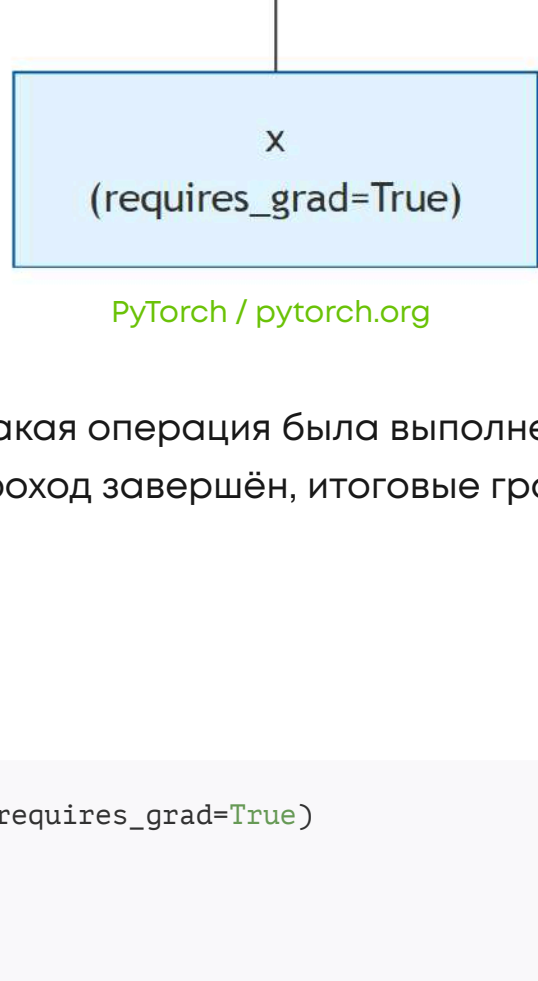
```
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = x ** 2
z = y.sum()
```

В этот момент PyTorch строит граф:



У каждого узла, который получился в результате операции, появляется `∇`. Это ссылка на ту операцию, которая этот узел создала.

Когда вы потом вызовете `z.backward()`, PyTorch пойдёт по этому графу в обратном направлении:



На каждом шаге он знает, какая операция была выполнена, и применяет правило цепочки. Когда обратный проход завершён, итоговые градиенты записываются в `.∇` исходных тензоров.

Смотрим на `grad_fn`

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
a = x * 2
b = a + 1
c = b ** 2
d = c.sum()

print(x.grad_fn) # None, потому что это исходный тензор, он не создан никакой операцией
print(a.grad_fn) # <MulBackward0 object at 0x...>
print(b.grad_fn) # <AddBackward0 object at 0x...>
print(c.grad_fn) # <PowBackward0 object at 0x...>
print(d.grad_fn) # <SumBackward0 object at 0x...>
```

У каждого тензора, кроме исходного, есть `∇`. По названиям легко понять, какая операция его создала:

- `MBBackward0` — умножение,
- `AddBackward0` — сложение,
- `PowBackward0` — возведение в степень,
- `SumBackward0` — суммирование.

Понимание вычислительного графа помогает отлаживать код. Если вы видите, что у тензора появилось `∇`, хотя вы этого не планировали, значит он участвовал в операциях с тензорами, у которых `requires_grad=True`. Это может приводить к лишнему расходу памяти и замедлению вычислений.

После вызова `.backward()` граф по умолчанию удаляется. Это сделано для экономии памяти.

Вычисление градиентов: backward()

Мы научились включать отслеживание градиентов и знаем, что PyTorch строит вычислительный граф. Теперь пора заставить этот граф работать и получить настоящие градиенты.

Для этого у скалярного тензора вызывается метод `.backward()`. Он запускает обратное распространение — проход по графу от конца к началу с вычислением всех производных.

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
print(x.grad) # None — градиентов ещё нет
y = (x ** 2).sum() # z — скаляр (одно число)
z.backward()
print(x.grad) # tensor([4., 6.])
```

Форма `.grad` всегда совпадает с формой исходного тензора. Градиенты хранятся в `.grad` до тех пор, пока их не обнулят.

Важные детали и типичные ошибки

1. **Градиенты накапливаются.** Если вызвать `.backward()` дважды, градиенты прибавятся к уже существующим.

```
x = torch.tensor([2.0], requires_grad=True)
(x ** 2).backward()
print(x.grad) # 4.0
(x ** 3).backward()
print(x.grad) # 16.0 (4 + 12)
```

Поэтому перед каждым шагом нужно обнулять градиенты: `x.grad.zero_()`.

2. **Нельзя дважды вызвать backward без retain_graph**

Как мы помним, после вызова `.backward()` вычислительный граф по умолчанию удалётся. Это экономит память.

```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2
y.backward() # граф построен, градиенты посчитаны, граф удалён
y.backward() # OOMError! графа больше нет
```

Если вам действительно нужно дважды вызвать `backward` (например, для отладки), используйте `retain_graph=True`:

```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2
y.backward(retain_graph=True) # граф сохраняется
print(x.grad) # 4.0
x.grad.zero_()
y.backward() # теперь можно снова
```

Но на практике это нужно редко. Обычно вы строите новый граф на каждой итерации обучения.

3. **torch.no_grad()**

Когда модель уже обучена и вы просто хотите получить предсказание (инференс), градиенты не нужны. Более того, они вредны — занимают память и замедляют вычисления.

```
x = torch.tensor([2.0, 3.0], requires_grad=True)

with torch.no_grad():
    y = x ** 2
    print(y.requires_grad) # False!
```

Внутри этого блока все операции работают как обычно, но граф не строится, и градиенты не считаются.

Ошибка 1. Забыли requires_grad. Если вызвать `.backward()` у тензора без `requires_grad`, будет ошибка. Добавьте `requires_grad=True` при создании или включите позже.

Ошибка 2. Не обнулили градиенты

```
x = torch.tensor([2.0], requires_grad=True)

for i in range(3):
    y = x ** 2
    y.backward()
    print(f"Шаг {i}, градиент: {x.grad.item()}")
```

Шаг 0, градиент: 4.0

Шаг 1, градиент: 8.0 (накопилось!)

Шаг 2, градиент: 12.0 (накопилось ещё!)

Градиенты накапливаются. Как исправить? Обнулять градиенты после каждого шага:

```
for i in range(3):
    y = x ** 2
    y.backward()
    print(f"Шаг {i}, градиент: {x.grad.item()}")
    x.grad.zero_() # обязательно!
```

Ошибка 3. Пытаемся превратить в NumPy тензор с градиентами

NumPy не умеет работать с тензорами, у которых есть градиенты.

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
arr = x.numpy() # OSError!
```

Текст ошибки: "RuntimeError: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy() instead."

PyTorch даже подсказывает, что делать! Надо сначала «отцепить» градиенты методом `.detach()`:

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
arr = x.detach().numpy() # теперь работает
```

`.detach()` создаёт новый тензор с теми же данными, но без привязки к графу вычислений. Этот новый тензор уже можно превращать в NumPy.

Ошибка 4. Градиенты на разных устройствах

Все тензоры с градиентами должны быть на одном устройстве. Проверяйте `.device`.

Вопрос 3.1

0 points possible (ungraded)

Что делает параметр `requires_grad=True` при создании тензора?

☐ Ускоряет вычисления на GPU

☐ Позволяет изменять форму тензора

☒ Включает отслеживание операций для вычисления градиентов

☐ Автоматически обнуляет градиенты

✓

Show answer

Отправить

✓ Верных

Вопрос 3.2

0 points possible (ungraded)

Вы написали следующий код:

```
x = torch.tensor([2.0], requires_grad=True)

for i in range(3):
    y = x ** 2
    y.backward()
    print(x.grad.item())
```

Что произойдёт при выполнении?

☐ На каждом шаге будет выводиться одно и то же значение градиента

☐ Код вызовет ошибку при втором вызове .backward()

☒ Градиенты будут накапливаться, и на каждом шаге значение будет увеличиваться: 4.0, 8.0, 12.0

☐ Градиенты будут автоматически обнуляться после каждого шага

✓

Show answer

Отправить

✓ Верных

В уроке «Autograd: автоматическое дифференцирование» мы разобрались с autograd. Теперь применим это на практике: реализуем линейную регрессию, используя только тензоры и градиенты. Никаких готовых слоев и оптимизаторов. Цель — увидеть, как выглядят все этапы обучения «под капотом», чтобы потом оценить, сколько работы берут на себя высокоуровневые инструменты PyTorch.

Практика_4.ipynb

Генерация данных

Мы знаем, что идеальная линейная зависимость выглядит как $y = wx + b$. Возьмём для примера $w = 2.0$ и $b = 1.0$:

```
import torch
import matplotlib.pyplot as plt

# фиксируем seed для воспроизводимости
torch.manual_seed(42)

# истинные параметры (мы их знаем, модель - нет)
true_w = torch.tensor([2.0])
true_b = torch.tensor([1.0])

# генерируем входные данные: 100 токенов от 0 до 10
x = torch.linspace(0, 10, 100).reshape(-1, 1)

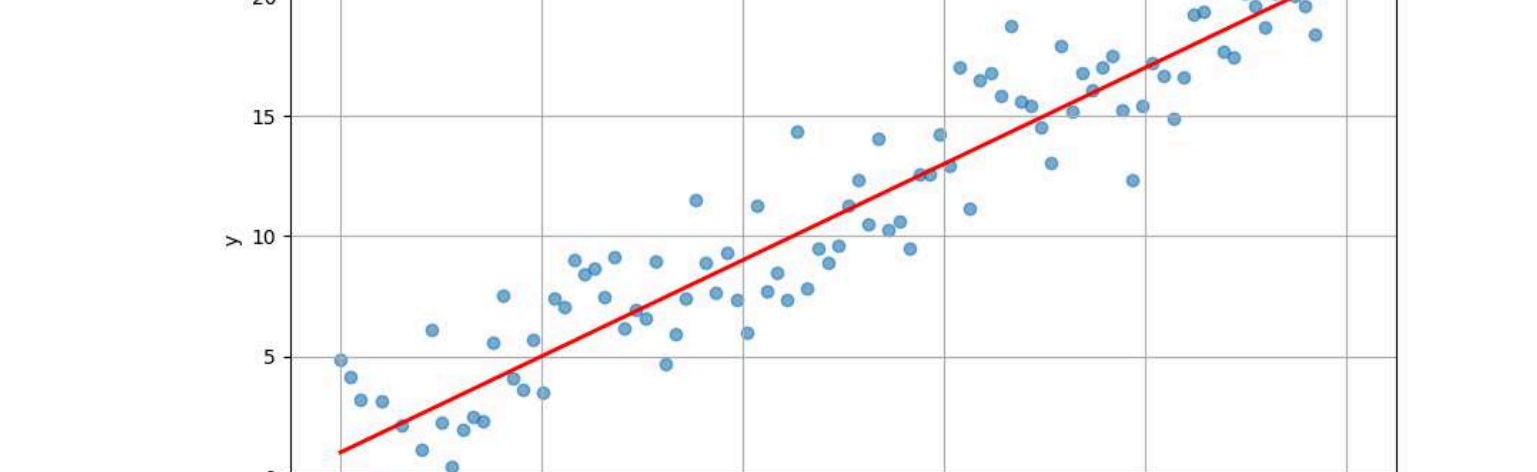
# считаем идеальные значения без шума
y_ideal = true_w * x + true_b
```

Если сейчас вывести y_{ideal} , мы получим идеальную прямую. Но в реальной жизни данные всегда зашумлены. Добавим случайный шум:

```
# добавляем нормальный шум со стандартным отклонением 2
y = y_ideal + torch.randn_like(x) * 2
```

Смотрим, что получилось:

```
plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.6, label='Данные с шумом')
plt.plot(x, y_ideal, 'r-', linewidth=2, label='Идеальная прямая (невидима для модели)')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```



Татьяна Булгакова для Skillfactory

Важный момент: данные пока на CPU. Позже увидим, как перенести их на GPU.

Инициализация модели и прямой проход

На старте мы не знаем, какие должны быть w и b . Поэтому задаём их случайно. Единственное важное условие: для этих тензоров нужно включить `requires_grad=True`, потому что мы будем по ним считать градиенты и обновлять их.

```
# инициализируем веса случайно (стандартное нормальное распределение от -1 до 1)
w = torch.randn(1, requires_grad=True)
b = torch.randn(1, requires_grad=True)

print(f'Начальный вес w: {w.item():.4f}')
print(f'Начальное смещение b: {b.item():.4f}')
```

Функция `torch.randn` даёт случайное число из нормального распределения (среднее 0, дисперсия 1). Можно использовать и другие способы инициализации, но для линейной регрессии этого достаточно.

Обратите внимание на `.item()` — это способ достать одно число из тензора размера 1, удобно для вывода.

Прямой проход — это вычисление предсказаний модели при текущих весах. Для линейной регрессии формула проста: $y_{pred} = w * x + b$

```
# прямой проход: считаем предсказания
y_pred = w * x + b
```

Для тех, кто знает NumPy: broadcasting работает точно так же, как вы привыкли.

Полезно сразу визуализировать предсказания модели до обучения. Это поможет потом сравнить «до» и «после».

```
plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.6, label='Данные с шумом')
plt.plot(x, y_pred.detach().numpy(), 'g-', linewidth=2, label='Предсказание модели (до обучения)')
plt.plot(x, y_ideal, 'r-', linewidth=2, label='Идеальная прямая')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```



Татьяна Булгакова для Skillfactory

Обратите внимание на `.detach().numpy()`. Мы используем `.detach()`, чтобы отцепить градиенты перед превращением в NumPy (помните ошибку из урока «Autograd: автоматическое дифференцирование»)? Без этого `plt.plot` упадёт с ошибкой.

Скорее всего, зелёная линия будет проходить совсем не там, где нужно. Это нормально: модель ещё ничему не научилась.

Функция потерь (loss function)

Модель сделала предсказание — зелёная линия на графике. Пока она далека от идеала. Но как измерить это математически? Нужна функция потерь. Функция потерь показывает, насколько сильно модель ошибается. Она принимает два аргумента — предсказания модели и правильные ответы — и возвращает одно число: величину ошибки.

Наша задача — это число минимизировать. Для линейной регрессии классический выбор — среднеквадратичная ошибка. Считается она просто:

```
# MSE своими руками
loss = ((y_pred - y) ** 2).mean()
```

Разберём по шагам:

- `y_pred` — y -разница между предсказанием и истиной (тензор из 100 чисел);
- `** 2` — возведение в квадрат (чтобы отрицательные ошибки не компенсировали положительные);
- `.mean()` — среднее арифметическое по всем элементам.

В результате получаем одно число — скаляр. Именно скаляр нужен для `.backward()`.

Добавим вычисление `loss` в код и посмотрим, какая ошибка у случайно инициализированной модели:

```
# прямой проход (уже сделали)
y_pred = w * x + b

# считаем loss
loss = ((y_pred - y) ** 2).mean()

print(f'Текущий loss: {loss.item():.4f}')
```

Число будет довольно большим, и это нормально: модель ещё не училась.

Обратный проход и шаг градиентного спуска

Loss посчитали. Теперь нужно понять, как изменить w и b , чтобы `loss` уменьшился. Здесь вступает в дело `autograd`. Запускаем обратное распространение:

```
# обратный проход
loss.backward()
```

После этого вызова в тензорах w и b появились градиенты. Давайте посмотрим на них:

```
print(f'Градиент по w: {w.grad.item():.4f}')
print(f'Градиент по b: {b.grad.item():.4f}')
```

Важно: `.backward()` прошёл по всему графу и посчитал производные. Градиент показывает, в какую сторону и насколько сильно нужно изменить параметр, чтобы `loss` вырос. А нам нужно `loss` уменьшить, поэтому мы будем двигаться в противоположную сторону.

Выбираем `learning rate`.

Скорость обучения (`learning rate`) — один из самых важных параметров. Слишком маленький шаг — обучение будет медленным. Слишком большой — можно перелететь минимум и `loss` начнёт расти.

Для начала возьмём 0.01:

```
learning_rate = 0.01

# сохраняем старые значения для сравнения
w_before = w.item()
b_before = b.item()

# обновляем веса
with torch.no_grad(): # отключаем градиенты для операции обновления
    w -= learning_rate * w.grad
    b -= learning_rate * b.grad

print(f'w: {w_before:.4f} -> {w.item():.4f}')
print(f'b: {b_before:.4f} -> {b.item():.4f}')
```

Обратите внимание на `with torch.no_grad()`. Это важно! Когда мы обновляем веса, мы не хотим, чтобы PyTorch строил граф вычислений для этих операций. Нам не нужно считать производные от обновления весов. После обновления весов нужно убедиться, что наши действия принесли пользу:

```
# снова считаем предсказания и loss с новыми весами
y_pred_new = w * x + b
loss_new = ((y_pred_new - y) ** 2).mean()

print(f'Loss до обновления: {loss.item():.4f}')
print(f'Loss после обновления: {loss_new.item():.4f}')
```

Loss должен стать меньше. Если нет — возможно, `learning rate` слишком большой. После обновления весов нужно обязательно обнулить градиенты. Иначе при следующем вызове `backward()` новые градиенты прибавятся к старым.

```
# обнуляем градиенты
w.grad.zero_()
b.grad.zero_()

print(f'Градиент w после обновления: {w.grad}') # tensor([0.])
```

Цикл обучения

Один шаг градиентного спуска немного улучшил модель, но до идеала ещё далеко. Нужно повторить эти шаги много раз — это и есть обучение. Собираем всё в цикл

Вот полный код обучения линейной регрессии с нуля:

```
import torch
import matplotlib.pyplot as plt

# фиксируем seed для воспроизводимости
torch.manual_seed(42)

# 1. генерируем данные
true_w = torch.tensor([2.0])
true_b = torch.tensor([1.0])
x = torch.linspace(0, 10, 100).reshape(-1, 1)
y = true_w * x + true_b + torch.randn_like(x) * 2

# 2. инициализируем веса
w = torch.randn(1, requires_grad=True)
b = torch.randn(1, requires_grad=True)

# 3. задаём гиперпараметры
learning_rate = 0.01
epochs = 100

# для сохранения истории loss
loss_history = []

# 4. цикл обучения
for epoch in range(epochs):
    # прямой проход
    y_pred = w * x + b

    # считаем loss
    loss = ((y_pred - y) ** 2).mean()
    loss_history.append(loss.item())

    # обратный проход
    loss.backward()

    # обновляем веса (без градиентов)
    with torch.no_grad():
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad

    # обнуляем градиенты
    w.grad.zero_()
    b.grad.zero_()

    # печатаем прогресс каждые 20 эпох
    if (epoch + 1) % 20 == 0:
        print(f'Эпоха {epoch + 1}/{epochs}, Loss: {loss.item():.4f}')
```

Смотрим на результаты. Посмотрим, что получилось после обучения:

```
print(f'Истинный w: {true_w.item():.4f}, Полученный w: {w.item():.4f}')
print(f'Истинный b: {true_b.item():.4f}, Полученный b: {b.item():.4f}')
print(f'Финальный loss: {loss.item():.4f}')
```

Скорее всего, w будет близко к 2.0, а b — к 1.0. Но из-за шума точного совпадения не будет — и не должно быть.

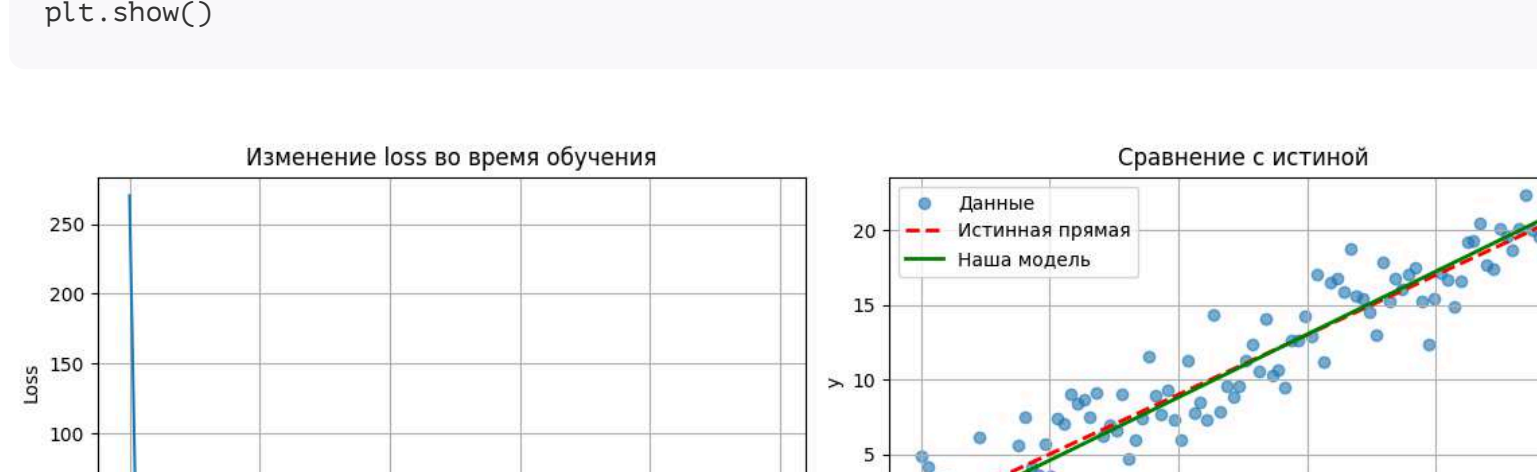
Построим два графика — как менялся `loss` и как выглядела финальная модель:

```
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(loss_history)
plt.xlabel('Эпоха')
plt.title('Изменение loss во время обучения')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.scatter(x, y, alpha=0.6, label='Данные')
plt.plot(x, (true_w * x + true_b).detach(), 'r-', linewidth=2, label='Истинная прямая')
plt.plot(x, (w * x + b).detach(), 'g-', linewidth=2, label='Наша модель')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.title('Сравнение с истинной')

plt.tight_layout()
plt.show()
```



Татьяна Булгакова для Skillfactory

На левом графике `loss` должен постепенно уменьшаться и выходить на плато. На правом зелёная линия должна лечь близко к красной.

Итоги урока

В этом уроке мы реализовали полный цикл обучения линейной регрессии с нуля, используя только тензоры и `autograd`. Научились генерировать синтетические данные с фиксированным `seed` для воспроизводимости результатов. Освоили прямой проход — вычисление предсказаний модели при текущих весах. Разобрались с функцией потерь MSE и тем, почему `loss` обязательно должен быть скаляром. Запустили обратное распространение через `backward()` и смотрели на градиенты в `.grad`. Научились обновлять веса вручную и поняли, зачем нужно обнулять градиенты после каждого шага. И наконец, собрали всё это в цикл обучения и увидели, как модель постепенно приближается к истинным параметрам.

Введение

В «Практике: линейная регрессия «с нуля»» мы сделали важную вещь: реализовали линейную регрессию с нуля, используя только тензоры и autograd. Сами создавали веса, сами писали прямой проход, сами обновляли параметры. Это был полезный опыт: вы увидели, как работает градиентный спуск изнутри.

Однако на практике ручное управление каждым параметром используют только в специальных случаях.

Представьте, что вам нужно построить сеть с 50 слоями. Писать для каждого слоя свои веса, вручную прописывать все преобразования — это тысячи строк кода, в которых легко запутаться. И в большинстве случаев в этом нет необходимости: PyTorch уже предоставляет готовые механизмы для работы со слоями и параметрами.

В этом уроке мы познакомимся с `torch.nn.Module` — базовым классом для всех нейросетей в PyTorch. Именно он превращает объёмный ручной код в несколько понятных строк.

Что такое nn.Module

Чтобы создать свою модель, нужно унаследовать класс от `nn.Module` и реализовать два метода:

- `__init__` — объявляем слои,
- `forward` — описываем прямой проход.

```
import torch, nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__() # вызовем конструктор родителя
        # здесь будут объявляться слои

    def forward(self, x):
        # здесь описывается прямой проход
        return x
```

Что даёт наследование от `nn.Module`

Когда вы наследуете от `nn.Module`, происходит несколько важных вещей.

1. Все объекты, которые вы объявите как атрибуты класса (например, `self.linear = nn.Linear(1, 1)`), автоматически становятся параметрами модели. PyTorch регистрирует их внутри модели, и они будут доступны через `model.parameters()`. Вам не нужно вручную собирать списки весов.

2. Метод `forward` определяет, как модель применяется к данным. При этом PyTorch автоматически строит вычислительный граф — так же, как мы видели в уроке «Autograd: автоматическое дифференцирование». Обратите внимание: вызывать `forward` напрямую не нужно — в коде вы пишете `model(x)`, а PyTorch автоматически вызывает `forward` за вас.

3. Появляются полезные методы: `parameters()`, `zero_grad()`, `to()`, `eval()` и другие. Всё необходимое для обучения уже есть.

Даже пустая модель уже работает:

```
model = MyModel()
print(model) # MyModel()
```

Но пользы от неё пока нет. В следующем блоке добавим первый настоящий слой.

Главная идея `nn.Module` — инкапсуляция. Модель сама знает свои параметры, сама умеет считать прямой проход, сама может обнулить градиенты. Вам не нужно думать о том, как устроены отдельные веса. Вы работаете с моделью как с целым объектом.

В «Практике по линейной регрессии» мы писали:

```
w_grad.zero_()
b_grad.zero_()
```

С `nn.Module` будет одна строка:

```
model.zero_grad_()
```

Разница не только в количестве кода. Если в модели 100 слоёв, ручное обнуление становится неуправляемым. А один вызов метода работает при любом числе параметров.

nn.Linear — первый настоящий слой

В прошлом блоке мы создали пустую модель-заготовку, пора добавить в неё полезное содержимое. Добавляем слой в модель:

```
import torch
import torch.nn as nn

class LinearRegression(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)
```

Теперь это полноценная модель линейной регрессии.

Что происходит внутри

Когда вы создаёте $\Gamma, L \in \text{eag}([1, 1])$, PyTorch автоматически:

- создаёт два тензора: вес (W) и смещение (b);
- поменяет их как `requires_grad_ = True`;
- инициализирует веса по методу `kaiming_uniform` (учитывает размерность слоя, чтобы сигнал не затухал и не взрывался при прохождении через сеть).

Данные из Практики по линейной регрессии работают без изменений:

```
# Данные (те же, что в практике по линейной регрессии)
x = torch.linspace(0, 10, 100).reshape(-1, 1)

# Прямой проход
y_pred = model(x)
print(y_pred.shape) # torch.Size([100, 1])

# Инициализируем модель
x = torch.linspace(0, 10, 100).reshape(-1, 1)
y_pred = model(x)
print(y_pred.shape) # torch.Size([100, 1])
```

Модель ведёт себя как функция: подали вход, получили предсказание.

В предыдущих блоках мы писали так:

```
w = torch.randn(1, requires_grad=True)
b = torch.randn(1, requires_grad=True)
y_pred = w * x + b

# Теперь то же самое выглядит короче:

model = LinearRegression()
y_pred = model(x)
```

Результат тот же, а кода меньше, и если понадобится более сложная модель, мы просто добавим ещё слои.

Параметры модели

Когда мы использовали ручную реализацию в «Практике по линейной регрессии», у нас было две отдельные переменные: w и b . Мы сами за них отвечали, сами доставляли их для обновления, сами следили за их состоянием.

В модели на основе `nn.Module` параметры живут внутри, и PyTorch предоставляет удобные способы посмотреть, что там лежит.

У любой модели есть метод `parameters()`:

```
model = LinearRegression()

for param in model.parameters():
    print(param.shape)
    print(param.data)
```

В нашем случае будет два параметра: вес и смещение. А если нужно ещё и понять, какой параметр за что отвечает, используйте `named_parameters()`:

```
for name, param in model.named_parameters():
    print(f"({name}): {param.shape}")

# Bias
# Linear.weight: torch.Size([1, 1])
# Linear.bias: torch.Size([1])
```

Имена собираются автоматически: название слоя + название параметра внутри него. Когда в модели десятки слоёв, это очень помогает ориентироваться.

Снимок состояния модели

Метод `state_dict()` возвращает полное содержимое модели в виде словаря:

```
print(model.state_dict())

OrderedDict(
  ('linear.weight', tensor([[0.2451]])),
  ('linear.bias', tensor([0.1245]))
)
```

Этот словарь понадобится нам позже для сохранения моделей и их загрузки.

Использование модели

Модель создана, параметры внутри неё есть. Теперь разберём, как с ней работать в коде.

В PyTorch модель является вызываемым объектом. Это значит, что для получения предсказаний достаточно передать данные прямо в модель:

```
# Данные из урока 4
x = torch.linspace(0, 10, 100).reshape(-1, 1)

# Прямой проход
y_pred = model(x)
```

Когда вы пишете `model(x)`, PyTorch автоматически вызывает метод `forward`, который мы определили в классе.

Батчевая обработка

Обратите внимание на форму тензоров:

- $x.shape = (100, 1)$ — 100 объектов, каждый с одним признаком,
- $y_{pred}.shape$ — тоже $(100, 1)$.

Модель обрабатывает сразу все объекты за один проход. Писать цикл по отдельным точкам не нужно: матричные операции внутри линейного слоя делают это эффективнее и быстрее.

Попробуйте изменить количество объектов:

```
x_small = torch.linspace(0, 10, 10).reshape(-1, 1)
y_small = model(x_small)
print(y_small.shape) # torch.Size([10, 1])
```

Количество объектов на входе может быть любым, модель подстроится автоматически.

Сейчас у нас один входной признак. На линейный слой можно создать с произвольным количеством входов и выходов:

```
class WiderModel(nn.Module):
    def __init__(self):
        super().__init__()
        # 10 входов, 5 выходов
        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)

model = WiderModel()
x2 = torch.randn(32, 10) # 32 объекта, 10 признаков
y2 = model(x2)
print(y2.shape) # torch.Size([32, 5])
```

Форма выхода определяется архитектурой слоя, а количество объектов в батче может свободно меняться.

Функция потерь

В «Практике: линейная регрессия «с нуля»» мы писали функцию потерь вручную:

```
loss = ((y_pred - y) ** 2).mean()
```

Это работало, но каждый раз воспроизводить эту формулу неудобно: чем сложнее задача, тем легче допустить ошибку.

Готовая MSE на nn

PyTorch предоставляет готовые функции потерь в модуле `nn`. Для нашей задачи подходит `MSELoss`:

```
import torch, nn as nn

criterion = nn.MSELoss()
```

`criterion` — так принято называть функцию потерь в коде (от англ. `criterion` — критерий оценки).

```
# Предсказания модели
y_pred = model(x)

# Считаем loss одной строкой
loss = criterion(y_pred, y)
```

`MSELoss` делает ровно то же самое, что мы писали руками: $((y_{pred} - y) * 2).mean()$. Разница в том, что код становится чище, меньше шансов ошибиться в скобках и при необходимости легко заменить на другую функцию потерь.

Цикл обучения с nn.Module

У нас есть модель, есть функция потерь, есть данные. Соберём всё вместе в цикл обучения — теперь с использованием `nn.Module`.

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

# Данные (те же, что в «Практике по линейной регрессии»)
torch.manual_seed(42)
train_x = torch.tensor([2.0])
train_y = torch.tensor([1.0])
x = torch.linspace(0, 10, 100).reshape(-1, 1)
y = train_x * 1 + train_y + torch.randn_like(x) * 2

class LinearRegression(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)

# модель и функция потерь
model = LinearRegression()
criterion = nn.MSELoss()

# гиперпараметры
learning_rate = 0.01
epochs = 100
# сохранение истории loss
loss_history = []

for epoch in range(epochs):
    # прямой проход
    y_pred = model(x)
    loss = criterion(y_pred, y)
    loss_history.append(loss.item())

    # обратный проход
    loss.backward()

    # обновление весов (звучит, пока без оптимизатора)
    with torch.no_grad():
        for param in model.parameters():
            param -= learning_rate * param.grad

# обучение закончилось
model.zero_grad_()

if (epoch + 1) % 20 == 0:
    print(f"Эпоха {epoch + 1}/{epochs}, Loss: {loss.item():.4f}")
```

Что изменилось по сравнению с уроком «Практика: линейная регрессия «с нуля» (PyTorch-фокус)?

Действие	Урок «Практика: линейная регрессия «с нуля» (PyTorch-фокус)	Урок «Построение моделей: torch.nn.Module»
Прямой проход	<code>y_pred = w*x + b</code>	<code>y_pred = model(x)</code>
Функция потерь	<code>((y_pred - y) ** 2).mean()</code>	<code>criterion(y_pred, y)</code>
Доступ к параметрам	отдельно <code>w</code> и <code>b</code>	<code>model.parameters()</code>
Обновление	<code>w -= lr * w_grad</code>	цикл <code>for el model.parameters()</code>
Обнуление градиентов	<code>w_grad.zero_()</code> , <code>b_grad.zero_()</code>	<code>model.zero_grad()</code>

Главное преимущество: код не зависит от количества параметров. Будет их два или два миллиона — цикл обучения останется тем же.

Сохранение модели

Модель обучена, веса подобраны. Но после закрытия ноутбука всё исчезнет. Чтобы сохранить результат, модель нужно записать в файл.

Параметры модели хранятся в специальном словаре — `state_dict()`:

```
# сохраняем параметры в файл
torch.save(model.state_dict(), 'linear_model.pth')
```

Чтобы загрузить параметры, нужно сначала создать модель с той же архитектурой, а потом заполнить её параметрами из файла:

```
# создаём модель с той же архитектурой
loaded_model = LinearRegression()

# загружаем параметры
loaded_model.load_state_dict(torch.load('linear_model.pth', weights_only=True))

# Теперь loaded_model — это точная копия обученной модели. Проверяем, что загрузилось

# сравниваем предсказания
with torch.no_grad():
    original_pred = model(x)
    loaded_pred = loaded_model(x)

print(torch.allclose(original_pred, loaded_pred)) # True
```

Почему мы сохраняем `state_dict`, а не всю модель? Можно сохранить и всю модель целиком:

```
# так тоже можно, но не рекомендуется
torch.save(model, 'model_full.pth')
```

Проблема в том, что при загрузке такой модели могут возникнуть конфликты версий и другие неочевидные ошибки. `state_dict` — это просто числа, они загружаются предсказуемо и без побочных эффектов.

Вопрос 5.1

0 points possible (ungraded)

Что такое батч в контексте обучения моделей?

☐ Один признак одного объекта

☐ Отдельный слой нейросети

☒ Группа объектов, которые модель обрабатывает одновременно

☐ Один шаг градиентного спуска

✓

Отправить

Верных

Вопрос 5.2

0 points possible (ungraded)

У вас есть тензор входных данных x с формой $(32, 10)$. Что это означает?

☐ 10 объектов, каждый с 32 признаками

☒ 32 объекта, каждый с 10 признаками

☐ 32 слоя по 10 параметров

☐ 10 батчей по 32 объекта

✓

Отправить

Верных

Итоги

В этом уроке мы сделали важный шаг: перешли от ручной реализации к использованию готовых компонентов PyTorch.

Главное, что мы освоили

- Познакомились с `nn.Module` — базовым классом для всех нейросетей, который берёт на себя управление параметрами и упрощает создание моделей.
- На примере `nn.Linear` узнали, как работают готовые веса. Теперь не нужно вручную создавать веса и смещения: слой делает это сам, сразу с корректной инициализацией и `requires_grad_ = True`.
- Научились работать с параметрами модели через `model.parameters()` и `named_parameters()`. Это позволяет обращаться ко всем весам единообразно, независимо от их количества и названий.
- Освоили готовые функции потерь, в частности `nn.MSELoss`, которая заменяет ручную формулу и делает код чище.
- Увидели, чем цикл обучения с моделью отличается от ручной реализации: прямой проход через модель, обнуление градиентов одной строкой, обновление параметров в цикле: код стал короче, а главное — он больше не зависит от количества параметров.
- Научились сохранять обученные модели через `state_dict` и загружать их обратно.

Введение

В уроке «Построение моделей: `torch.nn.Module`» мы научились строить модели через Γ . **Model** и познакомимся с первым готовым слоем — Γ . **L** $\in$ *ear*. Этого достаточно для линейной регрессии, но в реальных задачах требуется больше инструментов.

Линейная регрессия хороша, когда зависимость действительно линейная. А если нет? Если данные устроены сложнее, например нужно отличать кошек от собак, то без нелинейностей не обойтись. Именно их добавляют функции активации.

Кроме того, для каждой задачи нужна своя функция потерь для регрессии одна, для классификации совсем другая. А чтобы модель не переобучалась, применяются приёмы регуляризации вроде Dropout.

В этом уроке мы разберём компоненты, которые понадобятся в следующих уроках:

- функции активации, чтобы модель могла обучаться сложным зависимостям;
- функции потерь — для задач классификации;
- Dropout, чтобы бороться с переобучением;
- Flatten, чтобы подготавливать данные для полносвязных слоёв.

Функции активации

В уроке «Построение моделей: `torch.nn.Module`» наша модель была линейной: $y = wx + b$. Но если составить такую модель из многих слоёв, она всё равно останется линейной: композиция линейных функций даёт линейную функцию, но для большинства реальных задач линейности недостаточно.

Функции активации добавляют нелинейность. Без них нейросеть не смогла бы обучаться сложным зависимостям: распознавать лица, понимать речь, переводить тексты.

ReLU: самая популярная функция активации

```
import torch.nn as nn
activation = nn.ReLU()
```

ReLU (Rectified Linear Unit) работает очень просто: все отрицательные значения превращает в ноль, положительные оставляет как есть.

Математически: $f(x) = \max(0, x)$

ReLU очень быстро вычисляется и не страдает от проблемы затухающих градиентов для положительных значений. Кроме того, она даёт разреженные активации, то есть на выходе получается много нулей. Использовать ReLU можно практически везде в скрытых слоях.

Ссылка на документацию: [torch.nn.ReLU](#)

Sigmoid: для вероятностей

```
sigmoid = nn.Sigmoid()
```

Математически: $f(x) = 1 / (1 + e^{(-x)})$

На выходе Sigmoid всегда число от 0 до 1, поэтому результат можно интерпретировать как вероятность. Для больших положительных чисел функция стремится к 1, для больших отрицательных — к 0. Обычно Sigmoid используют только в выходном слое для бинарной классификации.

Ссылка на документацию: [torch.nn.Sigmoid](#)

Tanh: для диапазона (-1, 1)

```
tanh = nn.Tanh()
```

Tanh (гиперболический тангенс) похож на Sigmoid, но выдаёт значения от -1 до 1.

Математически: $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

В отличие от Sigmoid, Tanh центрирован около нуля, что иногда даёт лучшую сходимость при обучении. Применяют Tanh в скрытых слоях, если нужны значения с центром в нуле.

Ссылка на документацию: [torch.nn.Tanh](#)

Softmax: для многоклассовой классификации

```
softmax = nn.Softmax(dim=1) # dim=1: применен по измерению классов
```

Softmax превращает набор чисел (логиты) в распределение вероятностей. Сумма всех выходов равна 1.

Математически: $\text{softmax}(x)_i = \exp(x_i) / \sum(\exp(x_j))$

Все выходы Softmax положительные и в сумме дают 1, поэтому их удобно интерпретировать как вероятности классов. При этом функция подавляет маленькие значения и усиливает большие. Softmax применяют в выходном слое для многоклассовой классификации.

Ссылка на документацию: [torch.nn.Softmax](#)

Функции потерь

Функция потерь (loss function) показывает модели, насколько сильно она ошибается. От выбора функции потерь зависит, чему именно будет учиться сеть.

В «Практике по линейной регрессии» мы уже использовали *MSELoss* для регрессии. Теперь посмотрим на функции потерь для задач классификации, которые понадобятся в уроке «Практика: MLP для классификации музыкальных жанров».

Для регрессии (повтор)

```
mse_loss = nn.MSELoss() # среднеквадратичная ошибка
l1_loss = nn.L1Loss() # средняя абсолютная ошибка (MAE)
```

Когда использовать: задана предсказание числа (цена дома, температура, координаты).

Документация: [torch.nn.MSELoss](#)
[torch.nn.L1Loss](#)

Для бинарной классификации

Бинарная классификация означает выбор одного из двух классов (да/нет, кот/собака, спам/не спам).

Основные параметры:

```
bce_loss = nn.BCEWithLogitsLoss()
```

Почему не просто *BCELoss*? *BCEWithLogitsLoss* внутри делает две вещи:

1. применяет сигмоиду к выходу модели;
 2. считает бинарную кросс-энтропию.
- Это численно стабильнее, чем делать сигмоиду отдельно, а потом считать loss.

```
# правильно (численно стабильно)
loss_fn = nn.BCEWithLogitsLoss()
loss = loss_fn(logits, targets)

# неправильно (может давать NaN)
prob = torch.sigmoid(logits)
loss = loss_fn(probs, targets)
```

Документация: [torch.nn.BCEWithLogitsLoss](#)
[torch.nn.BCELoss](#)

Для многоклассовой классификации

Многоклассовая классификация означает, что классов больше двух (цифры от 0 до 9, породы собак, типы одежды).

```
ce_loss = nn.CrossEntropyLoss()
```

CrossEntropyLoss внутри делает:

1. применяет softmax к выходу модели;
2. считает кросс-энтропию между предсказаниями и правильными ответами.

Важный момент: *CrossEntropyLoss* ожидает на вход сырые логиты (выход последнего слоя без активаций), а не вероятности. Не нужно самостоятельно применять softmax перед этим loss.

```
# правильно
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, targets) # logits без активации, targets — индексы классов (0, 1, 2, ...)
```

```
# неправильно (softmax будет применён дважды)
prob = torch.softmax(logits, dim=1)
loss = loss_fn(probs, targets)
```

Документация: [torch.nn.CrossEntropyLoss](#)

Регуляризация: Dropout

Dropout — это метод регуляризации, при котором во время обучения случайным образом отключается часть нейронов. Это заставляет сеть не полагаться на отдельные нейроны и снижает переобучение.

Когда модель слишком сложная для имеющихся данных, она начинает переобучаться: запоминает примеры вместо того, чтобы выявлять закономерности. Симптомы: loss на обучении падает, а на валидации перестаёт уменьшаться или растёт. Dropout является одним из самых простых и эффективных способов борьбы с переобучением.

Как работает Dropout

Во время обучения Dropout случайно зануляет часть нейронов с вероятностью *p*. На каждой итерации выбирается новый случайный набор.

```
import torch.nn as nn
dropout = nn.Dropout(p=0.5) # p — вероятность отключения
```

Что при этом происходит:

- на каждом прямом проходе сеть использует только подмножество нейронов;
- модель не может чрезмерно полагаться на отдельные нейроны;
- сеть вынуждена распределять полезную информацию по всем весам.

Во время тестирования Dropout не применяется: используются все нейроны, а их выходы масштабируются на коэффициент $\frac{1}{1-p}$, чтобы компенсировать разницу между обучением и инференсом. В PyTorch это масштабирование выполняется автоматически уже на этапе обучения (режим "inverted dropout"), поэтому при тестировании дополнительных действий с весами не требуется.

Где размещать Dropout

Обычно Dropout ставят после активации в полносвязных слоях:

```
import torch
import torch.nn as nn

class ModelWithDropout(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 128)
        self.dropout2 = nn.Dropout(0.3)
        self.fc3 = nn.Linear(128, 10)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout1(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout2(x)
        x = self.fc3(x)
        return x
```

Перед выходным слоем Dropout обычно не ставят, чтобы не вносить шум в предсказания.

Как выбрать p

- $p = 0.5$ — стандартное значение для крупных сетей;
- $p = 0.2-0.3$ — для небольших сетей, чтобы не отбрасывать слишком много информации;
- если переобучения не наблюдается, Dropout можно не использовать.

Важный нюанс: режимы *train* и *eval*

Dropout ведёт себя по-разному в зависимости от режима модели:

```
model.train() # Dropout активен (нейроны зануляются случайным образом)
# ... обучение ...

model.eval() # Dropout неактивен (все нейроны работают)
# ... валидация / тестирование ...
```

Если забыть переключить режим перед валидацией, результаты будут некорректными: модель продолжит случайно занулять нейроны, и метрики окажутся заниженными и нестабильными от запуска к запуску.

Ссылки на документацию

[torch.nn.Dropout](#)
[torch.nn.Module.train](#)
[torch.nn.Module.eval](#)

Вспомогательные слои: Flatten

До сих пор мы работали с данными, которые уже были в нужной форме: вектор признаков подавался на вход Γ . **L** $\in$ *ear*, и всё было хорошо.

Но в реальных задачах данные часто приходят в другом виде. Например, картинка — это трёхмерный тензор (каналы, высота, ширина). Полносвязный слой ожидает на вход вектор, а не матрицу или тензор. Нужно как-то преобразовать многомерные данные в плоский вектор. Γ . **Flaen** распрямляет многомерный тензор в одномерный вектор, сохраняя при этом размер батча.

```
import torch
import torch.nn as nn
flatten = nn.Flatten()
```

Как это работает:

```
# батч из 32 картинок 32x28x28 (RGB, 28x28 пикселей)
x = torch.randn(32, 3, 28, 28)

# применен Flatten
x_flat = flatten(x)
print(x_flat.shape) # torch.Size([32, 2352])
```

Что произошло? Размер батча (32) остался на месте, все остальные размерности (3, 28, 28) объединились в одну и получились матрица: каждая картинка стала вектором длины 2352.

Flaen обычно ставят между свёрточными и полносвязными слоями:

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # свёрточная часть
        self.conv = nn.Conv2d(3, 16, kernel_size=3)
        self.pool = nn.MaxPool2d(2)

        # переход от свёрток к полносвязным слоям
        self.flatten = nn.Flatten()

        # классификатор
        self.fc = nn.Linear(16 * 15 * 15, 10) # размер нужно посчитать

    def forward(self, x):
        x = self.pool(torch.relu(self.conv(x)))
        x = self.flatten(x) # здесь картинка превращается в вектор
        x = self.fc(x)
        return x
```

Альтернатива: *view* и *reshape*

Того же эффекта можно добиться методами *view* или *reshape*:

```
x_flat = x.view(x.size(0), -1) # -1 означает "посчитай сам"
```

Но Γ . **Flaen** удобнее, потому что это готовый слой, который можно включить в Γ . *Sequential*, при этом не нужно вручную писать *view* в методе *forward*.

Документация

[torch.nn.Flatten](#)
[torch.Tensor.view](#)
[torch.reshape](#)

Вопрос 6.1

0 points possible (ungraded)

Какая функция активации превращает значения в диапазон от 0 до 1 и часто используется для бинарной классификации?

☐ ReLU

☒ Sigmoid

☐ Tanh

☐ Softmax

✓ Show answer

Отправить

✓ Верных

Вопрос 6.2

0 points possible (ungraded)

Почему не рекомендуется применять softmax перед использованием CrossEntropyLoss?

☐ Потому что softmax замедляет обучение

☐ Потому что softmax не работает с батчами

☒ Потому что CrossEntropyLoss уже включает softmax внутри

☐ Потому что softmax нельзя применять к логитам

✓ Show answer

Отправить

✓ Верных

Итоги урока

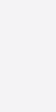
В этом уроке мы познакомились с компонентами, необходимыми для построения полноценных нейросетей.

Функции активации добавляют нелинейность, без которой нейросеть оставалась бы линейной моделью. Мы разобрали ReLU (самый распространённый выбор для скрытых слоёв), Sigmoid (для получения вероятностей), Tanh (для диапазона от -1 до 1) и Softmax (для многоклассовой классификации).

Функции потерь позволяют оценить качество предсказаний модели. Мы научились выбирать подходящую функцию под задачу: MSELoss для регрессии, BCEWithLogitsLoss для бинарной классификации, CrossEntropyLoss для многоклассовой классификации.

Регуляризация помогает бороться с переобучением. Мы разобрали Dropout, который случайно отключает нейроны во время обучения, не позволяя модели чрезмерно полагаться на отдельные признаки.

Вспомогательные слои упрощают преобразование данных между слоями. Мы узнали про Flatten, который превращает многомерные тензоры (например, изображения) в одномерные векторы для подачи в полносвязные слои.



7/12 **Загрузка данных: Dataset и DataLoader**

текущий урок

До этого урока мы работали с данными, которые сами генерировали или загрузили целиком в память. В «Практике по линейной регрессии» мы создавали синтетические данные для линейной регрессии. В уроках «Построение моделей: torch.nn.Module» и «Функции активации, функции потерь и регуляризация» использовали те же данные или небольшие случайные тензоры. В реальных проектах так не получится.

Представьте, что вы тренируете нейросеть на датасете из миллиона изображений и каждое весит несколько мегабайт. Загрузить всё сразу в оперативную память невозможно, нужно подавать данные частями, порциями. На каждой эпохе обучения эти порции нужно перемешивать, чтобы модель не запоминала порядок примеров. Кроме того, изображения нужно предварительно преобразовывать: нормализовать, приводить к единому размеру и так далее.

В PyTorch за всё это отвечают два класса: ***Dataset*** и ***DataLoader***.

В этом уроке мы разберёмся:

- ▶ что такое ***Dataset*** и как создать собственную реализацию;
- ▶ как ***DataLoader*** превращает ***Dataset*** в удобные батчи для обучения;
- ▶ какие готовые датасеты уже есть в PyTorch;
- ▶ как работают трансформации и зачем они нужны.

Ссылки на документацию:

- ▶ [Dataset](#)
- ▶ [DataLoader](#)
- ▶ [datasets](#)
- ▶ [transforms](#)

Dataset: что это и зачем

В PyTorch за доступ к данным отвечает класс ***Dataset***. Это абстракция, которая описывает, где лежат данные и как их получить по индексу. Любой наследник ***Dataset*** должен реализовать два метода:

- ▶ $_len_ (self)$ — возвращает количество примеров в датасете;
- ▶ $_getitem_ (self, idx)$ — возвращает пример по индексу (обычно пару: входные данные и целевая метка).

Простейший пример

Допустим, данные уже загружены в память (как в практике по линейной регрессии):

```
import torch
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, data, targets):
        self.data = data
        self.targets = targets

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.targets[idx]
```

Теперь можно создать датасет и обращаться к его элементам:

```
# синтетические данные (как в Практике по линейной регрессии)
x = torch.linspace(0, 10, 100).reshape(-1, 1)
y = 2 * x + 1 + torch.randn_like(x) * 2

dataset = MyDataset(x, y)
print(len(dataset)) # 100
print(dataset[0]) # (tensor([0.]), tensor([1.3060]))
```

Казалось бы, можно просто работать с тензорами x и y напрямую. Но Dataset даёт значимое преимущество. Неважно, как именно хранятся данные: они могут лежать в памяти, на диске, в облаке, генерироваться на лету; для того, кто использует датасет, это скрыто.

В следующих блоках увидим, как Dataset подключается к DataLoader и как это упрощает жизнь.

DataLoader: работа с батчами

Dataset умеет отдавать примеры по одному, но для обучения нужно отдавать их батчами (группами) и перемешивать каждую эпоху. Эту работу берёт на себя DataLoader. DataLoader принимает Dataset и превращает его в *итератор*, который выдаёт батчи:

```
from torch.utils.data import DataLoader

dataloader = DataLoader(
    dataset,          # наш Dataset
    batch_size=32,    # размер батча
    shuffle=True,      # перемешивать каждую эпоху
    num_workers=2      # сколько процессов на загрузку
)

for batch_data, batch_targets in dataloader:
    print(batch_data.shape) # (32, ..., ...)
    print(batch_targets.shape) # (32, ...)
    break
```

Основные параметры

Параметр	Что делает
batch_size	Размер одного батча (сколько примеров за раз)
shuffle	Перемешивать ли данные перед каждой эпохой
num_workers	Сколько процессов загружают данные параллельно
drop_last	Отбрасывать ли последний неполный батч

Почему shuffle важен

Без перемешивания модель видит примеры в одном и том же порядке каждую эпоху, и это может привести к тому, что она начнёт подстраиваться под порядок следования, а не под реальные закономерности в данных. При *shuffle ≤ True* DataLoader формирует новую случайную последовательность индексов перед каждой эпохой.

Зачем нужен num_workers

Загрузка данных может быть медленной, особенно при чтении с диска или применении тяжёлых трансформаций. Несколько рабочих процессов позволяют загружать следующий батч параллельно с обучением на текущем. Типичные значения — 2, 4, 8. При *num_workers = 0* всё выполняется в основном процессе.

Примечание для Windows и Jupyter Notebook: при *num_workers > 0* код создания DataLoader и цикл обучения необходимо оборачивать в *if __name__ == '__main__':*, иначе возникнет ошибка при порождении дочерних процессов.

Что получается на выходе? DataLoader автоматически собирает отдельные примеры в тензоры, объединяя их по нулевому измерению:

```
for images, labels in dataloader:
    print(images.shape) # torch.Size([32, 1, 28, 28]) — 32 картинки
    print(labels.shape) # torch.Size([32]) — 32 метки
    break
```

Полученные тензоры можно сразу отправлять на GPU и передавать в модель.

Встроенные датасеты torchvision

Писать свой Dataset для каждого датасета утомительно. Хорошая новость: для самых популярных датасетов в PyTorch уже есть готовые классы. Модуль *torchvision.datasets* содержит множество готовых датасетов:

```
from torchvision import datasets
```

Примеры для работы с изображениями:

- ▶ MNIST, FashionMNIST — рукописные цифры и предметы одежды (28 × 28, ч/б);
- ▶ CIFAR10, CIFAR100 — маленькие цветные картинки (32 × 32);
- ▶ ImageNet — большой датасет (миллионы изображений).

При этом данные скачиваются автоматически при первом использовании.

Возьмём знакомый вам по Модулю 2 FashionMNIST:

```
from torchvision import datasets
from torchvision import transforms

train_dataset = datasets.FashionMNIST(
    root='./data',
    train=True,
    download=True,
    transform=transforms.ToTensor()
)

test_dataset = datasets.FashionMNIST(
    root='./data',
    train=False,
    download=True,
    transform=transforms.ToTensor()
)

print(len(train_dataset)) # 60000
print(len(test_dataset)) # 10000
```

Каждый элемент датасета — пара (изображение, метка):

```
image, label = train_dataset[0]
print(image.shape) # torch.Size([1, 28, 28]) — ч/б картинка
print(label) # 9 (индекс класса: 9 — "Ankle boot")
```

Метка — число от 0 до 9. Расшифровка классов есть в документации, но для обучения модели она не требуется.

Всё работает точно так же, как с самодельным Dataset. Теперь можно подключать DataLoader:

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=2
)

for images, labels in train_loader:
    print(images.shape) # torch.Size([64, 1, 28, 28])
    print(labels.shape) # torch.Size([64])
    break
```

Зачем использовать встроенные датасеты

- ▶ Экономия времени на подготовку данных. Не приходится вручную скачивать файлы, распаковывать архивы и писать парсеры для разбора форматов: всё это уже сделано за вас.
- ▶ Корректность разбиения. Разделение на обучающую и тестовую выборки выполнено заранее и гарантированно правильно, нет риска случайно допустить утечку данных между выборками.
- ▶ Гибкость экспериментов. Переключение между датасетами сводится к замене одной строки кода, что позволяет быстро сравнивать поведение модели на разных данных.
- ▶ Универсальный инструмент для отладки. Даже если в реальном проекте вы работаете с собственными данными, встроенные датасеты остаются незаменимыми, на них удобно проверять гипотезы, тестировать новые архитектуры и убеждаться, что пайплайн работает корректно, прежде чем запускать обучение на полном объёме данных.

Трансформации (transforms)

Данные редко подаются в сеть в том виде, в котором они хранятся. Например, изображения нужно превратить в тензоры, отмасштабировать значения пикселей, привести к нужному размеру. В PyTorch для этого существует модуль *tran or ms*.

Трансформации представляют собой преобразования, которые применяются к данным в момент загрузки. Каждый раз, когда *DataLoader* запрашивает элемент из *Dataset*, к нему автоматически применяется заданная цепочка преобразований.

Композиция трансформаций

Несколько преобразований можно объединить в последовательную цепочку с помощью *Compose*:

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

Эту цепочку можно передать в датасет:

```
dataset = datasets.FashionMNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform # трансформации применяются автоматически
)
```

Основные трансформации

ToTensor выполняет три действия:

1. превращает PIL-изображение или numpy-массив в тензор;
2. меняет порядок размерностей на (C, H, W) ;
3. переводит значения пикселей из диапазона $[0, 255]$ в $[0.0, 1.0]$.

```
tensor_image = transforms.ToTensor()(pil_image)
```

Normalize приводит значения тензора к заданному распределению по формуле $output = \frac{p - \mu}{s} \cdot \sigma$:

```
# для одноканальных (чёрно-белых) изображений
normalize = transforms.Normalize((0.5,), (0.5,))

# для трёхканальных (цветных) изображений
normalize_rgb = transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
```

Числа $[0.485, 0.456, 0.406]$ и $[0.229, 0.224, 0.225]$ представляют собой среднее и стандартное отклонение, вычисленные на датасете ImageNet. Эти значения стали стандартом для большинства предобученных моделей.

Resize изменяет пространственный размер изображения:

```
resize = transforms.Resize((224, 224))
```

Порядок трансформаций

Порядок преобразований в цепочке имеет значение. Сначала необходимо вызвать *ToTens or* (конвертация в тензор), затем *N or malize* (нормализация). При нарушении этого порядка возникнет ошибка, поскольку *N or malize* принимает на вход только тензоры, но не PIL-изображения.

```
# правильный порядок
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# ошибочный порядок
transform = transforms.Compose([
    transforms.Normalize((0.5,), (0.5,)), # ошибка: на вход поступит PIL-изображение
    transforms.ToTensor()
])
```

Зачем нужна нормализация

Нейронные сети обучаются быстрее и стабильнее, когда входные данные имеют среднее около нуля и стандартное отклонение около единицы.

После *ToTens or* значения пикселей лежат в диапазоне $[0, 1]$. Нормализация с параметрами *mean = 0.5* и *std = 0.5* переводит их в диапазон $[-1, 1]$.

Вопрос 7.1

0 points possible (ungraded)

Какой класс в PyTorch отвечает за формирование батчей и итерацию по данным?

☐ Dataset

☒ DataLoader

☐ Transform

☐ TensorDataset

Show answer

Отправить

✓ Верных

Вопрос 7.2

0 points possible (ungraded)

Зачем параметр shuffle=True в DataLoader?

☐ Чтобы ускорить загрузку данных

☐ Чтобы уменьшить размер батча

☒ Чтобы перемешивать порядок примеров и избежать зависимости модели от порядка данных

☐ Чтобы автоматически нормализовать данные

Show answer

Отправить

✓ Верных

Итоги урока

В этом уроке мы разобрались, как в PyTorch организована работа с данными.

- ▶ Dataset — это абстракция, которая предоставляет доступ к данным. Мы научились создавать собственные датасеты, реализуя методы $_len_$ и $_getitem_$.
- ▶ DataLoader принимает Dataset и превращает его в итератор по батчам. Он умеет перемешивать данные, загружать их параллельно в несколько потоков и отдавать готовые тензоры.
- ▶ Встроенные датасеты в torchvision избавляют от необходимости вручную писать код для скачивания и подготовки популярных коллекций изображений. FashionMNIST, CIFAR, MNIST — всё доступно в одной строке.
- ▶ Трансформации позволяют преобразовывать данные на лету. Основные из них: *ToTens or* (превращает картинку в тензор) и *N or malize* (нормализует значения пикселей на нужный диапазон)

< Предыдущий урок Следующий урок >



8/12 Оптимизаторы и сохранение моделей

текущий урок

В уроках «Практика по линейной регрессии» и «Построение моделей: torch.nn.Module» мы обновляли веса руками. Сначала писали $w \equiv lr \cdot w$. ∇ для каждого параметра по отдельности, затем перешли к циклу по `model.parameters()`. Такой подход даёт понимание того, что происходит под капотом, но в реальных проектах так никто не делает. Писать цикл обновления каждый раз неудобно, к тому же обычный градиентный спуск далеко не лучший алгоритм оптимизации. Есть методы, которые накапливают информацию о предыдущих градиентах, подбирают свой шаг для каждого параметра и за счёт этого сходятся значительно быстрее.

В PyTorch все эти алгоритмы собраны в модуле → *rch.optim*. Они называются оптимизаторами. Достаточно выбрать подходящий, передать ему параметры модели, и он сам будет обновлять веса, обнулять градиенты и применять нужные приёмы.

Вторая ключевая тема урока — сохранение моделей. В уроке «Построение моделей: torch.nn.Module» мы лишь упомянули *state_dict*, но в реальных проектах нужно уметь сохранять не только финальную модель, но и промежуточные чекпоинты, чтобы можно было продолжить обучение после остановки.

В этом уроке:

- разберёмся с оптимизаторами: SGD, Adam,
- научимся сохранять и загружать модели,
- освоим чекпоинты и сохранение лучшей модели по метрике.

Что такое оптимизатор

В уроке «Построение моделей: torch.nn.Module» мы обновляли веса вручную:

```
with torch.no_grad():
    for param in model.parameters():
        param -= learning_rate * param.grad
```

Такой подход работает, но имеет ряд ограничений:

- один и тот же цикл приходится воспроизводить снова и снова, что загромождает код;
- здесь реализован простейший градиентный спуск без каких-либо приёмов ускорения сходимости;
- обнуление градиентов выполняется отдельным вызовом (`model.zero_grad()`), о котором легко забыть.

Что делает оптимизатор? Оптимизатор — это объект, который полностью отвечает за обновление параметров модели. Этот код реализует простейший оптимизатор, стохастический градиентный спуск (SGD):

```
import torch.optim as optim

# создаём оптимизатор
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

С ним цикл обучения принимает более компактный вид:

```
for inputs, targets in dataloader:
    optimizer.zero_grad()      # 1. обнуляем градиенты
    outputs = model(inputs)    # 2. прямой проход
    loss = criterion(outputs, targets)
    loss.backward()            # 3. вычисляем градиенты
    optimizer.step()           # 4. обновляем веса
```

Теперь вместо ручного цикла по всем параметрам достаточно вызвать *optimizer.step()*, а обнуление градиентов выполняется через *optimizer.zero_grad()*. В результате код становится компактнее и проще для восприятия.

PyTorch предоставляет несколько алгоритмов обновления весов:

- SGD — классический градиентный спуск с удобным интерфейсом;
- SGD с momentum — градиентный спуск с добавлением инерции;
- Adam — метод с адаптивным шагом для каждого параметра отдельно;
- RMSprop — ещё один метод с адаптивным шагом.

Обычно для большинства задач начинают с Adam. Если нужно больше контроля, пробуют SGD. Подробное сравнение и теорию оставим для дисциплины «Продвинутые методы».

Все оптимизаторы используют одинаковый интерфейс: *zero_grad()* и *step()*. Метод *backward()* вызывается на объекте loss, а не на оптимизаторе, но эти три вызова всегда идут вместе.

Сохранение модели

В уроке «Построение моделей: torch.nn.Module» мы только упомянули *state_dict*. Теперь разберём сохранение подробно, потому что это понадобится в каждом проекте. Правильный способ — сохранять только параметры:

```
# сохраняем
torch.save(model.state_dict(), 'model.pth')

# загружаем
model = GenreClassifier() # сначала создаём модель с той же архитектурой
model.load_state_dict(torch.load('model.pth'))
```

Сохранение только весов модели через `state_dict` считается правильным подходом по нескольким причинам. Файл с весами занимает значительно меньше места на диске, потому что в нём хранятся только числовые значения параметров без всей обвязки в виде структуры классов, метаданных и внутренних объектов PyTorch. Кроме того, такой файл не привязан к конкретной версии PyTorch, потому что это, по сути, обычный словарь с тензорами, а формат тензоров стабилен и не меняется между версиями. Если же сохранять модель целиком через `pickle`, то внутренние структуры PyTorch могут измениться в новой версии и модель просто не загрузится. Наконец, файл с весами легко переносится между проектами, потому что для загрузки достаточно иметь определение архитектуры модели в коде и вызвать `load_state_dict`. Не нужно тащить за собой конкретные модули, пути импорта и версии библиотек, которые были на момент сохранения. Мы получаем универсальный и надёжный способ хранения обученных моделей.

Нежелательный (но возможный) способ — сохранять всю модель целиком:

```
torch.save(model, 'model_full.pth')
model = torch.load('model_full.pth')
```

Так делать не рекомендуется: как поясняется в тексте выше, могут возникнуть проблемы при смене версии PyTorch или при переносе на другую машину.

Чекпоинты

Когда модель обучается долго, полезно сохранять не только финальные веса, но и промежуточное состояние. Такие промежуточные сохранения называются чекпоинтами. Что именно можно сохранять в чекпоинте:

```
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': loss,
    # можно добавить любую другую информацию
}
```

```
torch.save(checkpoint, 'checkpoint.pth')
```

Для того чтобы загрузить чекпоинт:

```
checkpoint = torch.load('checkpoint.pth')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
start_epoch = checkpoint['epoch'] + 1
```

Теперь можно продолжить обучение с того же места. Сохранение чекпоинтов во время обучения нужно в первую очередь для защиты от потери прогресса. Если обучение прервалось из-за сбоя питания, нехватки памяти или любой другой ошибки, не придётся начинать весь процесс с нуля — можно просто загрузить последний сохранённый чекпоинт и продолжить с того места, где остановились. Это особенно критично, когда обучение длится часами или даже днями. Также чекпоинты позволяют откатиться к лучшей версии модели, если на каком-то этапе она начала переобучаться. Например, на 50-й эпохе качество на валидации было максимальным, а дальше модель стала запоминать обучающую выборку и терять способность к обобщению. Имея сохранённый чекпоинт 50-й эпохи, можно просто вернуться к нему и использовать именно эту версию как финальную. Помимо этого, чекпоинты очень удобны для экспериментов, потому что можно сохранять промежуточные версии модели с разными гиперпараметрами и на разных этапах обучения, а потом спокойно сравнивать их между собой, выбирая наилучший вариант без необходимости заново всё пересчитывать.

Сохранение лучшей модели по метрике

В реальных проектах нужна не модель последней эпохи, а та, что показала лучший результат на валидации.

```
best_accuracy = 0.0

for epoch in range(epochs):
    # обучаем модель
    for inputs, targets in train_loader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

    # оцениваем на валидации
    accuracy = evaluate(model, val_loader) # эту функцию напомним в уроке по Валидации и режимам обучения

    # если результат лучше предыдущего, сохраняем
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        torch.save(model.state_dict(), 'best_model.pth')
        print(f"Сохранена лучшая модель! Accuracy: {accuracy:.4f}")
```

После обучения загружаем лучшую модель:

```
model.load_state_dict(torch.load('best_model.pth'))
# переводим в режим оценки (подробнее в уроке по Валидации и режимам обучения)
model.eval()
```

Вопрос 8.1

0 points possible (ungraded)

Какой метод оптимизатора в PyTorch отвечает за обновление параметров модели?

☐ optimizer.backward()

☐ optimizer.zero_grad()

☒ optimizer.step()

☐ optimizer.update()



Show answer



Верных

Вопрос 8.2

0 points possible (ungraded)

Зачем сохранять чекпоинты во время обучения модели?

☐ Чтобы ускорить обучение модели

☐ Чтобы уменьшить размер модели

☒ Чтобы можно было продолжить обучение и сохранить лучший результат

☐ Чтобы автоматически подбирать гиперпараметры



Show answer



Верных

Итоги урока

В этом уроке мы разобрали три важные практические темы, которые превращают учебный код в код, пригодный для реальной работы.

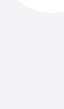
Оптимизаторы заменяют ручное обновление весов. Все они используют единый интерфейс: *zero_grad()* для обнуления градиентов и *step()* для обновления параметров. Детали работы разных оптимизаторов разберём в дисциплине «Продвинутые методы».

Далее мы научились правильно сохранять и загружать модели. Ключевой инструмент здесь это `state_dict`, который содержит только параметры модели без лишнего багажа. Для загрузки нужно сначала создать модель с той же архитектурой и затем подгрузить в неё сохранённые веса — самый надёжный и переносимый подход.

Наконец, мы познакомились с чекпоинтами, которые сохраняют не только модель, но и состояние оптимизатора вместе с номером эпохи. Благодаря этому можно продолжить обучение точно с того места, где оно было прервано. А сохранение лучшей модели по метрике на валидации позволяет не потерять оптимальную версию, даже если дальнейшее обучение ухудшает результат.

< Предыдущий урок

Следующий урок >



9/12 Валидация и режимы обучения

текущий урок

Введение

До сих пор мы делали одно и то же: обучали модели, строили график функции потерь и наблюдали, как значение снижается от эпохи к эпохе. Но возникает важный вопрос: модель действительно научилась решать задачу или всего лишь запомнила тренировочные примеры?

Чтобы вовремя заметить переобучение, недостаточно отслеживать ошибки только на обучающей выборке. Нужен отдельный валидационный набор, данные, которых модель никогда не видела в процессе обучения.

В этом уроке мы разберёмся:

- как разделить данные на `train`, `val` и `test`;
- чем отличаются режимы `train()` и `eval()` и почему это важно;
- зачем отключать градиенты при валидации с помощью `torch.no_grad()`;
- какие метрики использовать для оценки качества;
- как остановить обучение вовремя, не дожидаясь переобучения.

Всё это про то, как сделать обучение осознанным, а не просто запустить его и надеяться на лучшее.

Как разделить данные

Прежде чем начать обучение, данные нужно разбить на три части.

- Train (обучение)** — на этих данных модель учится. Обычно это 60–80% от всего объёма.
- Validation (валидация)** — на этих данных мы проверяем модель в процессе обучения, чтобы вовремя заметить переобучение. Обычно 10–20%.
- Test (тест)** — на этих данных оцениваем финальную модель после завершения обучения. Используется только один раз в самом конце. Обычно 10–20%.

Как это сделать в PyTorch:

```
from torch.utils.data import random_split

# предполагаемый датасет
full_dataset = MyDataset(data, targets)

# определить размеры частей
train_size = int(0.7 * len(full_dataset)) # 70%
val_size = int(0.15 * len(full_dataset)) # 15%
test_size = len(full_dataset) - train_size - val_size # остаток ~15%

# разбиваем
train_dataset, val_dataset, test_dataset = random_split(
    full_dataset, [train_size, val_size, test_size]
)
```

Важное правило: тестовый набор не должен использоваться в процессе обучения ни при каких условиях. Ни для подбора гиперпараметров, ни для ранней остановки. Более того, тестовые данные не должны участвовать даже в предобработке. Например, если вы вычисляете среднее и стандартное отклонение для нормализации, делайте это только по обучающей выборке. Иначе информация о тестовых данных косвенно просочится в модель, и итоговая оценка качества окажется завышенной. Возникает проблема утечки данных.

Режимы: `model.train()` и `model.eval()`

Итак, данные разделены. Теперь во время обучения мы будем периодически проверять модель на валидационном наборе. Но есть важная техническая деталь: некоторые слои в PyTorch работают по-разному на этапе обучения и на этапе проверки.

Самые важные из таких слоёв:

- Dropout** — на обучении случайно отключает часть нейронов, чтобы бороться с переобучением; на проверке должны работать все нейроны;
- BatchNorm** — на обучении использует статистику текущего батча, на проверке — статистику, накопленную за всё обучение.

Чтобы модель знала, в каком режиме она находится, используются два метода:

```
model.train() # режим обучения (Dropout работает, BatchNorm считает по батчу)
model.eval()  # режим оценки (Dropout выключен, BatchNorm использует накопленные статистики)
```

Если забыть переключить режим перед валидацией, результаты будут непредсказуемыми: с включённым Dropout проход будет случайно колебаться. А если забыть переключить обратно после валидации, модель продолжит обучение с выключенным Dropout, что изменит поведение тренировки. Типичный паттерн использования:

```
# обучение
model.train()
for inputs, targets in train_loader:
    outputs = model(inputs)
    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()

# валидация
model.eval()
with torch.no_grad():
    for inputs, targets in val_loader:
        outputs = model(inputs)
        # считаем loss и метрики
```

`torch.no_grad()`: отключаем градиенты

При валидации мы не обновляем веса модели, мы только прогоняем данные через сеть, считаем `loss` и метрики. Градиенты в этом случае не нужны.

Для этого в PyTorch есть контекстный менеджер → `torch.no_grad()`:

```
with torch.no_grad():
    outputs = model(inputs)
    loss = criterion(outputs, targets)
    # градиенты не считаются
```

Когда градиенты отключены, PyTorch не хранит их для промежуточных тензоров, что экономит память. Кроме того, прямой проход без построения вычислительного графа выполняется быстрее, а значит, вычисления ускоряются.

Как это выглядит в цикле валидации:

```
model.eval()
val_loss = 0
correct = 0
total = 0

with torch.no_grad():
    for inputs, targets in val_loader:
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        val_loss += loss.item()

        # считаем accuracy
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += (predicted == targets).sum().item()

val_loss /= len(val_loader)
accuracy = correct / total
```

Обратите внимание: здесь нет `optimizer.zero_()`, нет `loss.backward()`, нет `optimizer.step()`. Только прямой проход и подсчёт метрик.

Метрики качества

`Loss` показывает, насколько хорошо модель оптимизируется. Но для человека это число малоинформативно. Что значит `loss = 0.237`? Это хорошо или плохо? Зависит от задачи.

Поэтому помимо `loss` используют метрики — понятные числа, которые отражают качество модели с человеческой точки зрения.

Ассигасу — главная метрика для классификации

Ассигасу — это доля правильных ответов.

```
correct = 0
total = 0

for inputs, targets in val_loader:
    outputs = model(inputs)
    _, predicted = outputs.max(1) # берём класс с максимальной вероятностью
    total += targets.size(0)
    correct += (predicted == targets).sum().item()

accuracy = correct / total
print(f"Accuracy: (accuracy: .4f)" # например, 0.85 означает 85% правильных ответов
Когда смотреть loss, а когда ассигасу
```

Ситуация	Что это значит
Loss падает, ассигасу растёт	Всё в порядке, модель учится
Loss падает, ассигасу не растёт	Модель становится увереннее в своих ответах, но ошибается в тех же местах
Loss растёт, ассигасу падает	Модель переобучается или что-то сломалось в обучении

Другие метрики классификации

Ассигасу хорошо работает, когда классы сбалансированы. Но если в данных 95% одного класса и 5% другого, модель может просто всегда предсказывать большинство и получить ассигасу 95%. В таких случаях используют другие метрики.

Precision показывает, сколько действительно положительных из всех предсказанных положительных. Важна, когда цена ложного срабатывания высока (например, спам-фильтр: не хочется отправить нужное письмо в спам).

Recall показывает, сколько единиц из всех реально положительных нашла модель. Важна, когда нельзя пропустить положительный пример (например, диагностика болезни: лучше перестраховаться, чем пропустить).

F1-score — среднее гармоническое `precision` и `recall`. Используется, когда нужен баланс между ними.

В этой дисциплине мы ограничимся ассигасу, но полезно знать, что эти метрики существуют.

Ранняя остановка (early stopping)

Мы научились следить за метрикой на валидации. Но что делать, когда она перестала улучшаться? Продолжить обучение — значит рисковать переобучением. Прекратить — значит упустить возможное улучшение через пару эпох.

Ранняя остановка — это техника, которая автоматически прекращает обучение, когда метрика на валидации перестала улучшаться.

Нужны три переменные:

- `best_val_loss`** — лучшее значение метрики за всё время обучения;
- `patience`** — сколько эпох ждать улучшения;
- `patience_counter`** — сколько эпох подряд улучшения не было.

```
best_val_loss = float('inf')
patience = 5
patience_counter = 0

for epoch in range(epochs):
    # обучаем...
    val_loss = validate(model, val_loader)

    if val_loss < best_val_loss:
        # улучшение! сохраняем модель и сбрасываем счётчик
        best_val_loss = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), 'best_model.pth')
    else:
        # улучшения нет — увеличиваем счётчик
        patience_counter += 1

        if patience_counter >= patience:
            print(f"Ранняя остановка на эпохе {epoch}")
            break
```

Порог улучшения (`min_delta`)

Иногда `loss` снижается на ничтожно малую величину, например на 0.0001. Формально это улучшение, но практической пользы нет. Параметр `min_delta` задаёт минимальный порог, ниже которого улучшение не считается значимым:

```
min_delta = 1e-3

if val_loss < best_val_loss - min_delta:
    # значимое улучшение
    best_val_loss = val_loss
    patience_counter = 0
    torch.save(model.state_dict(), 'best_model.pth')
else:
    patience_counter += 1
```

Чаще всего отслеживают **`val_loss`**, но можно следить за любой метрикой. Если мониторим ассигасу, логика меняется на противоположную: ищем максимум, а не минимум:

```
if val_accuracy > best_val_accuracy + min_delta:
    best_val_accuracy = val_accuracy
    patience_counter = 0
```

Ранняя остановка нужна прежде всего для экономии времени, потому что обучение не продолжается бесконечно, когда результат уже не улучшается. Кроме того, она помогает бороться с переобучением, поскольку фиксируется до того, как начнёт загромождаться шум в данных. Наконец, это способ автоматизации процесса: не нужно вручную смотреть на графики и решать, когда пора остановиться.

Вопрос 9.1

0 points possible (ungraded)

Какой метод переводит модель в режим оценки (валидации) в PyTorch?

☐ `model.test()`

☐ `model.validate()`

☒ `model.eval()`

☐ `model.freeze()`

Show answer

Отправить

✓ Верных

Вопрос 9.2

0 points possible (ungraded)

Зачем использовать `torch.no_grad()` во время валидации?

☐ Чтобы ускорить обучение модели

☒ Чтобы не считать градиенты, снизить потребление памяти и ускорить вычисления

☐ Чтобы автоматически обновить веса модели

☐ Чтобы улучшить точность модели

Show answer

Отправить

✓ Верных

Вопрос 9.3

0 points possible (ungraded)

Почему нельзя использовать тестовую выборку при обучении модели?

☐ Потому что она слишком маленькая

☐ Потому что она не содержит правильных ответов

☒ Потому что это приводит к утечке данных и завышенной оценке качества

☐ Потому что она используется только для ускорения обучения

Show answer

Отправить

✓ Верных

Итоги урока

В этом уроке мы научились правильно оценивать модель во время обучения и вовремя замечать переобучение.

Разделение данных. Данные делятся на три части: `train` (обучение), `validation` (настройка гиперпараметров и ранняя остановка), `test` (финальная оценка). Тестовый набор не участвует в процессе обучения и используется только один раз в самом конце.

Режимы модели. `model.train()` переключает модель в режим обучения: Dropout работает, BatchNorm использует статистику батча; `model.eval()` переключает в режим оценки: Dropout выключен, BatchNorm использует накопленные статистики. Если забыть переключить режим, результат будет непредсказуемым.

Отключение градиентов. Блок `with torch.no_grad()`: при валидации экономит память и ускоряет вычисления, потому что градиенты на этом этапе не нужны.

Метрики. Помимо `loss`, важно отслеживать понятные метрики. Для классификации основная из них — это ассигасу (доля правильных ответов): она помогает понять, улучшается ли модель с точки зрения реальной задачи, а не только с точки зрения функции потерь.

Ранняя остановка. Техника, которая автоматически прекращает обучение, когда метрика на валидации перестала улучшаться. Для реализации нужны два параметра: `patience` (сколько эпох ждать без улучшения) и `best_metric` (лучшее значение метрики, зафиксированное за всё время обучения). Это позволяет сэкономить время, предотвратить переобучение, а также сохранить веса модели, соответствующие лучшему результату на валидации.

< Предыдущий урок

Следующий урок >

Введение

В предыдущих уроках мы изучили все необходимые компоненты: создание тензоров (урок «Введение в PyTorch и тензоры»), управление устройствами (урок «Работа с устройствами (CPU/GPU)'), автоматическое дифференцирование (урок «Autograd: автоматическое дифференцирование»), построение моделей через nn.Module (урок «Построение моделей: torch.nn.Module»), функции активации и потерь (урок «Функции активации, функции потерь и регуляризация»), загрузку данных через Dataset и DataLoader (урок «Загрузка данных: Dataset и DataLoader»), настройку оптимизаторов (урок «Оптимизаторы и сохранение моделей») и валидацию (урок «Валидация и режимы обучения»).

Теперь пришло время собрать всё вместе и решить реальную задачу: классификацию музыкальных жанров.

В этом уроке мы:

- познакомимся с датасетом GTZAN, классическим набором для распознавания музыкальных жанров;
- узнаем, что такое спектрограмма и как она позволяет «увидеть» звук;
- сравним два подхода: обучение на сырых пикселях спектрограмм и на извлечённых инженерных признаках;
- увидим, почему MLP хорошо работает с табличными данными, но плохо справляется с изображениями;
- сделаем выводы о том, какие архитектуры для каких типов данных подходят.

Знакомство с датасетом

Датасет GTZAN (назван по имени автора, George Tzanetakis) представляет собой классический набор данных для классификации музыкальных жанров, который часто называют «MNIST для звука». Он содержит 1000 аудиотреков длительностью 30 секунд каждый, распределённых по 10 жанрам:

- blues,
- classical,
- country,
- disco,
- hiphop,
- jazz,
- metal,
- pop,
- reggae,
- rock.

В датасете представлены три типа данных, мы будем использовать два из них.

- Изображения спектрограмм — визуальное представление звукового сигнала. Каждый трек преобразован в изображение, где горизонтальная ось соответствует времени, вертикальная — частоте, а цвет отражает интенсивность (яркость). Это позволяет буквально «увидеть» музыку.
- CSV с извлечёнными признаками — 57 числовых характеристик каждого трека: MFCC (мел-частотные кепстральные коэффициенты), спектральные центроиды, темп (BPM) и другие. Это готовое табличное представление, с которым MLP может работать напрямую. Признак ≤ 11 не используется — это техническая информация о длительности файла.

Важно: в датасете 1000 треков, но спектрограмм только 999 (один файл отсутствует). Это типичная ситуация: в реальных проектах данные почти всегда содержат подобные дефекты.

Полностью код практики представлен в [данном ноутбуке](#), в уроке содержится теоретическое обоснование.

Что такое спектрограмма и как её интерпретировать

Прежде чем загружать данные, разберёмся, что мы видим на спектрограмме. Это ключевое понятие для понимания задачи. Аудиофайл представляет собой колебания давления воздуха, записанные как оцифрованный во времени, но для анализа музыки важнее не сама амплитуда, а частотный состав сигнала: высота тона, тембр, ритмические структуры. Спектрограмма показывает, какие частоты присутствуют в звуке и как они меняются со временем.

Оси спектрограммы

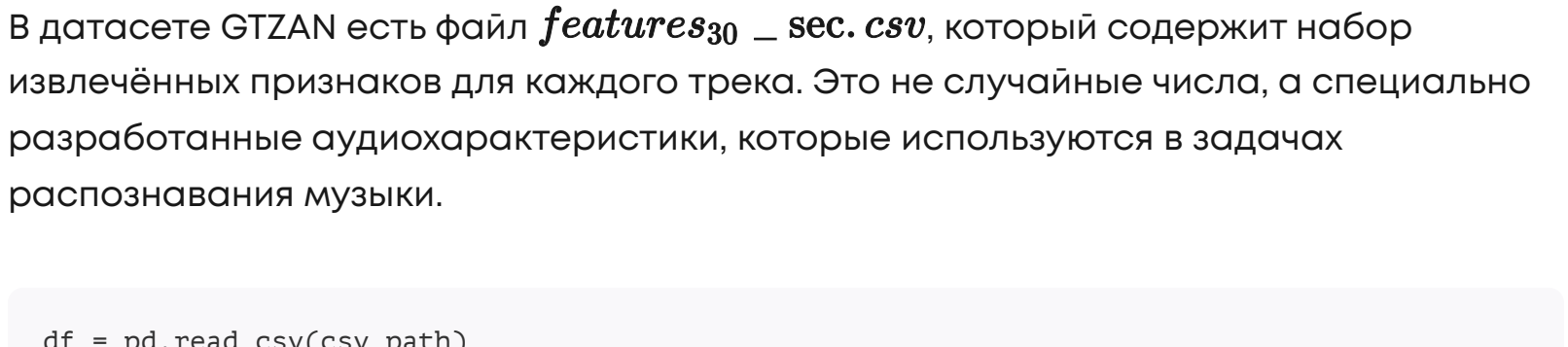
- Горизонтальная ось — время (от начала фрагмента к концу).
- Вертикальная ось — частота (низкие частоты внизу, высокие сверху).
- Цвет/яркость — интенсивность (энергия) на данной частоте в данный момент времени (чем ярче, тем громче).

Примечание. В датасете GTZAN используются мел-спектрограммы. Обычная спектрограмма показывает частоты в линейной шкале (Гц), но человеческое ухо воспринимает звук нелинейно: мы лучше различаем низкие частоты, чем высокие.

Мел-шкала — это способ измерения звука, приближенный к человеческому слуху. Формула перевода: $\text{mel} = 2595 * \log_{10}(1 + f/700)$.

Преимущество для машинного обучения: мел-спектрограмма сжимает информацию о высоких частотах, выделяя те детали, которые действительно важны для восприятия музыки человеком. Поэтому она часто используется в задачах распознавания жанров, речи и музыкальной информации.

Посмотрим на спектрограммы нескольких треков из датасета:



Что можно заметить на спектрограммах

- Классика** — относительно разреженная картина с четкими горизонтальными линиями (обертоны акустических инструментов), динамика сильно меняется от тихих до громких фрагментов.
- Металл** — плотное заполнение широкого диапазона частот, яркие вертикальные всплески на местах ударных, высокая средняя энергия по всему спектру.
- Джаз** — средняя плотность заполнения, хорошо различимы отдельные инструменты (духовые, фортепиано), характерна вариативность ритма.
- Диско** и хип-хоп — выраженные ритмичные повторяющиеся паттерны, четкая структура битов, заметная энергия на низких частотах (бас-бочка, басовая линия).
- Рок** — средняя и высокая плотность, присутствуют как ритмические паттерны ударных, так и протяженные гармонички электрогитар.

Именно по этим визуальным паттернам модель должна научиться определять жанр: она будет выделять локальные и глобальные признаки изображения, подобно тому как мы сами замечаем различия «на глаз».

CSV-признаки: что это за числа

В датасете GTZAN есть файл **features99.csv**, который содержит набор извлечённых признаков для каждого трека. Это не случайные числа, а специально разработанные аудиохарактеристики, которые используются в задачах распознавания музыки.

```
df = pd.read_csv(csv_path)
display(df.info())
print('-> n')
display(df.head())
```

Ключевой момент, который объясняет количество признаков: для каждой аудиохарактеристики в файле хранятся два значения: среднее (mean) и дисперсия (var), вычисленные по всем временным окнам 30-секундного трека. Единственное исключение — tempo, для которого хранится только одно число.

Вот откуда берётся итоговое количество. Восемь скалярных характеристик (chroma_stft, rms, spectral_centroid, spectral_bandwidth, rolloff, zero_crossing_rate, harmony, percent) дают 16 значений (по два на каждую). Двадцать MFCC-коэффициентов дают 40 значений (по два на каждый). Плюс одно значение tempo. Итого 57 числовых признаков.

Что описывают эти характеристики

- MFCC (Mel-Frequency Cepstral Coefficients)** — 20 коэффициентов, каждый со средним и дисперсией, итого 40 значений. Это самые важные признаки для музыкальных и речевых задач. Они описывают форму спектральной огибающей, то есть тембр звука. Первые несколько MFCC отражают общий характер тембра, последующие — более тонкие детали. Среднее показывает типичный тембр трека, а дисперсия — насколько он меняется на протяжении записи.
- Chroma STFT** — среднее и дисперсия хроматического признака. Хроматические признаки показывают распределение энергии по 12 нотам хроматической гаммы. В CSV хранится агрегированное значение, а не все 12 компонент по отдельности. Этот признак помогает различать жанры по гармонической структуре: классическая музыка часто использует сложные аккорды, а поп — простые тональности.
- Spectral centroid** — «центр тяжести» спектра, отражающий субъективную яркость звука у металла и электронной музыки он выше, у классики и джаза — ниже.
- Spectral bandwidth** — ширина спектра. Широкий спектр характерен для шумных, насыщенных записей, узкий — для чистых тональных звуков.
- Spectral rolloff** — частота, ниже которой сосредоточено 85% энергии спектра. Ещё один способ описать форму спектра.
- Zero crossing rate** — показатель того, как часто сигнал пересекает нулевую линию. Высокие значения характерны для перкуссионных и шумных звуков, низкие — для тональных.
- RMS energy** — среднеквадратичная энергия, отражающая громкость сигнала.
- Harmony и percent** — оценки гармонической и перкуссионной составляющих сигнала, полученные при разделении аудио на гармоническую и ударную компоненты.
- Tempo (BPM)** — темп музыки в ударах в минуту. Диско и поп обычно быстрые (120–130 BPM), джаз и блюз — медленнее (60–100 BPM). Для этого признака хранится только одно значение, без дисперсии.

В отличие от сырых пикселей спектрограммы, которые для MLP представляют собой просто набор несвязанных чисел, CSV-признаки уже содержат осмысленную информацию. Они инвариантны к сдвигу во времени: если сместить трек на секунду, средние значения признаков почти не изменятся, они нормализованы по громкости. Каждый признак имеет конкретную интерпретацию: темп, тембр, гармоническую структуру, яркость звука.

Сравнение двух подходов: изображения vs признаки

	Подход А. Спектрограммы	Подход Б. CSV признаки
Что подаём	12 288 пикселей (64 × 64 × 3)	57 числов
Как обрабатывает MLP	Каждый пиксель независимо	Каждый признак имеет смысл
Связь с предидеями	Как если бы в уроке «Практика по линейной регрессии» мы подавали все точки графика	Как в «Практике по линейной регрессии» — осмысленные признаки

Почему CSV признаки работают лучше: они уже содержат высокоуровневую информацию (тембр, темп, гармонию), инвариантны к сдвигу во времени и нормализованы. MLP хорошо работает с табличными данными — мы видели это в «Практике по линейной регрессии».

Почему спектрограммы дают низкий результат: MLP не видит локальных паттернов. Для него соседние пиксели не связаны. Это задача для свёрточных сетей (CNN) — они учитывают пространственную структуру.

Загрузка и синхронизация данных

Рассмотрим, как загрузить данные и синхронизировать их, чтобы каждое изображение и соответствующая строка CSV относились к одному и тому же треку.

Ключевой момент: в CSV имена файлов записаны в формате **blues.00000.wav**, а в папке с изображениями файлы называются **blues00000.png**. Необходимо привести имена к единому формату, чтобы корректно сопоставить записи.

```
filename_to_idx = {}
for i, name in enumerate(df['filename'].values):
    clean_name = name.replace('_', '').replace('.', '')
    filename_to_idx[clean_name] = i
image_data = [] # изображения
image_labels = [] # метки из изображений
csv_labels = [] # метки из CSV
valid_filenames = [] # имена файлов для проверки

# проходим по каталагу в папке
class_names = sorted(df['class_dir'].values)
os.path.isdir(csv_path.join(image_path, 6))
for class_idx, class_name in enumerate(class_names):
    (class_dir, ) = os.path.join(image_path, class_name)
    image_files = sorted(glob.glob(os.path.join(class_dir, '*.png')))

    for img_path in image_files:
        # получаем имя файла без расширения
        img_name = os.path.basename(img_path).replace('.', '').replace('_', '')

        # ищем в CSV
        csv_idx = filename_to_idx.get(img_name)

        if csv_idx is not None:
            # загружаем изображение
            img = Image.open(img_path).convert('RGB')
            image_data.append(img)
            image_labels.append(class_idx)
            csv_labels.append(csv_idx)
            valid_filenames.append(img_name)
```

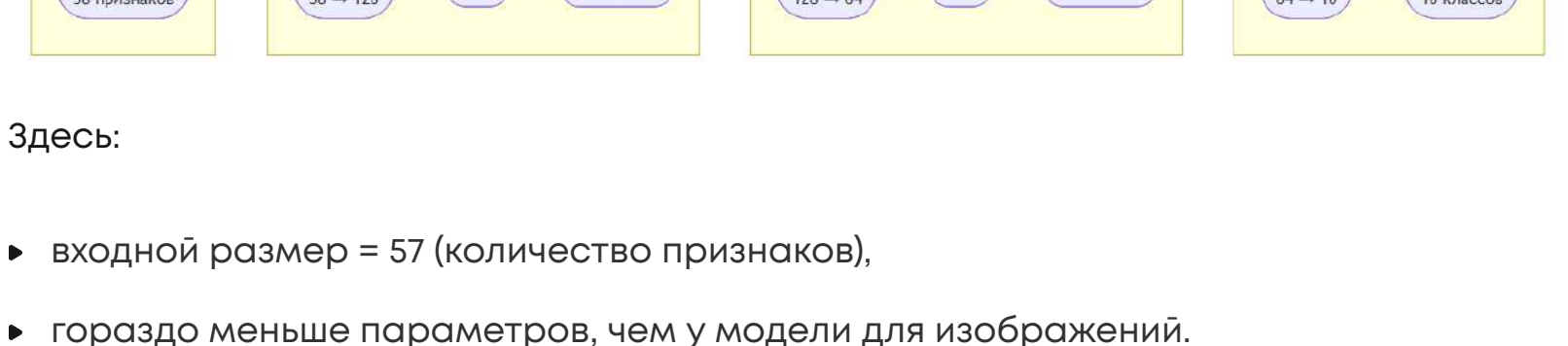
В результате мы получаем 999 синхронизированных образцов (один трек не нашёл пару). Для реальных данных это нормальная ситуация: отдельные файлы могут отсутствовать или быть повреждены. Важно убедиться, что потеря минимальна и не затрагивает целый класс.

Архитектуры моделей

Для спектрограмм (MLPImages):

```
class MLPImages(nn.Module):
    def __init__(self, input_size=64*64*3, hidden_sizes=[312, 256], num_classes=10, dropout_rate=0.1):
        super().__init__()
        self.flatten = nn.Flatten()
        prev_size = input_size
        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(prev_size, hidden_size))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(dropout_rate))
            prev_size = hidden_size
        layers.append(nn.Linear(prev_size, num_classes))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        x = self.flatten(x)
        return self.network(x)
```



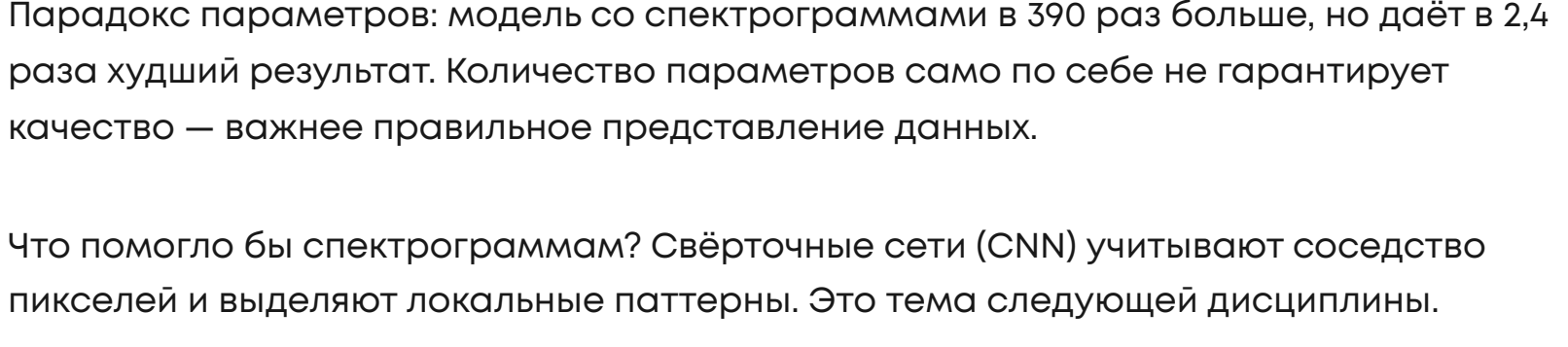
Обратите внимание:

- Flatten превращает 3D-изображение (3 × 64 × 64) в 1D-вектор (12288).
- ReLU — нелинейность (из урока 6).
- Dropout — регуляризация для борьбы с переобучением (из урока 6).

Для CSV признаков (MLPFeatures):

```
class MLPFeatures(nn.Module):
    def __init__(self, input_size=58, hidden_sizes=[128, 64], num_classes=10, dropout_rate=0.1):
        super().__init__()
        prev_size = input_size
        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(prev_size, hidden_size))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(dropout_rate))
            prev_size = hidden_size
        layers.append(nn.Linear(prev_size, num_classes))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

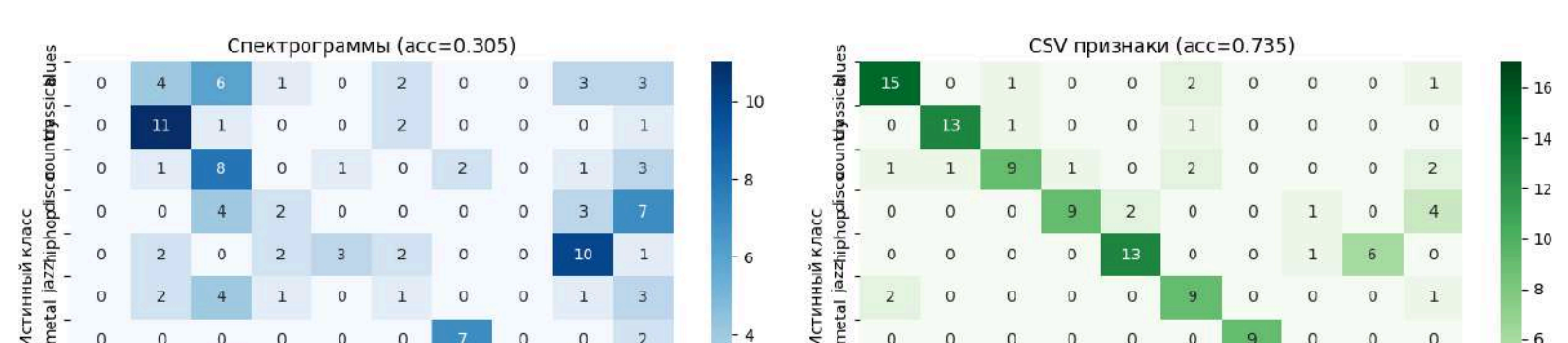


Здесь:

- входной размер = 57 (количество признаков),
- гораздо меньше параметров, чем у модели для изображений.

Обучение и результаты

После обучения мы получили:



Татьяна Булгакова для Skillfactory

Подход	Точность на тесте	Параметры
MLP на спектрограммах	30.46%	6 425 866
MLP на CSV признаках	73.57%	16 458

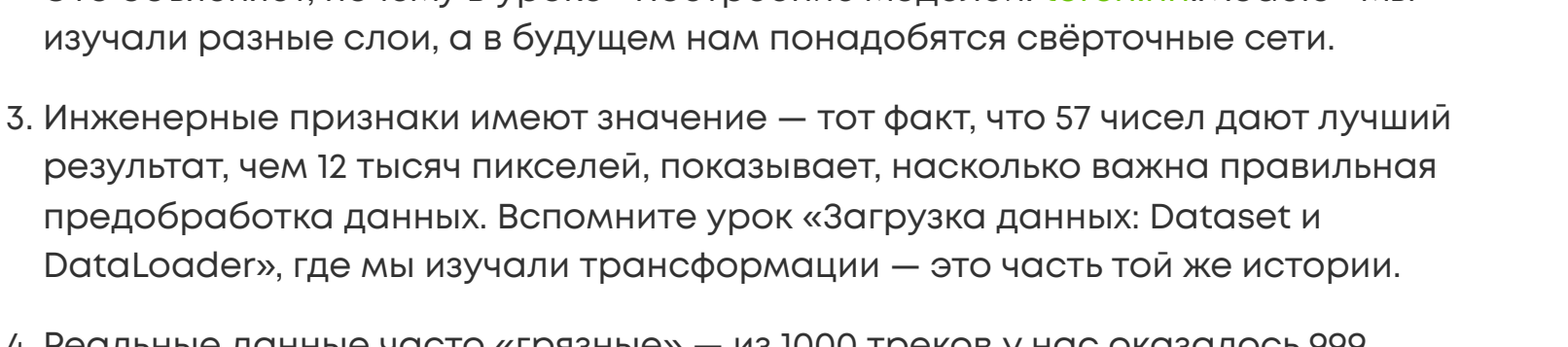
Парадокс параметров: модель со спектрограммами в 390 раз больше, но даёт в 2.4 раза худший результат. Количество параметров само по себе не гарантирует качество — важнее правильное представление данных.

Что помогло бы спектрограммам? Свёрточные сети (CNN) учитывают соседство пикселей и выделяют локальные паттерны. Это тоже следующей дисциплины.

Что мы подтвердили: эксперимент подтвердил наше предположение из блока 4: MLP эффективен для табличных признаков, но не способен извлечь пространственные закономерности из изображений. Для этого нужны специализированные архитектуры.

Матрица ошибок и анализ

Матрица ошибок показывает, какие жанры модель чаще всего путает:



Татьяна Булгакова для Skillfactory

Анализ ошибок

- Jazz и Blues путаются. У них схожие инструменты (духовые, гитара, фортепиано), близкая гармоническая структура и импровизационная манера исполнения.
- Metal и Rock тоже смешиваются. Оба жанра характеризуются высокой плотностью спектра, выраженной ролью электрогитары и быстрым темпом.
- Classical распознаётся хорошо. Этот жанр выделяется разреженным спектром, широким динамическим диапазоном, медленными переходами и отсутствием ударных в привычном смысле.
- Disco и HipHop иногда путаются с Pop. Все три жанра объединяют ритмичные электронные паттерны, четкий бит и схожая структура рэп-припевов.

Эти ошибки закономерны. Даже человеку бывает непросто провести четкую границу между роком и металлом или между джазом и блюзом. Жанры исторически влияли друг на друга, а многие треки находятся на стыке стилей. Точность 73.5% на десяти жанрах на случайном угадывании в 10% говорит о том, что модель уловила содержательные закономерности в данных.

Итоги урока

- MLP хорошо работает с табличными данными — мы это знали ещё из «Практики по линейной регрессии», где линейная регрессия легко справлялась с синтетическими признаками. Здесь мы подтвердили это на реальной задаче: 73.5% точности на CSV-признаках — достойный результат.
- MLP плохо работает с изображениями — и это важный урок. Даже простая задача классификации жанров становится сложной, если подавать пиксели напрямую. Это объясняет, почему в уроке «Построение моделей: torch.nn.Module» мы изучили разные слои, а в будущем нам понадобятся свёрточные сети.
- Инженерные признаки имеют значение — тот факт, что 57 чисел дают лучший результат, чем 12 тысяч пикселей, показывает, насколько важна правильная подготовка данных. Вспомните урок «Загрузка данных: Dataset и DataLoader», где мы изучали трансформации — это часть той же истории.
- Реальные данные часто «грязные» — из 1000 треков у нас оказалось 999 спектрограмм. В уроке «Загрузка данных: Dataset и DataLoader» мы учились работать с Dataset и DataLoader, чтобы гибко обрабатывать такие ситуации.

11/12

Итоги модуля

текущий урок

Мы прошли путь от первого знакомства с PyTorch до решения реальной задачи классификации музыкальных жанров.

Что мы освоили

- Тензоры. Научились создавать, изменять форму и выполнять операции. Тензоры работают аналогично массивам NumPy, но с поддержкой GPU.
- Устройства. Разобрались, как переносить данные на видеокарту и почему все тензоры в одной операции должны находиться на одном устройстве.
- Autograd. Изучили, как PyTorch автоматически строит вычислительный граф и вычисляет градиенты, избавляя нас от ручного дифференцирования.
- Линейную регрессию с нуля. Написали полный цикл обучения, чтобы увидеть внутреннюю механику процесса.
- nn.Module. Освоили построение моделей с помощью готовых слоёв и получили доступ ко всем параметрам через *model.parameters()*.
- Функции активации и потери. Разобрались, как выбирать ReLU, Sigmoid, Softmax и соответствующие функции потерь для различных задач.
- Dataset и DataLoader. Реализовали загрузку данных, трансформации и работу с батчами.
- Оптимизаторы и сохранение. Научились использовать Adam и SGD, сохранять модели через *state_dict* и создавать чекпоинты.
- Валидация. Поняли назначение режимов *train()* и *eval()*, научились отключать градиенты и применять раннюю остановку.
- Практический проект. Применили все знания к задаче классификации музыкальных жанров, сравнили два подхода к представлению данных и сделали выводы.

Главные выводы

PyTorch отличается философией, а не только синтаксисом: больше контроля, больше прозрачности, проще отладка.

Понимание принципов важнее запоминания API. Тот, кто понимает, как работают тензоры, градиенты и вычислительный граф, легко освоит любую новую версию фреймворка.

Далее вы углубитесь в продвинутые архитектуры: свёрточные сети для изображений, рекуррентные сети для последовательностей и трансформеры. А пока вы владеете двумя основными фреймворками (TensorFlow и PyTorch) и можете выбирать инструмент под задачу.

←

Предыдущий урок

Следующий урок

→